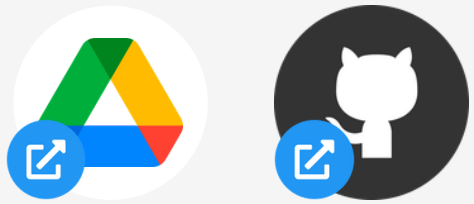


# CoreMerge

---



Technical Documentation

**김주향 Ju Hyang KIM**

---

 jjudykim0@gmail.com  010. 2421. 7343

# 프로젝트 개요

## CoreMerge

Unity 2D를 활용한 액션 플랫폼머 + 아이템 머지 로직의 게임

**프로젝트 기간** 2025.11 - 2025.12 (10일)

**개발 인원** 1인 개발

**주요 기술** State Pattern + FSM, Scriptable Object



본 프로젝트 CoreMerge는 전투에서 획득한 **코어**를 머지(조합) 및 장착하여 전투력을 강화하고, 이를 반복하며 성장하는 전투 - 수집 - 성장 루프의 2D 횡스크롤 액션 게임입니다.

콘텐츠 확장과 밸런스 조정 과정에서도 게임 루프가 안정적으로 유지되도록, 코어의 능력치와 필드 드랍 정보를 **Scriptable Object** 로 분리해 데이터로 관리하는 구조로 구현했습니다.

이를 통해 코어 추가, 수치 조정, 드랍 테이블 변경을 로직 수정없이 진행할 수 있습니다.

또한 플레이어의 연속 액션과 보스의 패턴/페이지 전환처럼 전환 로직이 복잡한 구간은 **상태 패턴 (State Pattern)** 을 적용해 구현하여, 예외 상황에서도 동작이 흔들리지 않도록 제어 흐름을 정리했습니다. 전투 판정 이후의 UI 갱신과 연출 피드백은 이벤트로 연결해 시스템 간 직접 의존도를 줄이고, 수정 범위가 명확한 클라이언트 구조를 목표로 개발했습니다

# 프로젝트 개요



|           |                                                                                                                                                                       |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 장르        | 2D 횡스크롤 액션 플랫폼머                                                                                                                                                       |
| 게임 콘셉트    | 2D 플랫폼머의 조작감과 플레이어 성장 구조를 결합하여, 전투 중 얻은 코어를 직접 합성 · 활용하여 성장하는 액션 게임                                                                                                   |
| 개발 환경     | Unity 6.2, Rider, C#                                                                                                                                                  |
| 핵심 구현 콘텐츠 | <ul style="list-style-type: none"><li>코어 조합을 통한 전략적 성장 시스템</li><li>여러 공격 연계를 활용할 수 있는 2D 액션</li><li>체력 상황에 따른 멀티 페이지 보스 배틀</li><li>스테이지별 저장 및 재도전하는 클리어 콘텐츠</li></ul> |

## 게임 핵심 3 요소

### 핵심 루프

몬스터 처치  
↓  
아이템(코어) 드랍  
↓  
인벤토리에서 동일 등급 코어 머지  
↓  
상급 코어 획득  
↓  
장착 및 전용 스탯 강화  
↓  
보스 페이지 별 대응 전투  
↓  
승리

### 차별점

플레이어의 성장에 개입이 적극적으로 이루어지고, 수치를 시각화함으로써 성장 체감이 즉각적인 게임

### 타겟

깊이 있는 액션 조작과 전략적인 인벤토리 관리를 동시에 선호하는 게이머

## 기술 핵심 3 요소

### 코어 인벤토리 시스템

코어의 이동, 정렬, 머지가 이루어질 수 있는 인벤토리 시스템  
인벤토리 데이터(모델) 와 UI 입력 처리(컨트롤러) 를 분리해 UI 조작의 복잡도와 무관하게 데이터 안정성을 확보

### 상태 패턴 + FSM으로 구현한 캐릭터들

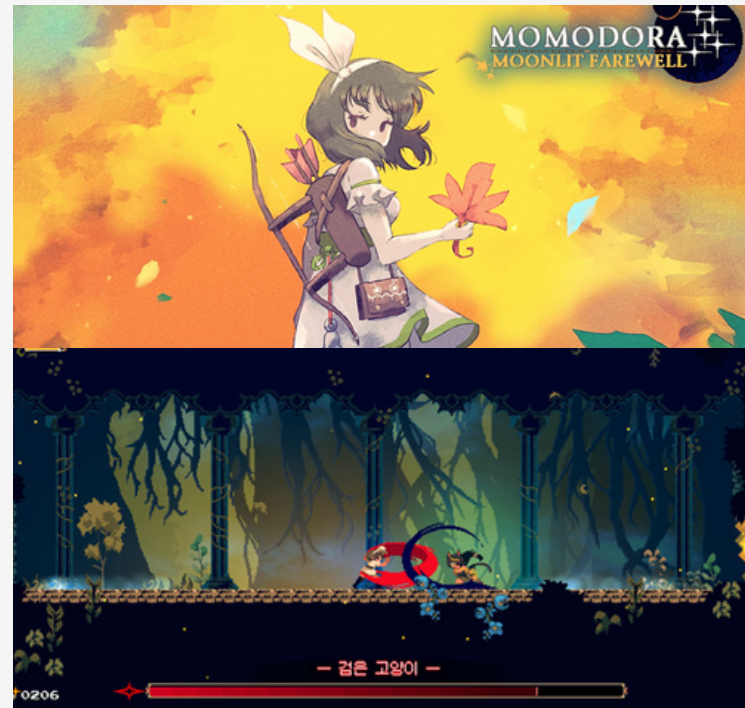
상태 전환이 복잡한 대상(플레이어/보스) → 클래스 단위의 State Pattern을 적용해 상태 전환을 구조적으로 관리  
비교적 단순한 패턴의 몬스터 → 가벼운 상태 관리 방식으로 구현 (FSM)

### 데이터 기반 시스템 설계 (ScriptableObject-Driven)

코어 능력치 / 플레이어 스탯 / 코어 드랍 테이블을 ScriptableObject로 분리해 데이터로 관리  
코드 수정 없이 인스펙터를 통해 밸런스와 콘텐츠 값을 조절 가능

# 프로젝트 소개

## 레퍼런스 모델



## 모모도라 : Moonlit Farewell

### 참조 포인트

#### 정교한 조작감과 빠른 템포

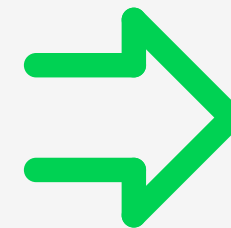
빠른 조작과 이동, 즉각적인 반응성이 강조된 플레이 경험을 제공

#### 명확한 보스 패턴 구조

2~3개의 패턴 페이즈, 공격 텔레그래프를 통한 예측 및 회피

#### 매트로배니아 요소

맵 탐험 기반 아이템/장비 획득으로 능력이 점진 확장



### 반영 포인트

#### 조작감 / 전투 템포 기준

- 여러 이동 / 공격은 짧은 발동 지연 및 빠른 이동 / 종료
- 즉발 및 연속 액션의 감각을 목표로 설계 기준 수립

#### 보스 패턴 구조의 간결함

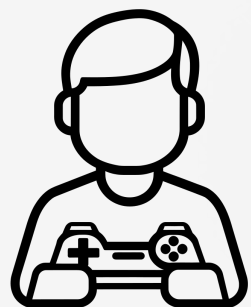
- 페이즈 2~3개 구성을 설계 원칙으로 채택
- 보스전 경험을 관찰 → 학습 → 대응 흐름으로 유도
- 텔레그래프 기반 예측 가능한 회피

#### 매트로배니아의 '점진적 성장' 감각

- 전투 → 코어 획득 → 조합/장착(성장) → 보스전

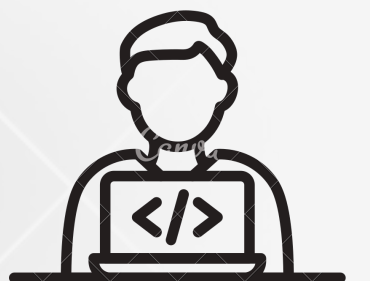
## 핵심 가치

전략적인 코어 머지로 성장 경로를 직접 설계하고, 강화 결과를 수치·연출로 즉각 체감하는 '합성과 성장' 중심 액션 플랫폼



코어를 머지·장착해 성장 결과를 즉시 체감하고, 보스의 페이즈 패턴을 파훼하는 전투를 경험

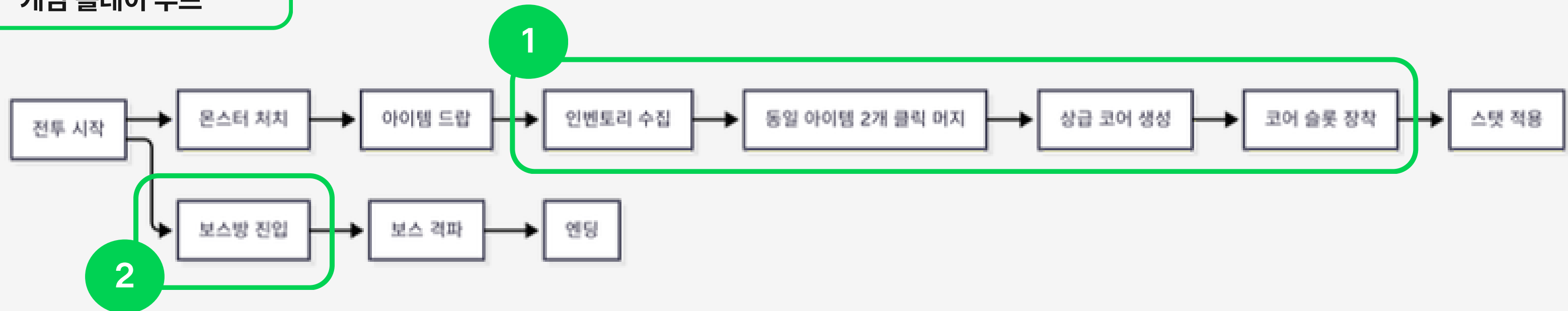
SoC 기반 분리 설계와 상태 패턴으로 복잡한 시스템을 안정적으로 유지보수·확장할 수 있는 클라이언트 아키텍처를 구현





# 프로젝트 소개

## 게임 플레이 루프



## 핵심 재미 요소

1

**안정적인 진행** : 여러 코어가 주는 커버리지(범위·빈도·안정성)가 유리  
저등급 코어를 여러개 유지할까?

VS

**보스/단일 타겟 집중** : 상급 코어의 집중 화력(한방·DPS·효율)이 유리  
머지로 상급 코어를 만들어 강력한 한 방을 노릴까?

2

**안정적인 클리어** : 충분한 파밍을 통해 안정적인 보스전 진입  
스탯을 조금 더 업그레이드한 후에 도전할까?

VS

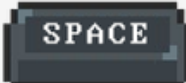
**하드코어 도전** : 낮은 스탯으로 진입해 패턴 대응 난이도와 긴장감 상승  
지금까지 모은 코어로 즉시 보스전에 진입할 것인가?

유저의 플레이 스타일에 맞추어 선택할 수 있는 전략적 요소로 **코어 시스템**을 적용

# 프로젝트 소개

## 플레이어 조작 방법 및 UI/UX

### 기본 이동 조작

|                                                                                                                                                                     |           |                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|---------------------------------------|
|   | 좌우 이동     | 캐릭터를 좌우로 이동시키며, 이동 방향으로 시선이 자동 전환     |
|   | 상하 상호작용   | 사다리(Climb) 이용 시 상하 이동 및 특정 오브젝트와 상호작용 |
|                                                                                    | 대시 (Dash) | 바라보는 방향으로 빠르게 돌진, 대시 중 공격 시 대시 공격 발동  |
|                                                                                   | 점프        | 캐릭터가 공중으로 도약                          |

### 전투 및 특수 액션

|                                                                                     |         |                                     |
|-------------------------------------------------------------------------------------|---------|-------------------------------------|
|  | 일반 공격   | 검을 휘두르는 일반 공격                       |
|  | 슬라이딩 공격 | 지면을 미끄러지며 적의 하단을 공격하는 특수 액션 수행      |
|  | 대시 공격   | 바라보는 방향으로 빠르게 돌진하며 검을 휘두르는 특수 액션 수행 |

### 플레이어 피드백 설계

#### 실시간 반응성

모든 입력은 InputManager를 통해  
중앙 집중식으로 관리되어  
프레임 단위의 정교한 판정을 보장

#### 상태 기반 제어

캐릭터의 현재 상태에 따라  
(ex. 점프 중, 대시 중 등)  
특정 입력이 제한되거나 강화되어  
의도치 않은 조작 실수를 방지

# 프로젝트 소개

## 플레이어 조작 방법 및 UI/UX

### UI



- 1 **체력** : 플레이어의 현재 체력 및 최대 체력을 표시
- 2 **스탯** : 플레이어의 기본 스탯과 함께 코어 장착으로 인해 변화된 스탯을 표시
- 3 **인벤토리** : 보유중인 코어들을 확인하고 머지할 수 있음, 토글 방식으로 동작
- 4 **퀵슬롯** : 인벤토리에 보유중인 코어를 장착하거나 해제할 수 있음

### UX



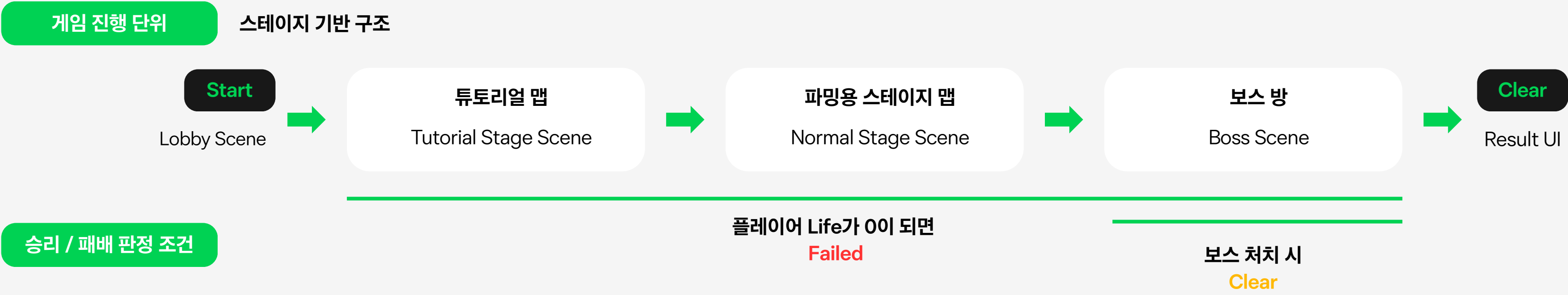
- 게임 시작 시 튜토리얼 스테이지로 이동
- 튜토리얼 스테이지에서 가이드에 따라 지형지물을 이동하거나 공격방법을 직접 시전
- 이를 통해 유저가 게임 조작법을 빠른 시간 내에 숙지하고, 본격적인 전투에 들어가기 전에 전투 전략과 감각을 경험할 수 있도록 유도

# 프로젝트 소개

## 주요 콘텐츠 구성

| 콘텐츠 항목           | 상세 설명 및 역할                                            | 루프(Loop)에서의 역할                        |
|------------------|-------------------------------------------------------|---------------------------------------|
| 플레이어 캐릭터         | 상태 기반의 액션 주체. 여러 상태와 히트박스 판정 제어                       | 조작 숙련도를 통한 전투의 재미 제공 및 보상 획득의 직접적인 수단 |
| 몬스터 및 보스         | 등급별 드랍 테이블을 가진 몬스터 및 페이즈 기반 AI 보스                     | 성장을 시험하고 체감하는 매개체이자 상위 티어 코어를 공급하는 원천 |
| 코어 (Core)        | 몬스터 처치 시 드랍되는 핵심 성장 자원. CoreDataSO 기반의 스탯 결정체         | 전투의 결과물이자 다음 전투를 위한 성장의 핵심 재료         |
| 인벤토리 시스템         | 32개 슬롯의 CoreInventoryData 관리. 2개 조합(Merge) 및 티어 정렬 지원 | 수집한 자원을 정리하고 상위 티어로 진화시키는 성장의 중추      |
| 퀵슬롯 (Quick Slot) | 3개 슬롯의 QuickSlotData. 수집한 코어를 실시간으로 장착 / 교체해 스탯 변조    | 플레이어의 전투 빌드를 결정짓는 전략적 의사결정 지점         |

## 게임 진행 구조





# 주요 시스템 & 기능 상세 설명

캐릭터

---

전투

---

코어 / 인벤토리

---

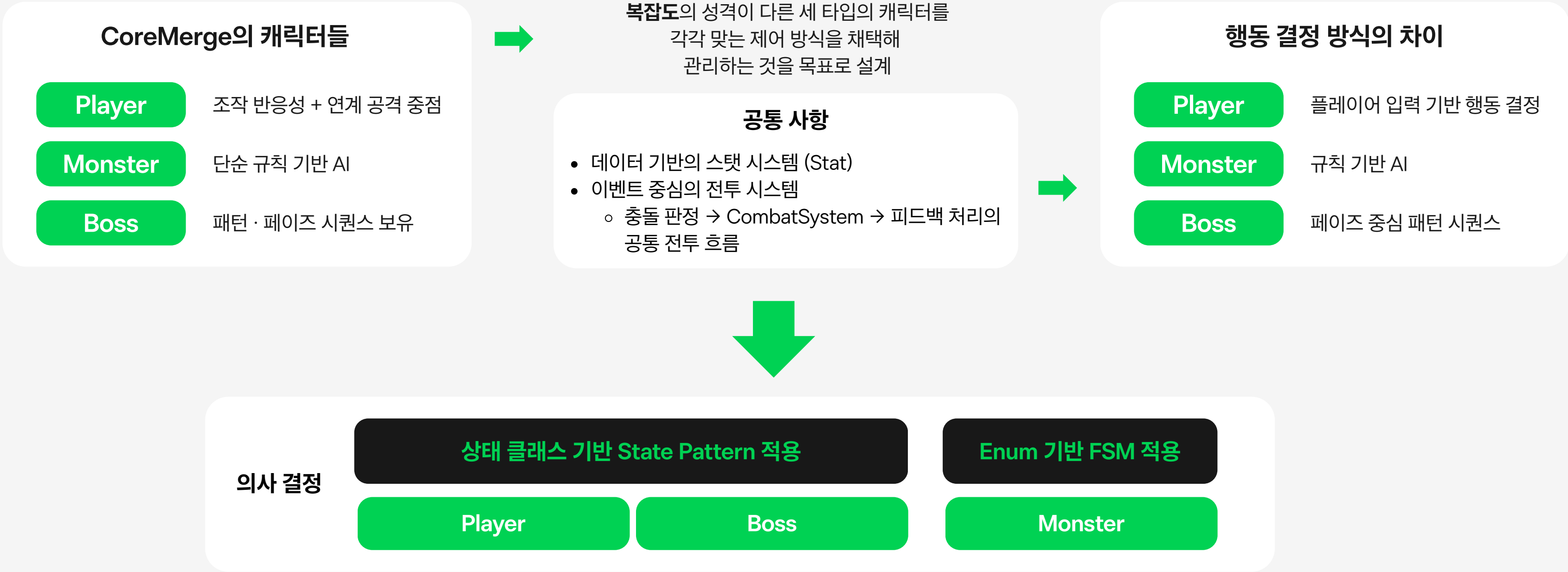
# 주요 시스템 & 기능 상세 설명

## 캐릭터

- 플레이어 캐릭터
- 적 캐릭터 (몬스터 / 보스)

# 캐릭터 시스템 개요

Player의 조작 반응성, Monster AI 규칙 기반 행동, Boss 패턴의 단계적 연출이라는 서로 다른 성격의 요구사항을 동시에 충족하기 위한 설계 방향을 모색  
캐릭터가 **현재 어떤 상태**에 있는지, 왜 그 **행동**을 하고 있는지를 구조적으로 명확히 만들기 위함



# 캐릭터 시스템 개요

## Player & Boss

### 상태 클래스 기반 상태 패턴

#### 선택 이유

##### 1. 복잡도 증가 방지

- 액션 게임은 입력 / 물리 / 전투 / 애니메이션이 동시에 영향을 주기 때문에, 이를 한 함수에 처리하면 조건문(if-else / switch)이 빠르게 커지고 복잡해짐
- 상태 패턴을 통해 로직을 상태별 클래스 / 파일로 분리해, **기능 추가 및 수정 시 변경 범위가 특정 상태로 제한**되도록 선택

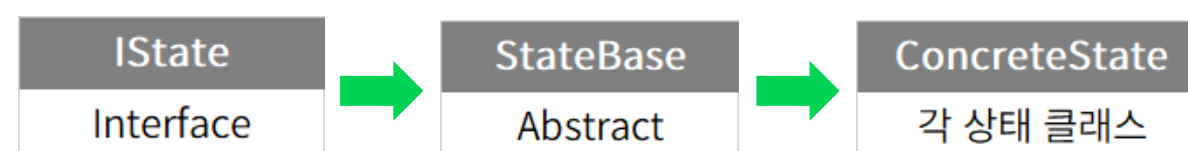
##### 2. 확정적 규칙 제어

- 상태간 제약이 많아질 수록 규칙 충돌이 발생하기 쉬움
  - ex) 공격 중 이동 불가, 피격 중 대시 불가
- 상태 패턴은 현재 상태를 기준으로 허용 가능한 행동을 제한하므로, **충돌 규칙을 구조적으로 차단**할 수 있다고 판단

##### 3. 애니메이션과 로직의 동기화

- 상태 전이 시점인 Enter / Exit에서 애니메이션 트리거를 실행해, **로직 전이와 표현 전이가 1:1로 대응**되도록 설계

#### 구조



## Monster

### Enum 기반 유한 상태 머신 (FSM)

#### 선택 이유

##### 1. 단순한 행동 로직과 낮은 복잡도

- 일반 몬스터의 행동 (순찰 → 추적 → 공격)은 직관적이고 선형적이므로, 클래스 단위로 나누는 것보다 한 파일 내에서 전체 흐름을 파악하는 것이 쉬움

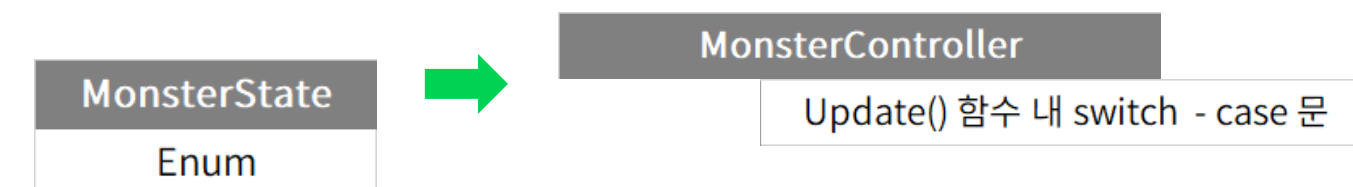
##### 2. 가벼운 메모리 할당을 고려

- 일반 몬스터는 한 화면에 다수가 동시에 존재할 수 있어, 상태별 객체를 운용하게 된다면 몬스터 개체 수가 늘어날수록 객체 수 / 참조가 선형적으로 증가
- Enum 기반 FSM은 상태가 단일 정수 값으로 관리되므로 상대적으로 가벼움

##### 3. Boss와의 역할 / 복잡도 차이를 구조로 반영

- 일반 몬스터는 Boss와 다르게 반복적으로 대량 스폰되는 규격형 엔티티
- Enum FSM을 사용해 가볍고 낮은 실행 비용을 유지하는 것이 합리적

#### 구조

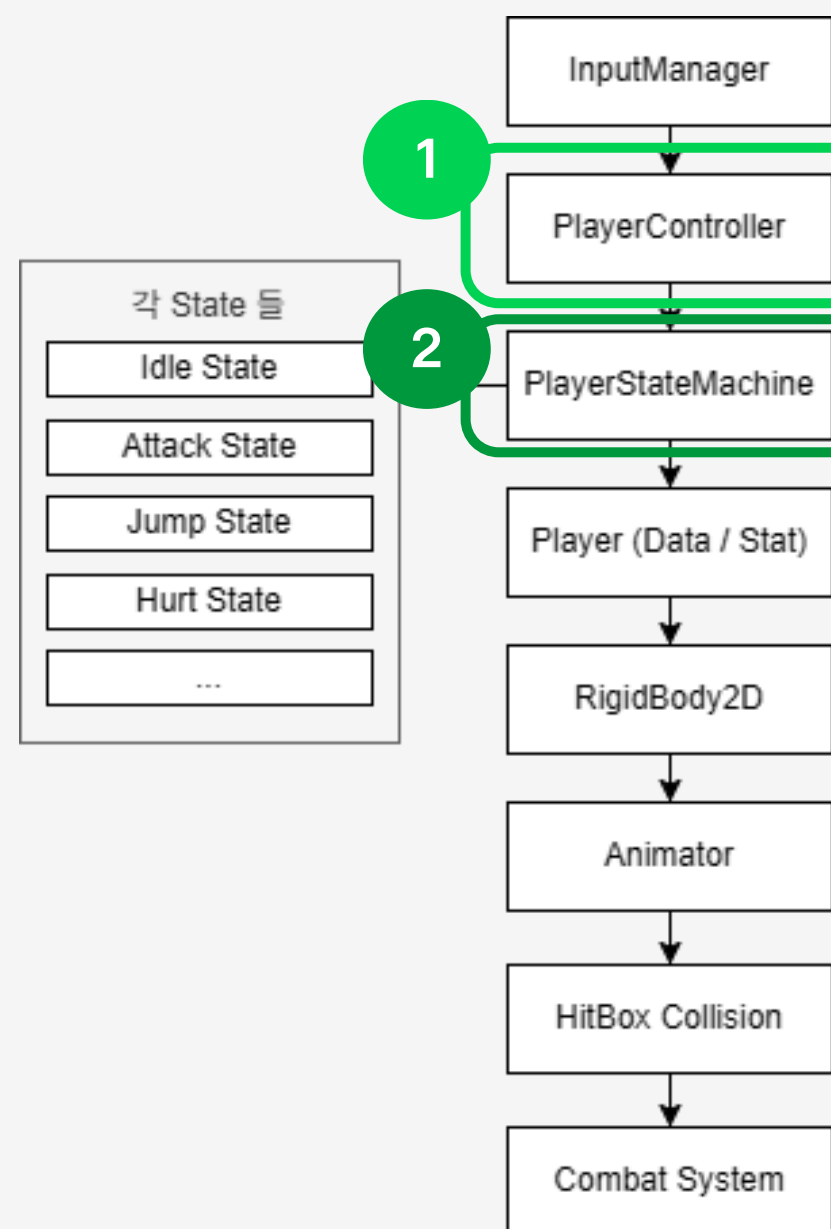




# 플레이어 캐릭터

플레이어 캐릭터는 **직접적인 조작 반응성**이 가장 중요한 요소로, 이동, 점프, 공격, 피격 등의 행동이 동시에 발생할 수 있고, 연계 공격등의 특수한 동작들을 수행하게 위해 **각 행동 간 충돌을 방지하는 구조**가 필요

## 시스템 구조



1

### PlayerController

InputManager를 통해 입력 이벤트 형태로 받은 플레이어 입력을 기반으로, **현재 상태에 맞는 행동 요청**을 생성

- 이동 입력 → Move State 요청
- 공격 입력 → AttackState 요청
- 점프 입력 → JumpState 요청
- ...

이때 PlayerController는 상태 전이를 직접 수행하지 않고 StateMachine에 상태 전이 요청만 전달

2

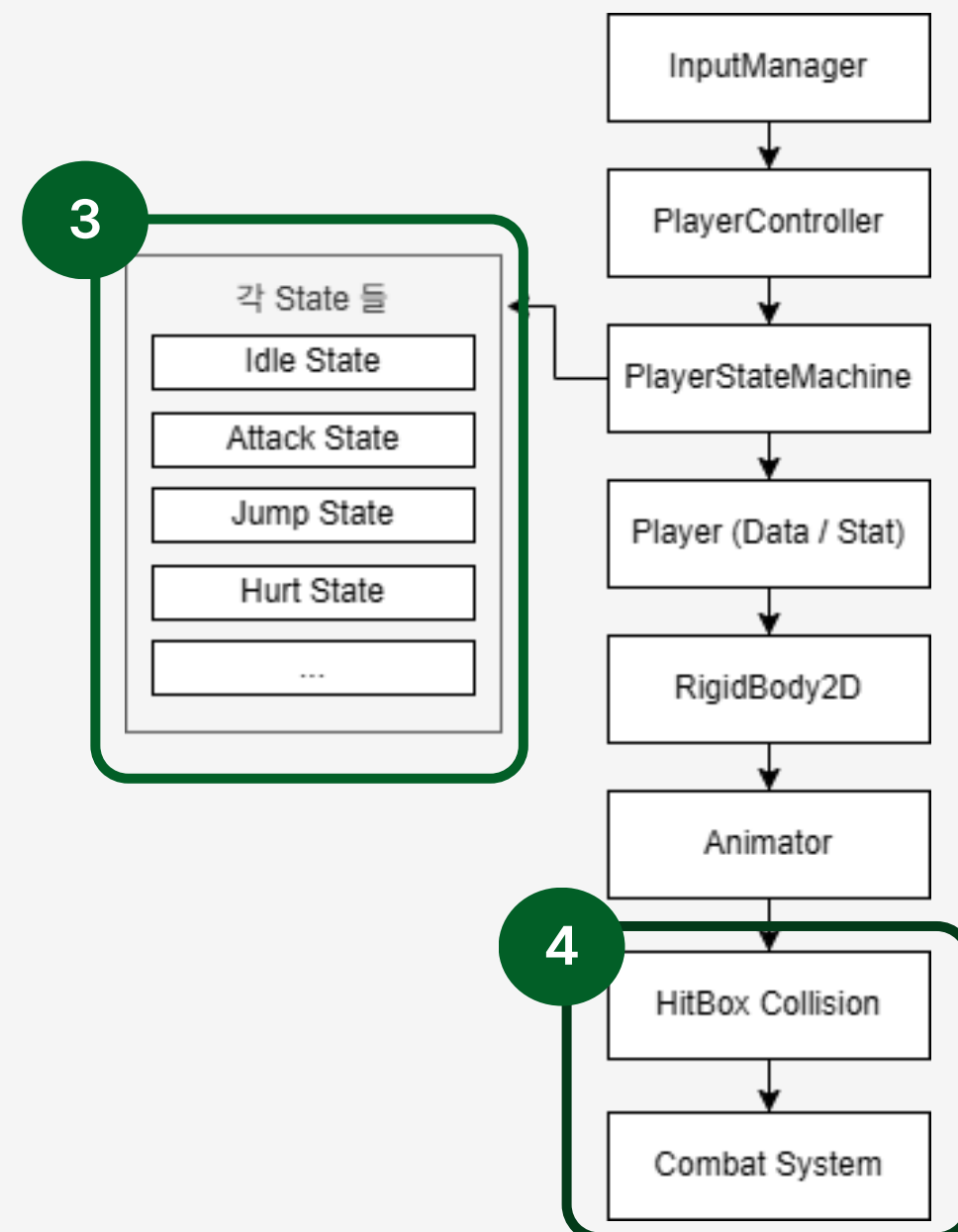
### PlayerStateMachine

**플레이어의 현재 상태를 관리**하며, 상태 전이 규칙을 담당

- 한 프레임에 하나의 상태만 활성화 (**현재 상태**를 실행시키는 실행기 역할)
  - 상태 진입 (Enter) → 상태가 시작될 때 실행되는 준비 작업
  - 상태 실행 (Update / FixedUpdate) → 상태가 유지되는 동안 반복 실행되는 메인 로직, 지금 프레임에 해야하는 일
  - 상태 종료 (Exit) → 상태를 떠날 때 다른 상태에 영향이 가지 않도록 정리하는 작업
- 상태 전이 규칙 관리
- 행동 간 충돌 방지

# 플레이어 캐릭터

## 시스템 구조



3

## 각 State 들

Player가 가진 공통 자원(데이터 / 컴포넌트)을 규칙적으로 사용하는 실행 단위  
PlayerStateBase를 정의해, 모든 State가 따라야하는 공통 실행 규칙을 고정함

- State가 접근할 수 있는 자원을 통일해 컴포넌트들의 참조를 미리 확보
- 각 State들은 컴포넌트 집합을 통해 캐릭터를 제어하며, 상태 전이는 PlayerStateMachine이 통제

```
public abstract class PlayerStateBase : IPlayerState
{
    protected PlayerController playercontroller;
    protected PlayerStateMachine stateMachine;
    protected Collider2D collider;
    protected Animator animator;
    protected Rigidbody2D rigidBody;

    [11] 사용 위치 2 jjudykim
    protected PlayerStateBase(PlayerController player, PlayerStateMachine stateMachine)[...]

    자주 호출됨 [10+2] 사용 위치 [10] 재정의 2 jjudykim
    public virtual void Enter()[...]
    자주 호출됨 [9+1] 사용 위치 [10] 재정의 2 jjudykim
    public virtual void Exit()[...]
    자주 호출됨 [9+1] 사용 위치 [9] 재정의 2 jjudykim
    public virtual void UpdateLogic()[...]
    자주 호출됨 [7+1] 사용 위치 [7] 재정의 2 jjudykim
    public virtual void UpdatePhysics()[...]
}
```

### PlayerController

런타임 / 데이터 스탯을 조회,  
"행동 규칙"을 결정

### Collider2D

상태 전이 조건의 근거

### Animator

표현 출력을 담당,  
상태와 동기화해 Enter/Exit 시 확정

### Rigidbody2D

물리 출력을 담당,  
UpdatePhysics()을 통해 수행

4

## 공격 판정

플레이어 공격은 애니메이션 이벤트 기반 HitBox 활성화 방식으로 처리

# 플레이어 캐릭터

## 시스템 흐름

### [Update - Every Frame 1]

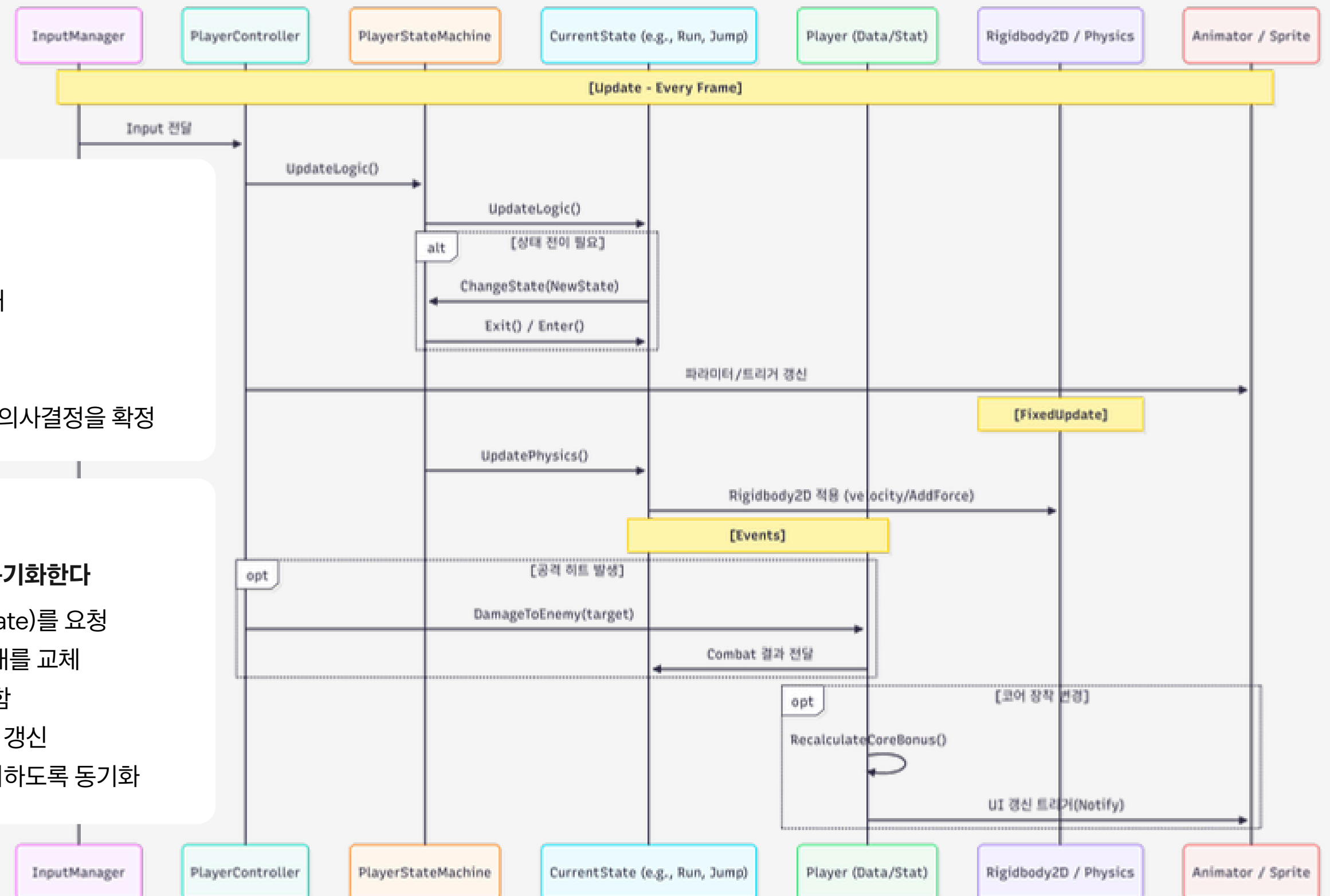
입력을 먼저 확보하고, 행동을 결정한다

- InputManager로 받은 입력을 PlayerController가 해석해 PlayerStateMachine의 UpdateLogic()을 호출
- StateMachine은 현재 State의 UpdateLogic()을 실행
- 이 단계에서 Player(Data / Stat)를 참조해 이번 프레임의 의사결정을 확정

### [Update - Every Frame 2]

상태 전이는 Exit→Enter로 정리하고, 표현은 Animator로 동기화한다

- 전이가 필요하면 CurrentState가 ChangeState(NewState)를 요청
- StateMachine은 Exit(정리) → Enter(초기화) 순서로 상태를 교체  
→ 이전 상태의 잔여값(플래그/판정/속도)이 남지 않도록 함
- 이후 PlayerController가 Animator의 파라미터/트리거를 갱신
- 논리 상태(결정)와 시각 표현(재생)이 같은 프레임에서 일치하도록 동기화



# 플레이어 캐릭터

## 시스템 흐름

### [Fixed Update]

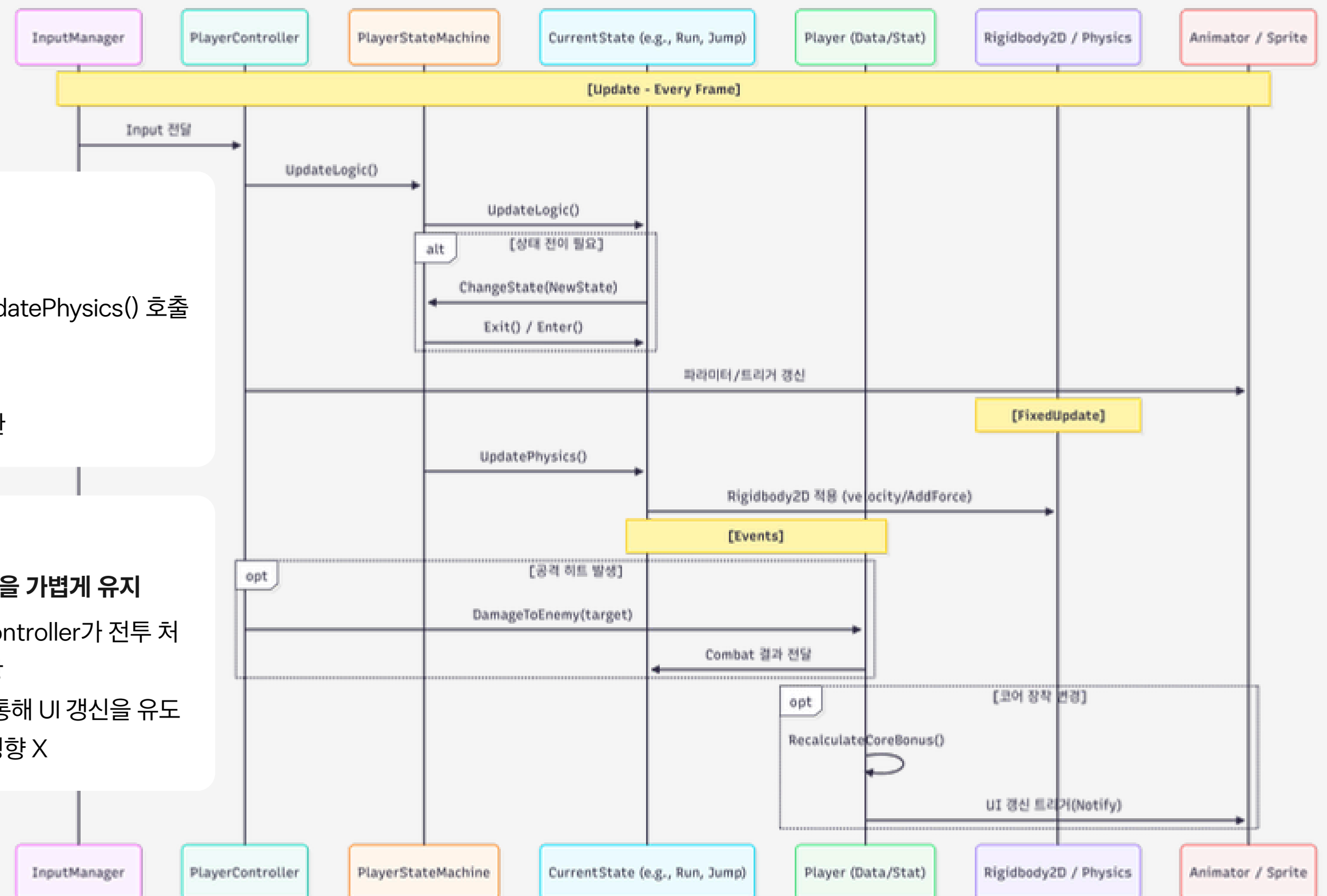
결정된 행동을 Rigidbody2D에 적용해 "물리 결과"로 만든다

- 고정된 시간 주기에서 StateMachine이 현재 State의 UpdatePhysics() 호출
- State는 Rigidbody2D에 속도 / 힘을 적용
- Update에서는 무엇을 할지를 정했다면,  
Fixed Update에서는 해당 결정을 일관된 물리 결과로 변환

### [Events]

전투 히트 / 장착 코어 변경은 루프 밖에서 처리해 상태 시스템을 가볍게 유지

- 공격 히트는 충돌 / 애니메이션 이벤트로 발생해, PlayerController가 전투 처리를 트리거하고 Player(Data/Stat)에서 전투 계산을 담당
- 장착 코어 변경은 Player가 보너스를 재계산 한 뒤 알림을 통해 UI 갱신을 유도  
→ State 로직을 수정하지 않고 Stat 파라미터만 갱신해 영향 X



# 플레이어 캐릭터

실제 구현

플레이어 캐릭터의 관심사 분리

## Player — 데이터와 규칙 (정체성)

- 플레이어의 "정체성"에 해당하는 수치/규칙을 관리
- PlayerStat과 장착 코어 보너스(CoreStatModifier)를 합산해 **최종 스탯(FinalStat)**을 산출
- CurrentHp를 중심으로 회복/사망 같은 생명 주기를 처리
- 데미지 계산(치명타/분산 등) 같은 전투 수식을 담당하고, 전투 처리 흐름으로 전달

▶ 물리/애니메이션과 분리되어 밸런스·시너지 변경이 엔진 동작에 영향을 주지 않음

## PlayerController — 물리적 실체와 감지 (신체 역할)

- Unity 컴포넌트와 직접 통신하는 "몸체"로서 Rigidbody2D/Animator/Collider2D를 캐싱하고 제어
- 지면/사다리 등 환경 조건(IsGrounded, IsOnLadder 등)을 판단해 상태 전이의 근거를 제공

▶ 물리(FixedUpdate)와 표현(Animator 파라미터)이 한 지점에서 정리되어 움직임과 시각 피드백의 불일치를 줄임

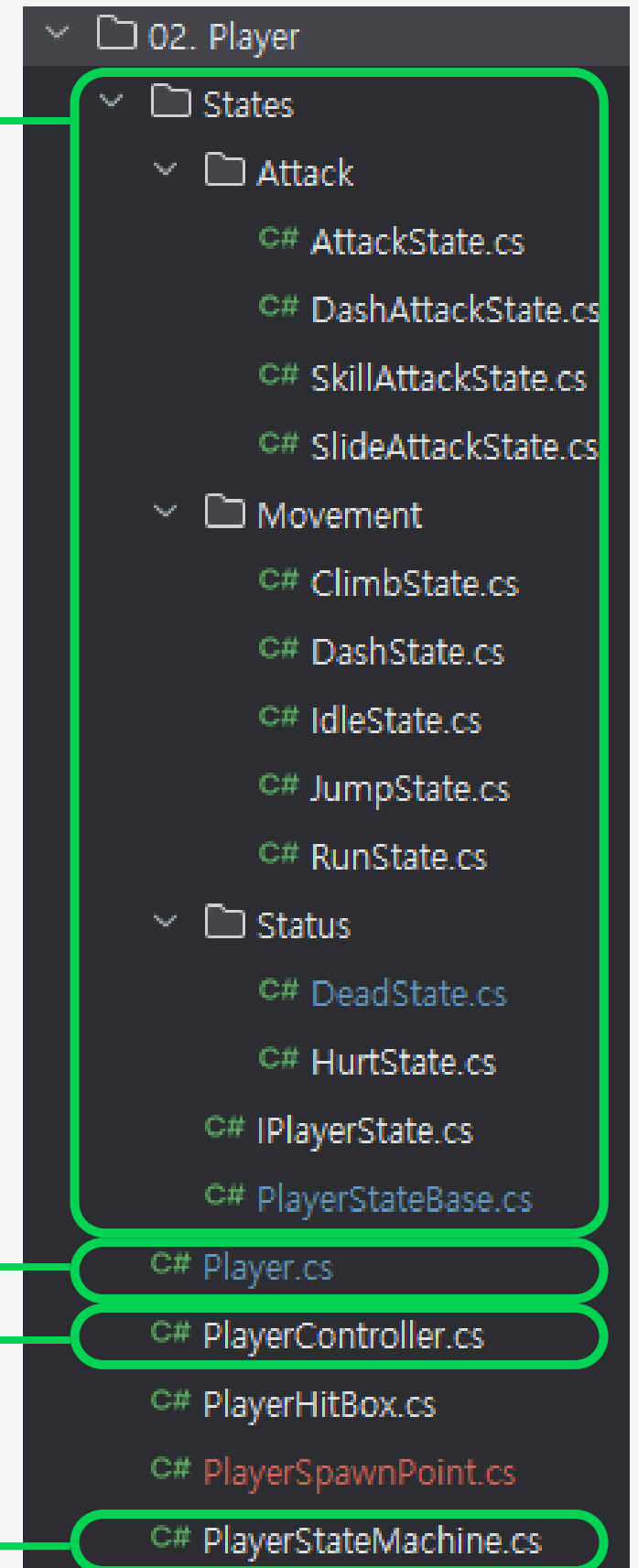
## PlayerStateMachine — 행동 의사결정 (두뇌 역할)

- "지금 무엇을 해야 하는가"를 결정하고, 현재 상태(CurrentState)를 실행
- 상태 전이는 Exit() → Enter() 순서를 보장하여, 잔여 플래그/판정/속도가 남지 않도록 제어
- Update/FixedUpdate 신호를 현재 활성 상태에만 전달해 실행 흐름을 통제

▶ 결과적으로 Controller에 조건문이 누적되는 구조를 피하고, 비행/수영 같은 신규 행동도 상태 추가만으로 확장 가능

세부 상태들

Player (데이터)  
PlayerController  
(물리 / 신체)  
PlayerStateMachine  
(논리 / 두뇌)





# 플레이어 캐릭터

## 실제 구현

복잡한 애니메이션 트랜지션을 State와 연동해 Trigger 기반으로 구현

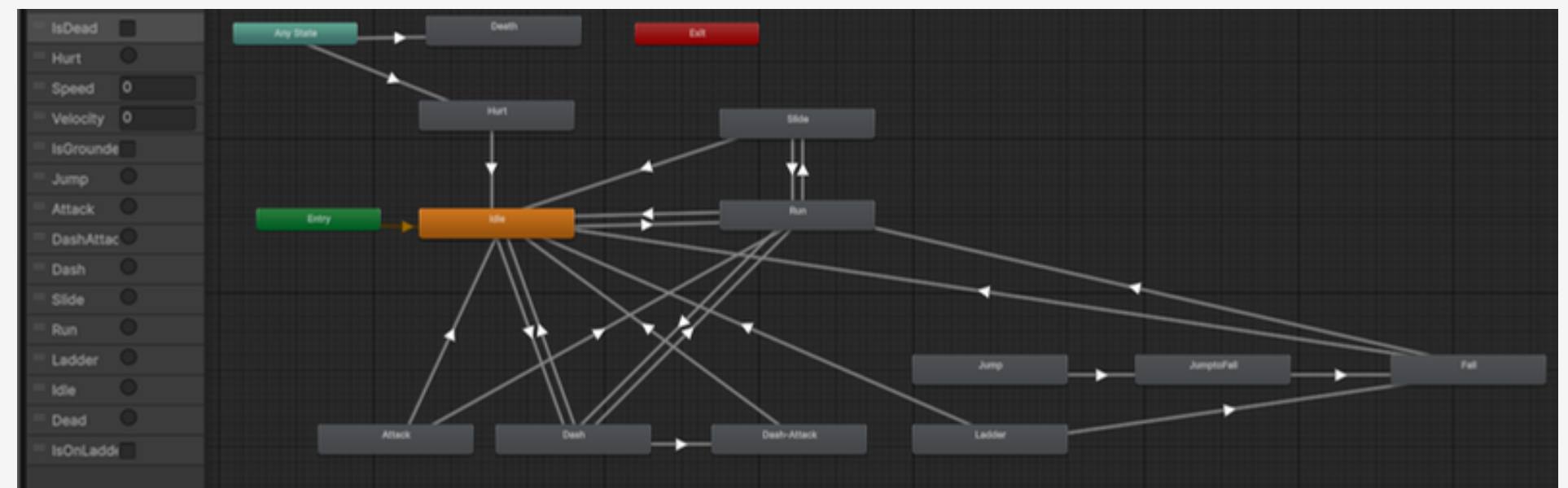
애니메이션 전이 시점을 State 전이(Enter)에서 확정하고,  
Animator는 Trigger로 즉시 전이하도록 구성

### 작동 방식

1. `PlayerStateMachine.ChangeState(AttackState)` 가 호출
2. 현재 상태의 `Enter()` 메서드 내부에서 `animator.SetTrigger(애니메이션 키)` 를 실행
3. 애니메이터는 복잡한 조건 체크 없이 해당 애니메이션으로 전이

### 장점

'언제 애니메이션이 바뀔 것인가'가 C# State 코드에 명시적으로 남아 디버깅이 쉬워짐



## 애니메이션 이벤트 기반으로 HitBox를 On / Off 해 공격

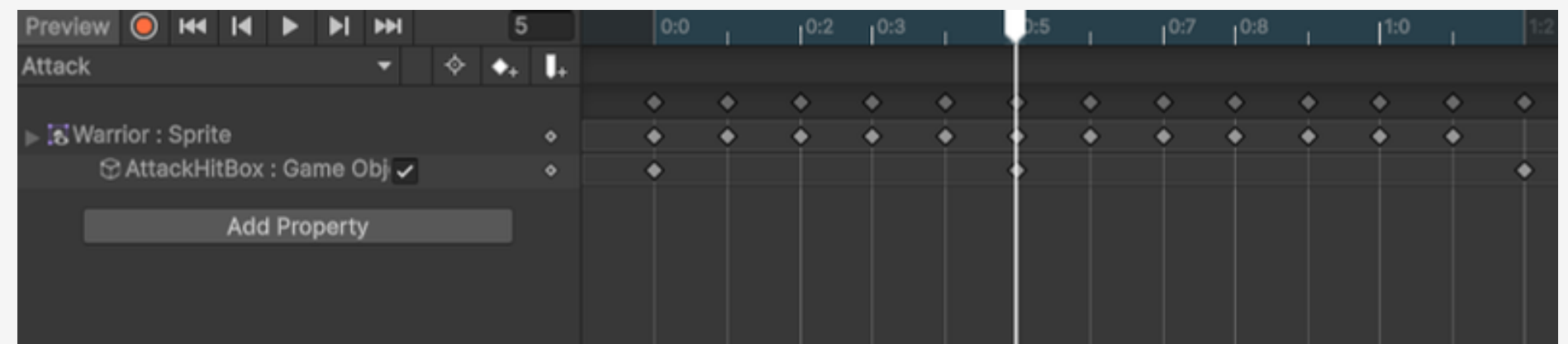
공격 판정을 코드가 아닌 애니메이션 이벤트로 제어해,  
검을 휘두르는 “보이는 프레임”과 실제로 “때리는 프레임”을 일치

### 작동 방식

1. 공격 애니메이션에서 실제로 검을 휘두르는 프레임에서 Hitbox Collider를 활성화
2. 해당 Hitbox의 OnTriggerEnter2D를 통해 Player에게 데미지 처리를 요청
3. 공격 모션이 끝나는 프레임에서 다시 히트박스를 비활성화

### 장점

프레임 동기화로 타격감이 안정적이고, 이벤트 위치만 조정하면 되므로 판정 구간에 대한 조정이 쉬워짐





# 적 캐릭터 (몬스터 / 보스)

CoreMerge의 적 캐릭터 시스템은 **Monster**라는 공통의 데이터/기능 기반 위에,  
일반 몬스터(MonsterController)와 보스(BossController)의 제어 로직이 완전히 갈라지는 구조를 선택

## 시스템 구조

1

### Monster

모든 적 개체는 Monster를 상속받아 유니티 엔진 및 전투 시스템과 소통하는 **공통 규격을 공유**  
→ 적이 늘어나도 피격·사망·드랍 같은 생명주기 규칙이 일관되게 유지되도록 설계

- 스탯 시스템 (BaseStat) : HP/공격력/방어력 등 공통 수치를 관리
- 피격 · 사망 로직 : 데미지 처리 함수로 피해 처리를 수행, 사망 시 코어 드랍 함수로 코어를 드랍
- 컴포넌트 캐싱 : 유니티 엔진 기반 핵심 컴포넌트를 공통 방식으로 캐싱
- 중앙 추적 지원 : 정적 메소드를 정의해 씬 내 모든 적 개체를 중앙 매니저에서 추적하고 관리하도록 설계

**Monster**라는 뿌리는 같지만, 이를 움직이는 **두뇌(Controller)**에서 구현의 분기가 극명하게 나뉨

2

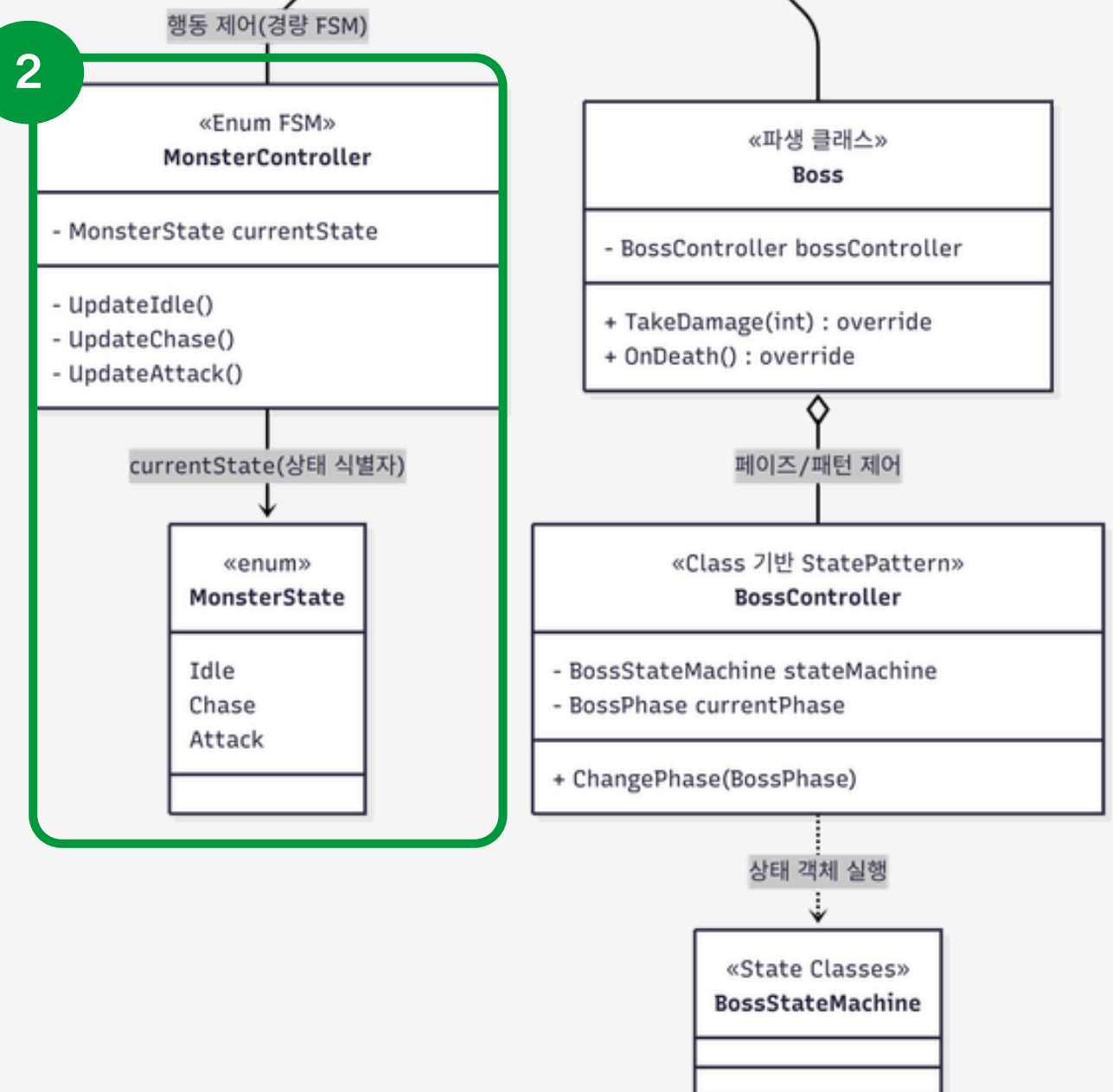
### Monster Controller — 일반 몬스터 (Normal / Epic)

- 제어 방식: Enum 기반 FSM (상태가 enum으로 정의됨)
- 상태 전이: 하나의 Update 메서드 내에서 switch-case 문을 통해 다양한 상태 로직을 실행
- 공격 패턴: 거리 기반 공격 상태 전환, 단일 공격 애니메이션과 접촉 데미지 중심의 단순한 패턴
- 성능 최적화: 개별 상태 클래스를 생성하지 않아 메모리 사용량이 적고, 다수의 몬스터 동시 처리에 용이

1



2



# 적 캐릭터 (몬스터 / 보스)

## 시스템 구조

3

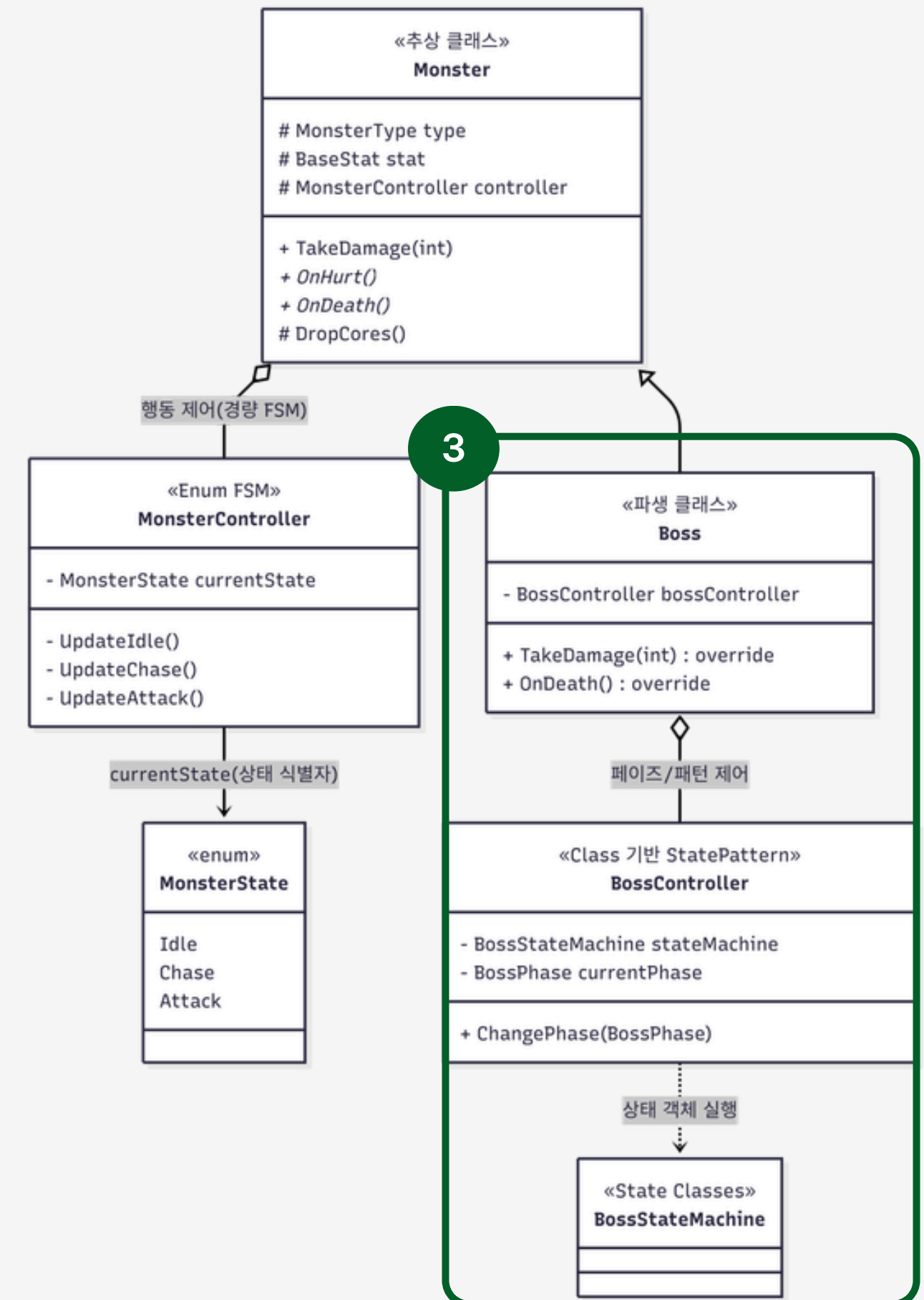
### Boss — 보스 (BossController & BossStateMachine)

- 제어 방식: 클래스 기반 상태 머신 (상태가 독립된 클래스로 정의됨)
- 상태 전이: BossStateMachine을 통해 Enter, UpdateLogic, Exit가 엄격히 구분된 상태 전환을 수행
- 페이지 시스템: BossPhase라는 상위 개념이 존재  
→ 체력에 따라 Phase1에서 Phase2로 완전히 다른 행동 세트로 전환
- 복잡한 시퀀스: 대시 공격, 투사체 발사 공격, 낙하 공격 등 복잡한 공격 패턴을 개별 클래스로 관리

## 제어 방식을 왜 분기했는가

| 구분      | 일반 몬스터 (Monster) | 보스 (Boss)              |
|---------|------------------|------------------------|
| 등장 빈도   | 높음 (다수 등장)       | 낮음 (단일 등장)             |
| 행동 복잡도  | 낮음 (단순 추적 및 타격)  | 매우 높음 (페이지 패턴 및 패턴 대응) |
| 유지보수 초점 | 성능 및 개수 확장성      | 패턴의 정교함 및 연출 제어        |
| 구조 선택   | Enum FSMn (가벼움)  | State Machine (복잡, 견고) |

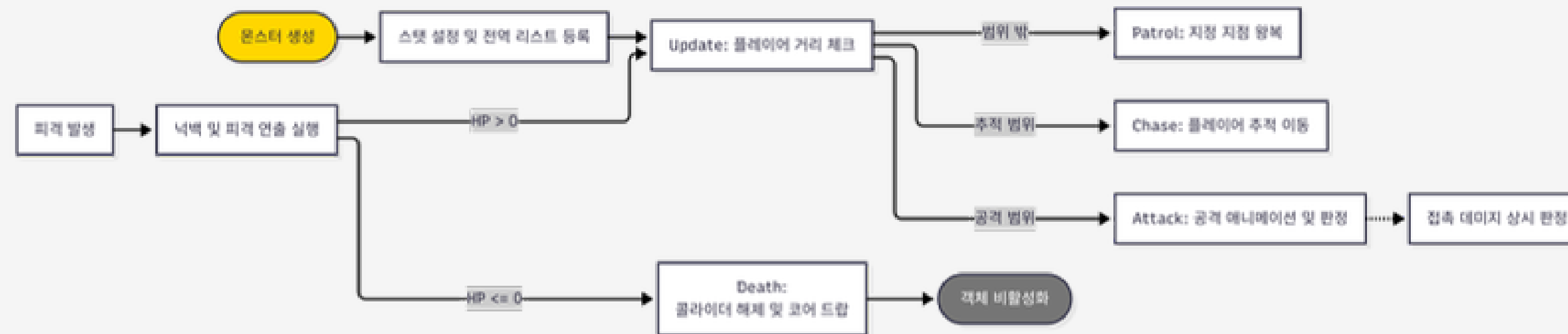
- ▶ 모든 적 개체를 Monster로 규격화하여 전투 시스템과의 호환성을 유지하되,  
다수 개체의 성능 효율과 단일 개체의 패턴 전이의 정교함을 동시에 확보하기 위해 행동 제어 방식을 이원화



# 적 캐릭터 (몬스터 / 보스)

시스템 흐름

Monster의 실행 흐름



## 1. 초기화 및 등록

생성 시 BaseStat을 할당하고, 중앙 매니저가 씬 내의 모든 몬스터들을 관리하는 [정적 리스트]에 자신을 등록하여 매니저가 관리할 수 있게 함

## 2. 거리 기반 AI 판단

매 프레임 **플레이어와의 거리**를 계산하여 상태를 전환

- Patrol: 플레이어가 감지 범위 이상으로 거리가 멀 때 지정된 두 지점 사이를 왕복하며 배회
- Chase: 감지 범위 내에 들어오면 플레이어 방향으로 가속 이동
- Attack: 공격 사거리 내에 진입 시 쿨다운을 확인하고 공격을 실행

## 3. 전투 상호작용 (Combat)

OnTriggerEnter2D를 통해 플레이어와 접촉 시 즉시 데미지를 입히며, 공격 상태에서는 애니메이션 이벤트와 연동된 히트박스를 활성화

## 4. 피격 대응 및 사망

- 데미지를 입으면 Hurt 상태로 전이하여 시각적 피드백(반짝임)과 넉백을 적용
- 체력이 0이 되면 DropCores 메서드를 호출해 아이템을 드랍하고 월드에서 제거

# 적 캐릭터 (몬스터 / 보스)

## 시스템 흐름

### Boss의 실행 흐름

#### 1. 초기화 : 상태 인스턴스화 → 상태 머신 주입 → 최초 진입

- BossController의 Awake 단계에서, 보스가 사용할 수 있는 상태들을 모두 클래스 단위로 미리 인스턴스화
- 각 상태들은 생성 시 컨트롤러와 상태머신에 대한 참조를 전달받아, 보스 컴포넌트와 전이 로직에 접근
- 최초 진입 : IdleState로 초기화된 상태에서 전투를 시작해 패턴을 선택할 준비를 함

#### 2. Phase 1 - 기본 패턴 루프

- Idle 상태의 의사결정: 플레이어와의 거리와 설정된 확률 데이터를 실시간으로 체크해 패턴을 결정
- 전이 (패턴 실행 상태로 이동)
  - 근접 패턴 : 플레이어가 특정 범위 내에 있거나 근접 타이밍이 되면 공격 예고 후 근접 공격
  - 원거리 견제 패턴 : 애니메이션 이벤트와 연동해 발사 시점을 제어하며 플레이어를 추적하는 발사체를 발사
- 복귀 : 각 공격 상태는 패턴 수행이 완료되면 다시 IdleState로 전이를 요청한 후 한번의 패턴 루프가 종료

#### 3. 페이즈 전환

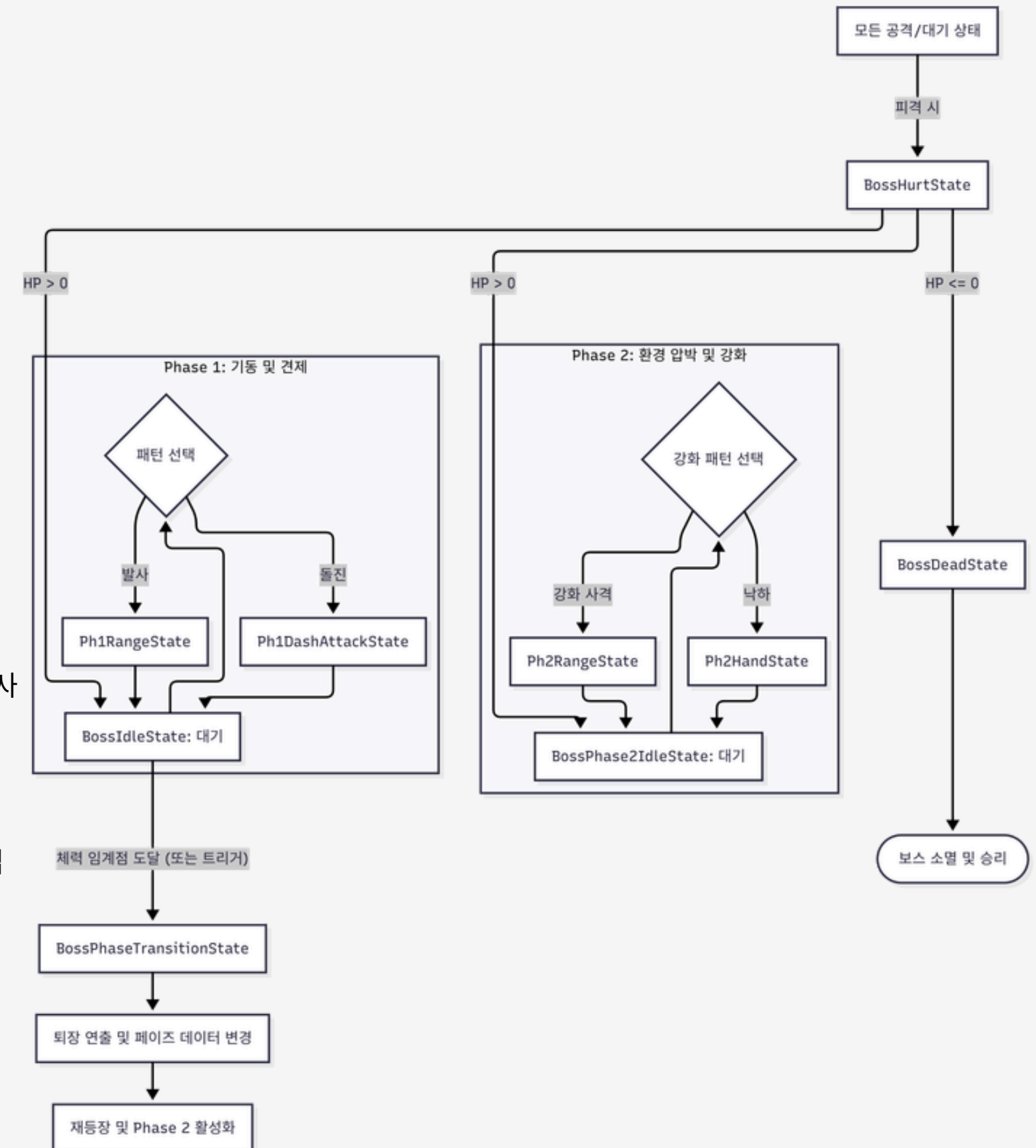
- 트리거 감지 : 보스의 체력을 상시 감지해, 체력이 임계점(50%)이 되면 즉시 페이즈 전환으로 강제 전이
- Despawn 연출 → 페이즈 값을 변경해 Phase2 패턴으로 전환 → Spawn 연출과 함께 Phase2의 Idle로 진입

#### 4. Phase 2 - 강화 패턴 루프

- 강화된 로직 : Phase2에서는 새로운 공격 형태의 상태 클래스가 동작
  - 낙하 공격 (Hand Attack) : 하늘에서 거대한 손 형상의 투사체가 각각 랜덤한 위치에서 여러 번 낙하

#### 5. 피격 및 사망으로 인한 종료

- Hurt : 보스는 Idle 상태에서만 피격이 가능하며, 피격 시 Hurt 상태로 전이가 되며 애니메이션 재생
- Dead : 체력이 0이 되면 모든 상태 로직을 중단하고, DeadState에서 최종 사망 연출과 함께 게임 종료 판정





# 적 캐릭터 (몬스터 / 보스)

실제 구현

Boss 패턴 구현 및 Phase 전환



## Phase 1 돌진 공격 패턴

플레이어의 위치를 예측해 강력한 물리 충격을 가하는 패턴

- 준비 단계 : 상태 진입 시 보스는 플레이어의 현재 위치를 타겟하고 **"공격을 예고"**하는 애니메이션을 재생  
→ 플레이어에게 회피할 시간을 제공
- 돌진 단계 : 준비 단계가 끝나면 보스의 히트박스를 활성화하고, 타겟 방향으로 급격하게 돌진을 시작해 플레이어에게 데미지
- 회기 및 종료 : 돌진 및 공격 후, 히트 박스를 비활성화 → 보스는 다시 원래의 위치로 부드럽게 복귀 후 Phase1의 Idle 상태로 전환



## Phase 1 원거리 사격 패턴

플레이어를 추적하는 유도탄을 살포해 연속된 이동을 강제하는 패턴

- 발사 준비 : 보스가 공격 자세를 취해 플레이어가 공격에 대비할 시간을 제공
- 연사 시퀀스
  - 발사 애니메이션 이벤트가 수신되면 실제 발사 로직이 활성화되어, 정해진 개수만큼의 투사체를 일정한 간격으로 생성
  - 각 투사체는 생성 시 플레이어 방향으로 발사되며, 날아가는 동안 실시간으로 플레이어를 향해 회전하는 유도 기능을 가짐
- 종료 : 발사가 모두 끝나면 애니메이션 이벤트를 통해 발사 루틴을 정지하고 다시 Phase1의 Idle 상태로 복귀

# 적 캐릭터 (몬스터 / 보스)

## 실제 구현

### Boss 패턴 구현 및 Phase 전환



#### Phase 1 → 2 페이즈 전환

페이즈 전환의 연출 및 내부 페이즈 및 상태 로직의 전환을 위한 전투 시퀀스의 휴식 지점

- 트리거 및 상태 진입 : 체력이 50% 이하로 떨어지면 페이즈 전환 상태로 진입, 모든 공격 패턴을 멈추고 전환 연출에만 집중
- 퇴장 연출 및 전투 격리 : 보스가 잠시 사라지는 연출과 함께 보스의 콜라이더를 끄고 잠시 동안 전투 환경에서 격리시킴  
→ 유저에게 보스가 어디서 언제 다시 등장할지 모르는 긴장감 + 장착 코어 변경할 수 있는 기회 제공
- 내부적인 데이터 변경 : 일정 시간 후 보스의 페이즈 정보를 변경 후, 특정 스폰 지점으로 이동 후 페이즈2 전투 환경 설정
- 강화된 등장 및 패턴 활성화 : 데이터 교체 직후 재등장 애니메이션을 재생하고, 연출이 끝나면 2페이즈 전용 강화 패턴 시전



#### Phase 2 낙하 공격 패턴

하늘에서 거대한 손 형상을 떨어뜨려 플레이어의 이동을 방해하고 범위 타격을 하는 패턴

- 발사 준비 : 일정한 높이에서 설정된 전장 범위(MinX ~ MaxX)에서 무작위로 낙하 지점을 결정
- 연사 시퀀스
  - 설정된 MinX ~ MaxX 사이의 무작위 횡수만큼 손이 생성되고 낙하
  - 설정된 공격 간격에 맞춰 순차적으로 무작위 위치에 손이 생성되고 떨어지며, 모든 소환이 끝난 후 대기 시간을 거침
- 종료 : 발사가 모두 끝나면 애니메이션이 종료되고, 다시 Phase2의 Idle 태로 돌아가 다음 강화 패턴을 준비



# 주요 시스템 & 기능 상세 설명

## 전투

- 전투 이벤트
- HitBox & HurtBox
- 전투 이벤트에 대한 피드백

# 전투 시스템 개요

전투 시스템에 대한 고민의 시작점 : “공격이 맞았다”는 하나의 사건을 어떤 구조로 처리할 것인가

전투는 단순히 HP 감소 뿐만 아니라, 충돌 판정, 데미지 계산, 상태 변화, 시각적 피드백이 같은 흐름 안에서 유기적으로 연결되어야 한다는 점을 파악

## 전투 시스템에 대한 의사 결정 요소

전투 처리를 누가 담당할 것인가?

공격 / 피격 판정을 어떤 방식으로 분리할 것인가?

공격했다는 감각을 어떤 피드백으로 전달할 것인가?

### 전투 시스템 정의

전투는 “하나의 이벤트” → **CombatEvent** 정의  
이벤트 종합 처리자 → **CombatSystem** 정의

공격자

- 공격자는 “전투 이벤트”만 생성
- 이벤트 처리자에게 전달
- 공격자는 누가 맞는지 모름

Combat  
System

- 전달 받은 이벤트만 처리
- 대상자에게 데미지 적용
- 중간에서 데미지 / 보정값 등 계산

### HitBox / HurtBox 구조 도입

“공격”이란 개념에 포함되어야 하는 내용

- 어느 프레임에서 공격이 유효한지
- 누가 공격자이고 누가 피격자인지
- 어떤 종류의 공격인지

모든 충돌을 하나의 Collider에서 처리하면,  
이동 충돌과 공격 판정이 섞이거나 예외 처리 복잡도 ↑



공격 판정을 담당하는 **HitBox**  
피격 판정을 담당하는 **HurtBox** (대상 콜라이더)  
두 충돌 감지의 역할을 분리해서 담당

### 전투 피드백 - FloatingText, 피격 플래시

전투는 데미지 계산이 정확하더라도,  
플레이어가

- 지금 공격했다
- 강한 공격을 했다
- 피격 당했다

는 감각을 즉각적으로 받아야 한다는 점에 초점

즉, 유저가 전투를 체감할 수 있는 **전투 피드백**도  
전투의 중요 설계 요소 중 하나

➡ 머리 위 데미지, **FloatingText**로 수치를 즉시 표시  
흰색으로 깜빡이는 **피격 플래시**로 순간적인 시각 반응

# 전투 시스템 개요 — 전투 처리 방식에 대한 고민

## 방법 1) 직접 호출 방식

공격자가 충돌 대상을 확인한 후, 해당 대상의 컴포넌트를 가져와 직접 데미지 적용 함수(TakeDamage)를 호출

플레이어 : PlayerHitBox → **Monster** 컴포넌트 찾기 → **Monster.TakeDamage(damage)**

몬스터 : MonsterHitBox → **Player** 컴포넌트 찾기 → **Monster.TakeDamage(damage)**

### ○ 장점

- 구현이 단순함 : 작은 프로젝트에서 빠르게 붙이기 쉬움
- 흐름이 직관적 : '누가 누구를 때렸는가?'를 코드에서 바로 읽기 쉬움
- 초기 디버깅이 쉬움 : 호출 스택이 짧아, 추적이 쉬움

### ⚠ 단점

- 공격자가 피격자 타입을 알아야 함
  - Player는 Monster를 알아야 하고, Monster는 Player를 알아야 하고
  - Boss, Trap 등 대상이 늘어날수록 계속해서 알아야 하는 대상이 추가
- 피드백 로직이 분산 : 데미지 출력, 타격 이펙트 등이 각자 공격자 안에 분산
- 공통 규칙을 넣기 어려움 : 난이도 보정이나 디버프 시 각 객체를 각각 수정

## 방법 2) 이벤트 기반 + CombatSystem 중계 방식

공격자는 "전투 이벤트"만 생성해서 CombatSystem에게 전달하고, CombatSystem이 "어떻게 처리할지"를 중앙에서 결정

공격자 : HitBox → CombatEvent 생성 → **CombatSystem 전달**

**CombatSystem** : 대상 식별 → **해당 대상에게 데미지 적용** → 피드백 요청

### ○ 장점

- 결합도가 낮음
- 전투 규칙이 중앙 집중화 돼, 데미지 처리, 공통 보정 등을 한 곳에서 처리 가능
- 디버깅 포인트가 명확

### ⚠ 단점

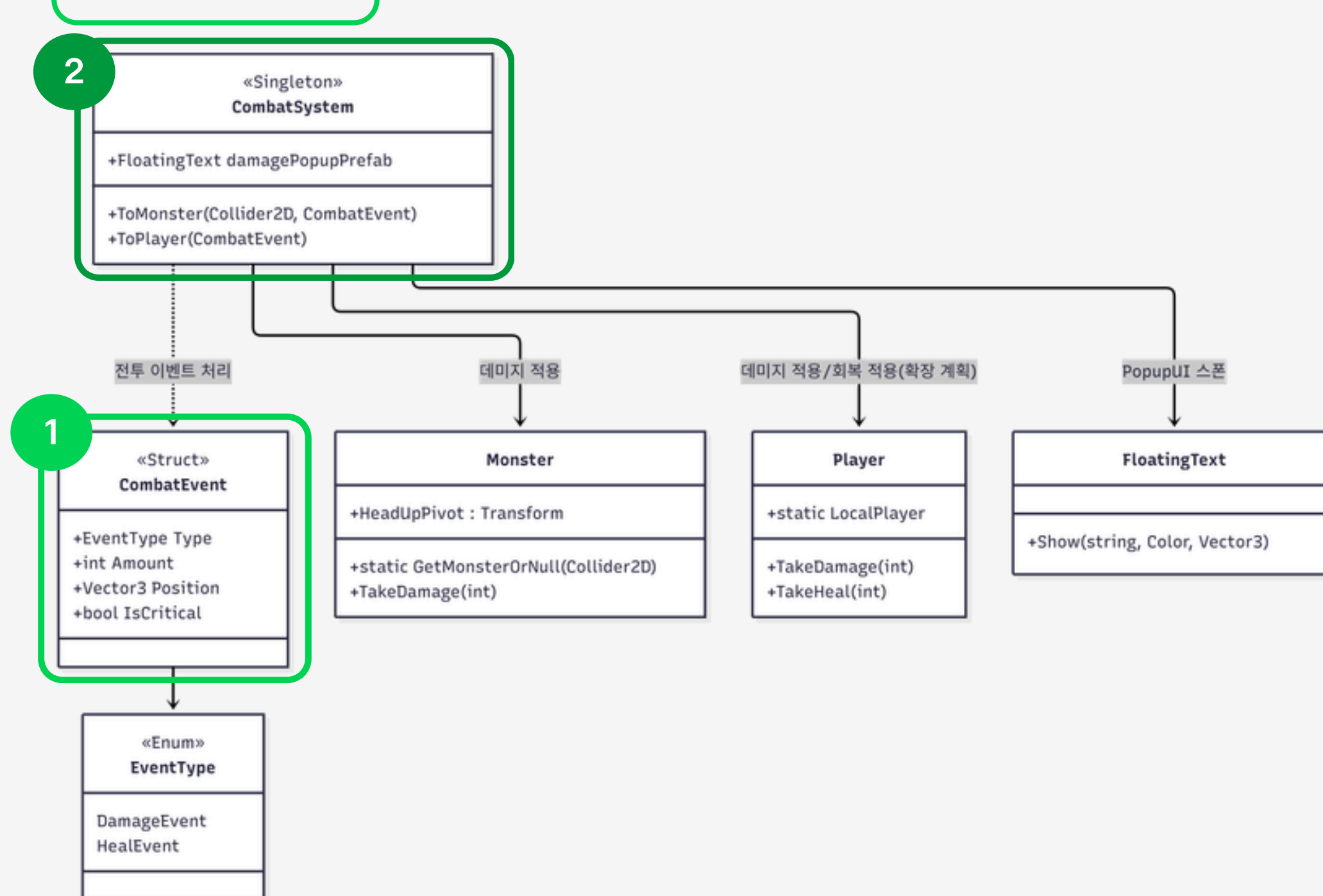
- 초기 구현의 복잡성
  - CombatEvent 구조, CombatSystem 중계 계층 필요
  - 호출 흐름이 한 단계 더 늘어남

✓ 방법 2 채택

# 전투 이벤트

CoreMerge의 전투 시스템은 CombatSystem라는 싱글톤 클래스를 중계자로 하여, 공격자와 피격자 사이의 모든 상호작용을 CombatEvent라는 데이터 규격으로 표준화

## 시스템 구조



1

## CombatEvent

Struct 타입으로 한 공격자가 발생시킨 '행위'인 전투 정보를 규격화

```

public struct CombatEvent
{
    public EventType Type { get; set; }
    public int Amount { get; set; }
    public Vector3 Position { get; set; }
    public bool IsCritical { get; set; }
}
    
```

|                |
|----------------|
| 데미지, 회복 이벤트 구분 |
| 실제 데미지 적용 수치   |
| 이벤트 발생 좌표      |
| 치명타 여부         |

모든 전투 행위는 이 구조체에 담겨 CombatSystem으로 전달, 피격자의 구체적인 상태를 알 필요 없이 모든 전투 계산이 가능하도록 설계

2

## CombatSystem

게임 내의 전투 행위를 중앙에서 제어하는 중계자이자 관제탑  
전달받은 CombatEvent를 기반으로 실제 데미지 적용 / 시각적 연출

- 공격 측에서 전투 상황을 CombatEvent로 정리한 내용을 전달 받음
- 대상을 식별하거나 확보해, EventType에 따라 어떤 처리를 할지 판단
- 실제 대상에게 피해 수치를 적용
- 전투 결과를 화면 피드백으로 연결해 전달

# 전투 이벤트

## 시스템 흐름

— Player 관점

### 1. 공격 데이터의 정밀한 생성

- 플레이어의 히트 박스가 몬스터와 충돌하면 **Player**에서 최종 데미지를 연산
- 이때, 장착된 코어 아이템의 시너지가 모두 합산된 **FinalAttack** 수치를 사용
- 연산된 수치를 **CombatEvent**라는 구조체에 담음

### 2. 중계자에게 위임

- CombatSystem이라는 중계자에게 “이 대상에게 이 데이터로 처리”를 요청

### 3. 대상 식별 및 로직 실행

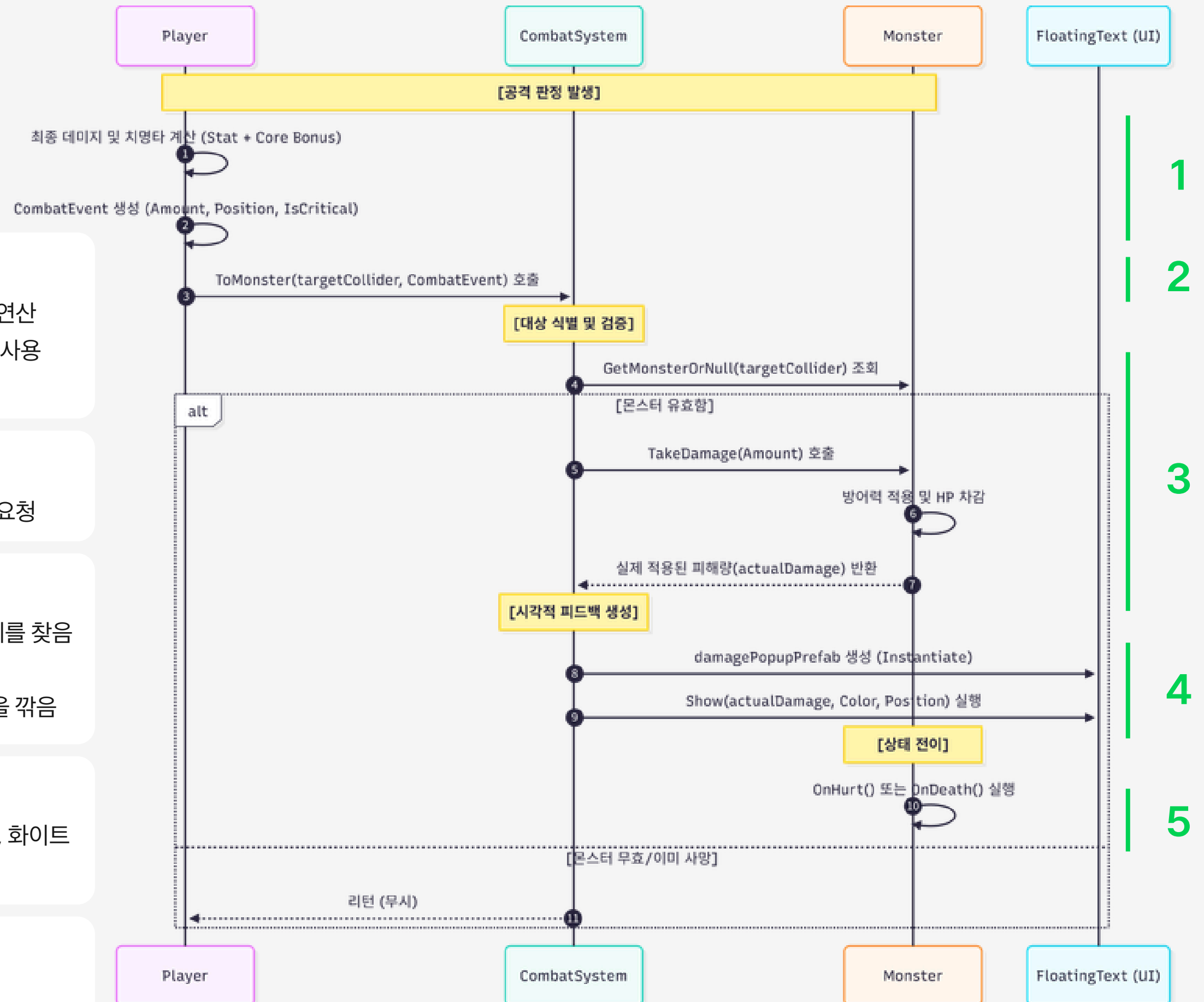
- CombatSystem은 전달받은 콜라이더 정보를 통해 실제 Monster 객체를 찾음
- 찾은 Monster의 TakeDamage(데미지 적용 함수)를 호출
- 몬스터는 자신의 방어력을 적용해 최종 피해량을 결정하고 자신의 체력을 깎음

### 4. 최종 피드백 출력

- 데미지 처리가 완료되면 CombatSystem은 FloatingText를 띄우거나, 화이트 플래시 효과를 적용해 시각적 효과를 전달

### 5. 피격 대상의 반응

- 몬스터는 남은 체력에 따라 아파하거나(Hurt), 아이템을 드랍(Death)





# 전투 이벤트

## 실제 구현

### 플레이어 캐릭터가 몬스터에게 데미지를 전달하는 흐름

플레이어는 **자신의 기본 스탯, 장착한 코어 보너스, 치명타 판정, 랜덤 분산도**를 모두 반영해 최종 데미지를 계산한 뒤, 이를 CombatEvent에 담아 CombatSystem으로 전달  
단순한 공격 입력 처리자가 아닌, 전투 수치를 동적으로 조합하는 계산 주체라고 볼 수 있음

#### 1. 스탯 합산

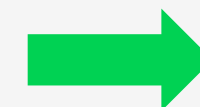
- PlayerStat의 기본 수치와 장착 중인 코어의 coreBonus를 합산  
→ 최종 전투 수치를 구성
- 이 값은 항상 현재 장착 상태를 반영한 최신값을 반환하는 프로퍼티들을 통해 노출
- 퀵슬롯의 이벤트를 구독해, 코어를 교체하는 즉시 재계산 로직이 실행 → 실시간 수치 반영

코어 보너스가 적용된  
각 스탯 프로퍼티들

```
public int FinalMaxHp => PlayerStat.MaxHp + coreBonus.maxHpBonus;  
public int FinalAttack => PlayerStat.Attack + coreBonus.attackBonus;  
public int FinalDefense => PlayerStat.Defense + coreBonus.defenseBonus;  
public float FinalCritChance => Mathf.Clamp01(PlayerStat.CritChance + coreBonus.critChanceBonus);  
public float FinalCritDamageMultiplier => Mathf.Max(1f, PlayerStat.CritDamageMultiplier + coreBonus.critDamageBonus);  
public float FinalSkillCooldownReduction => Mathf.Clamp(PlayerStat.SkillCooldownReduction + coreBonus.skillCooldownRate, min: 0f, max: 0.9f);
```

장착 코어 변경 이벤트 구독  
코어 보너스 재계산 유도

```
private void OnEnable()  
{  
    if (QuickSlots != null)  
        QuickSlots.OnChanged += RecalculateCoreBonus;  
}
```



현재 퀵슬롯에 담겨있는 코어의 정보들을 모두 읽고 코어 보너스를 재계산  
프로퍼티 호출 시 갱신된 수치가 반영된 정보를 활용할 수 있음

# 전투 이벤트

실제 구현

플레이어 캐릭터가 몬스터에게 데미지를 전달하는 흐름

## 2. 데미지 계산 알고리즘

### 1 기본 수치 설정

- FinalAttack을 베이스 데미지로 사용

### 2 치명타 판정

- 코어 시너지를 통해 연산된 치명타 확률(FinalCritChance)을 활용해 치명타 여부를 결정
- 치명타 발생 시 베이스 데미지에 치명타 피해량(FianlCritDamageMultiplier)를 곱해 상위 피해량을 계산

### 3 랜덤 분산도 적용

- 최종 데미지에 약 10% 범위의 오차를 부여
- 같은 공격이라도 매번 동일한 숫자가 반복되지 않는 미세한 전투 생동감을 더함

실제 플레이어의  
데미지 계산 알고리즘 구현

```
private int GetCalculatedDamage(string key, out bool isCritical)
{
    float baseDamage = FinalAttack;

    float damageAfterCrit = CalculateDamageWithCore(baseDamage, out isCritical);

    float variance = Random.Range(0.9f, 1.1f);
    float finalDamage = damageAfterCrit * variance;

    return Mathf.Max(1, Mathf.RoundToInt(finalDamage));
}
```

1. 기본 수치 설정

2. 치명타 판정

코어 효과에 따른 데미지를 연산하는 함수 내부에서,  
치명타 여부에 따라 현재 데미지에 치명타 피해량이 적용된 최종 데미지를 반환

3. 랜덤 분산도 적용

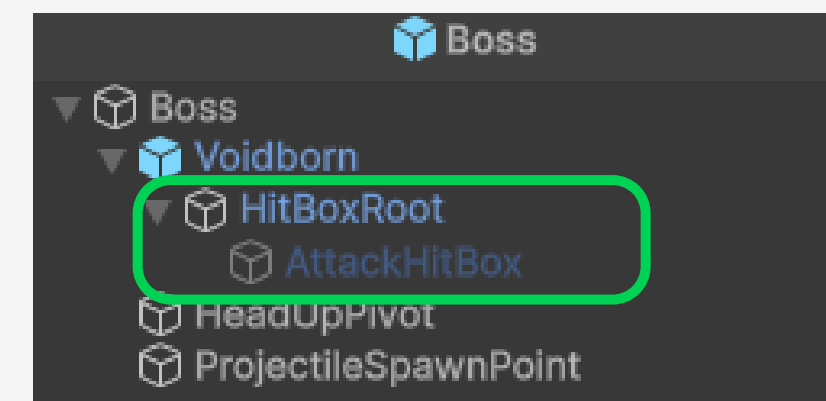
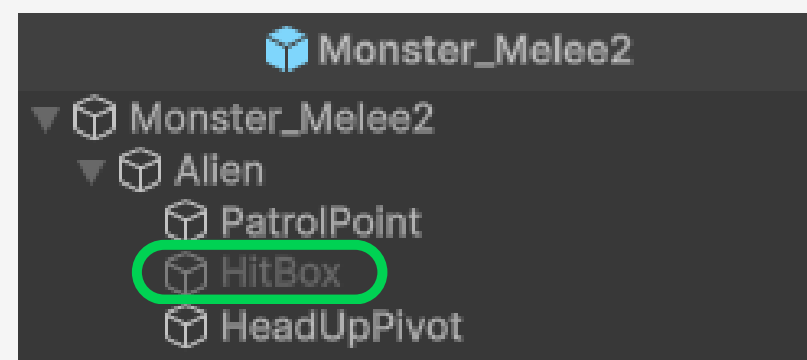
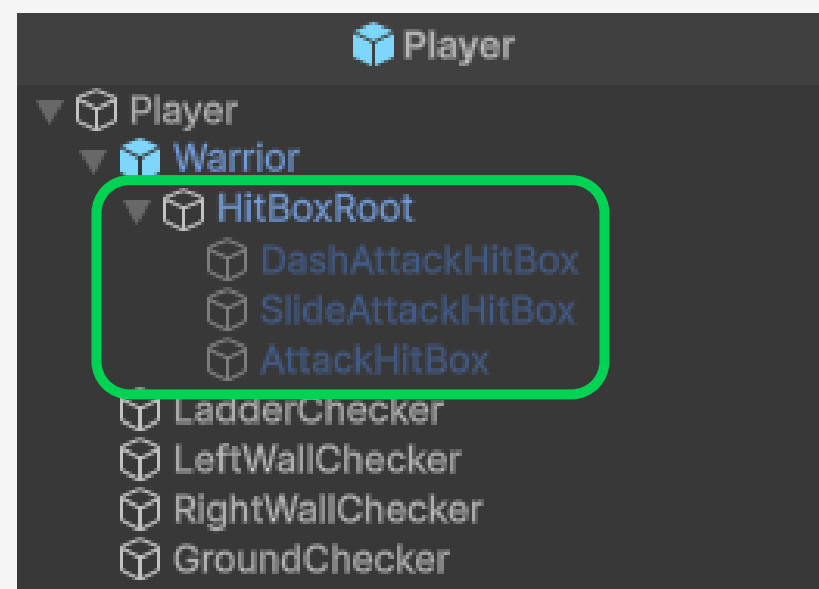
# HitBox / HurtBox

CoreMerge의 전투 판정은 **HitBox 기반 충돌 판정 시스템**을 중심으로 구성되어 있으며, 모든 공격은 애니메이션 또는 행동 로직에 의해 **HitBox가 짧은 시간 활성화** 충돌이 감지되면 해당 이벤트는 각 객체의 Controller를 통해 전투 시스템으로 전달

## 시스템 구조

모든 HitBox는 캐릭터 객체의 시각적인 스프라이트 하위에 공격 판정을 담당하는 자식 오브젝트로 존재

- **Player** : hitBoxes 배열로 관리, 공격 타입에 맞는 개별 콜라이더가 자식으로 부착
- **monster** : hitbox 변수로 MonsterHitBox 컴포넌트를 가진 자식 오브젝트를 참조
- **Boss** : 근접 공격 패턴 전용의 HitBox를 자식으로 가짐



공격을 발생시키는 주체와 활성화 방식은 각 캐릭터의 설계와 공격 목적에 맞게 다르게 구성

- |                          |                             |
|--------------------------|-----------------------------|
| Player / Boss (근접 패턴 공격) | — 직접 제어 방식 (코드가 주도권을 가짐)    |
| Monster                  | — 중계 제어 방식 (애니메이션이 주도권을 가짐) |

# HitBox / HurtBox

## 시스템 구조

### Player의 HitBox 제어

1

#### PlayerController — 히트박스 이벤트의 관리자

- Awake 시점에 **자식 오브젝트의 PlayerHitBox들을 배열로 캐싱**, 자신을 해당 히트박스의 소유자(Owner)로 주입
- 각 히트박스는 자신이 어떤 공격 동작에 해당하는지 HitBoxType 데이터를 직접 보유하고 있어, 공격 식별을 도움

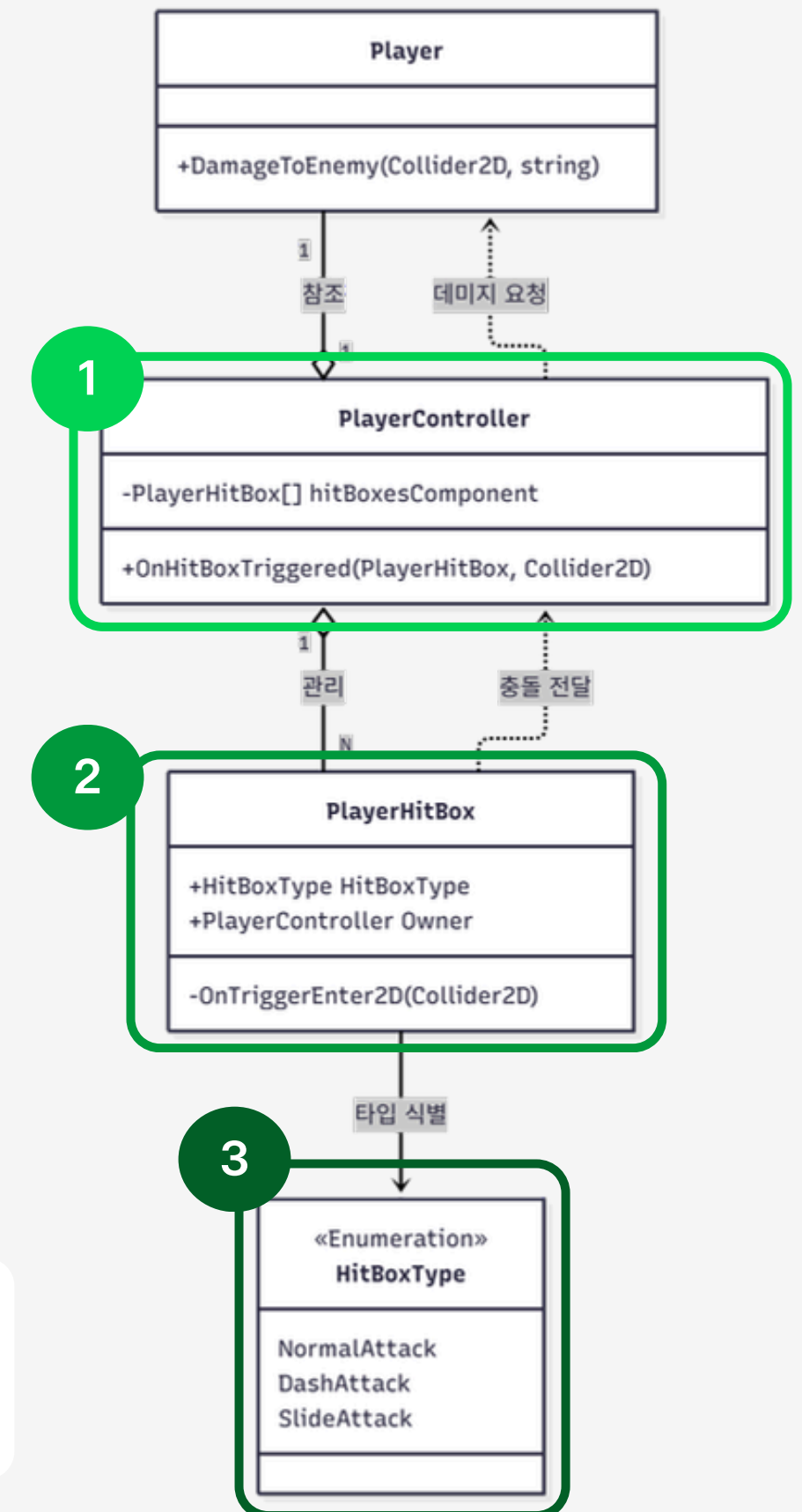
2

#### PlayerHitBox — 히트박스 이벤트의 창구

- 감지 : 특정 히트박스가 몬스터를 감지
- 보고 : 히트박스가 자신의 **3 HitBoxType** 과 함께 **소유자의 히트박스 트리거 함수**를 호출 (Owner.OnHitBoxTriggered)
- 분기 : PlayerController는 전달된 공격 타입을 보고, 적절한 로그나 특수한 처리를 수행할 수 있음
- 실행 : 최종적으로 Player가 **공격 이벤트를 발행해 CombatSystem에게 전달**하는 함수를 호출해 데미지 연산 수행 (DamageToEnemy)

#### ✓ 애니메이션 이벤트 기반이 아닌 **코드**로 제어하는 이유

- **정밀한 조작감** : 플레이어의 공격은 즉각적인 피드백이 중요하므로, 애니메이션 이벤트를 기다리기보다 히트박스를 직접 제어
- **다중 히트박스 관리** : 플레이어는 공격 종류마다 다른 히트박스를 사용, Controller가 현재 상태를 보고 직접 캐싱된 배열에서 활성화



# HitBox / HurtBox

## 시스템 구조

### Monster의 HitBox 제어

1

**AnimEventReciever** — 히트박스 이벤트의 관리자

적 캐릭터들은 애니메이션 이벤트 기반으로 공격 시점을 결정

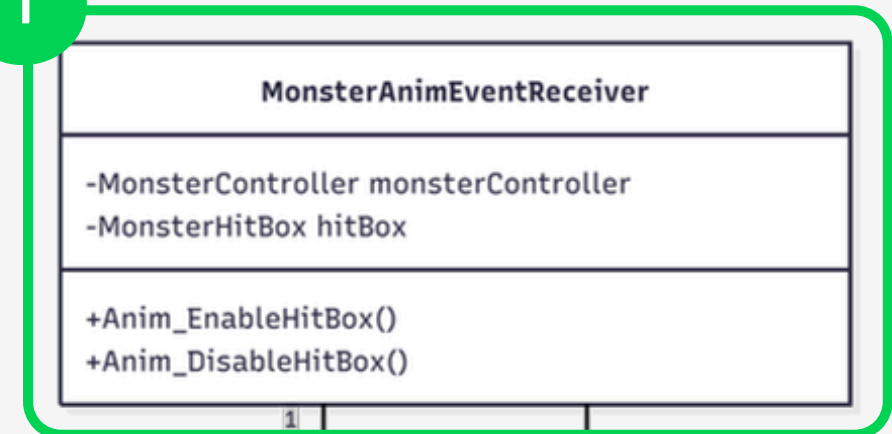
- 실제로 공격을 가하는 프레임 / 끝나는 프레임에 이벤트를 심어둠 → HitBox의 활성화 / 비활성화
- 시각적인 공격 시점과 실제 판정 시점을 일치시키기 위한 판단

2

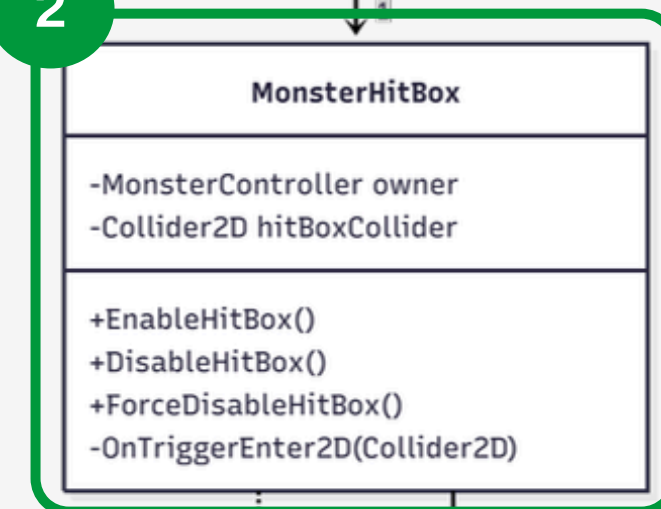
**MonsterHitBox**

- 애니메이션 이벤트에 따라 HitBox가 소유한 Collider의 활성화 / 비활성화가 이루어짐
- **Trigger 판정**을 수행하는 감지기
- 히트박스가 플레이어를 감지하면, 컨트롤러를 거치지 않고 **즉시 CombatEvent**를 생성
- 자신이 소유한 Monster 객체의 스탯 중 Attack 수치를 가져와 데미지를 확정  
→ Player와 달리 데미지 계산 로직이 복잡하지 않고, 공격 타입 분기가 필요하지 X
- 최종적으로 CombatSystem에게 생성한 이벤트를 전달하여 전투를 마무리

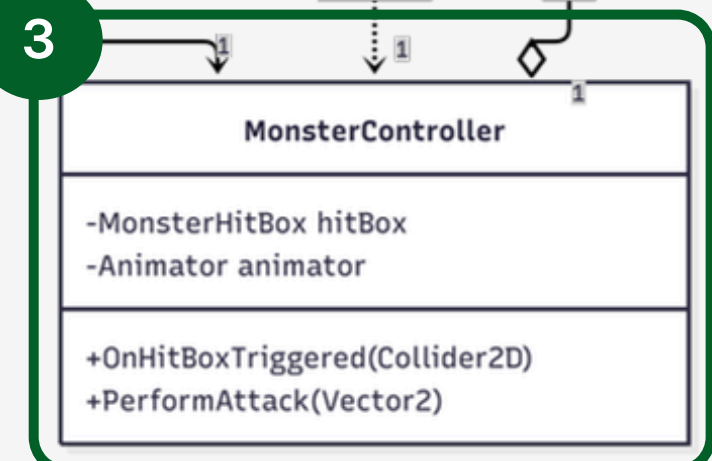
1



2



3



활성화/비활성화 트리거

상태 확인

충돌 전달

관리

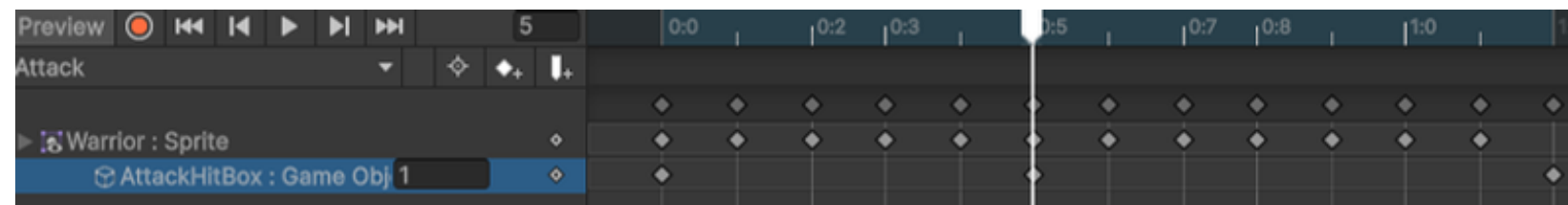


# HitBox / HurtBox

실제 구현

Player

## 1. 공격 타입마다 할당된 HitBox를 애니메이션 키로 활성화 / 비활성화



## 2. 각각의 HitBox들은 공격에 따른 타입이 정해져있으며, Trigger 시 Controller에서 각각 다른 로직 수행 가능

```
public enum HitBoxType
{
    DashAttack,
    SlideAttack,
    NormalAttack,
}

public class PlayerHitBox : MonoBehaviour
{
    public HitBoxType HitBoxType;
    public PlayerController Owner { get; set; }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Monster"))
        {
            Owner.OnHitBoxTriggered(this, other);
        }
    }
}
```

```
public void OnHitBoxTriggered(PlayerHitBox playerHitBox, Collider2D other)
{
    switch (playerHitBox.HitBoxType)
    {
        case HitBoxType.DashAttack: Debug.Log("OnHitBoxTriggered :: DashAttack Done!!!"); break;
        case HitBoxType.SlideAttack: Debug.Log("OnHitBoxTriggered :: SlideAttack Done!!!"); break;
        case HitBoxType.NormalAttack: Debug.Log("OnHitBoxTriggered :: Attack Done!!!"); break;
    }

    string attackKey = playerHitBox.HitBoxType.ToString();

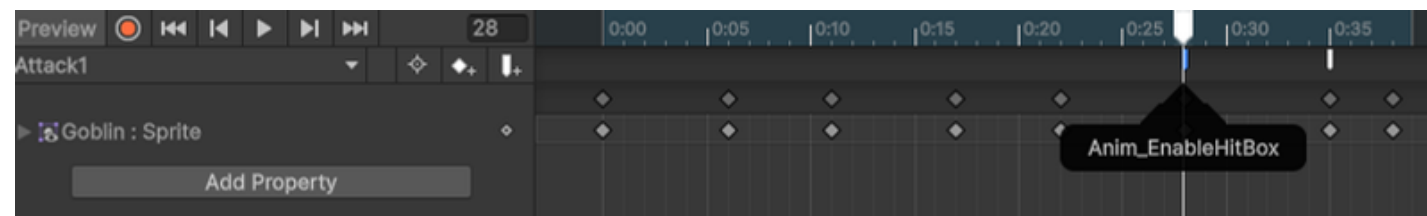
    Player.LocalPlayer.DamageToEnemy(other, attackKey);
}
```

# HitBox / HurtBox

실제 구현

Monster

1. 애니메이션 타임라인에서 공격이 시작 / 끝나는 지점에 애니메이션 이벤트 할당



2. 이벤트 수신자가 해당 이벤트를 통해 히트박스를 활성화 / 비활성화

```
public class MonsterAnimEventReceiver: MonoBehaviour
{
    [SerializeField] private MonsterController monsterController;
    [SerializeField] private MonsterHitBox hitBox;

    public void Anim_EnableHitBox()
    {
        if (monsterController.CurrentState != MonsterState.Attack)
            return;

        hitBox.EnableHitBox();
    }

    public void Anim_DisableHitBox()
    {
        hitBox.DisableHitBox();
    }
}
```

```
protected internal void EnableHitBox()
{
    if (hitCollider != null)
        hitCollider.enabled = true;
}

protected internal void DisableHitBox()
{
    if (hitCollider != null)
        hitCollider.enabled = false;
}
```

3. 히트박스는 Trigger 시 Player 검사 후 전투 이벤트 송신

```
private void OnTriggerEnter2D(Collider2D other)
{
    if (other.CompareTag("Player") == false)
        return;

    int damage = fixedDamage;

    if (useOwnerAttackStat && owner != null)
        damage = owner.Attack;

    CombatEvent ev = new CombatEvent
    {
        Type = EventType.DamageEvent,
        Amount = damage,
        Position = other.transform.position
    };

    CombatSystem.Instance.ToPlayer(ev);
}
```

# 전투 이벤트에 대한 피드백

CoreMerge의 전투 피드백은 타격의 순간, 피격된 대상의 반응, 수치 정보 전달이 한 흐름에서 자연스럽게 이어지도록 설계  
전투 피드백을 크게 두 레이어로 나누어 구성 : 신체적 반응 레이어(Flash) | 정보 표현 레이어(Popup UI)

## 시스템 구조

1

### MonsterController — 신체적 반응 레이어 피드백

몬스터가 피격 시 잠시 하얀색으로 깜빡이는 화이트 플래시 효과를 적용

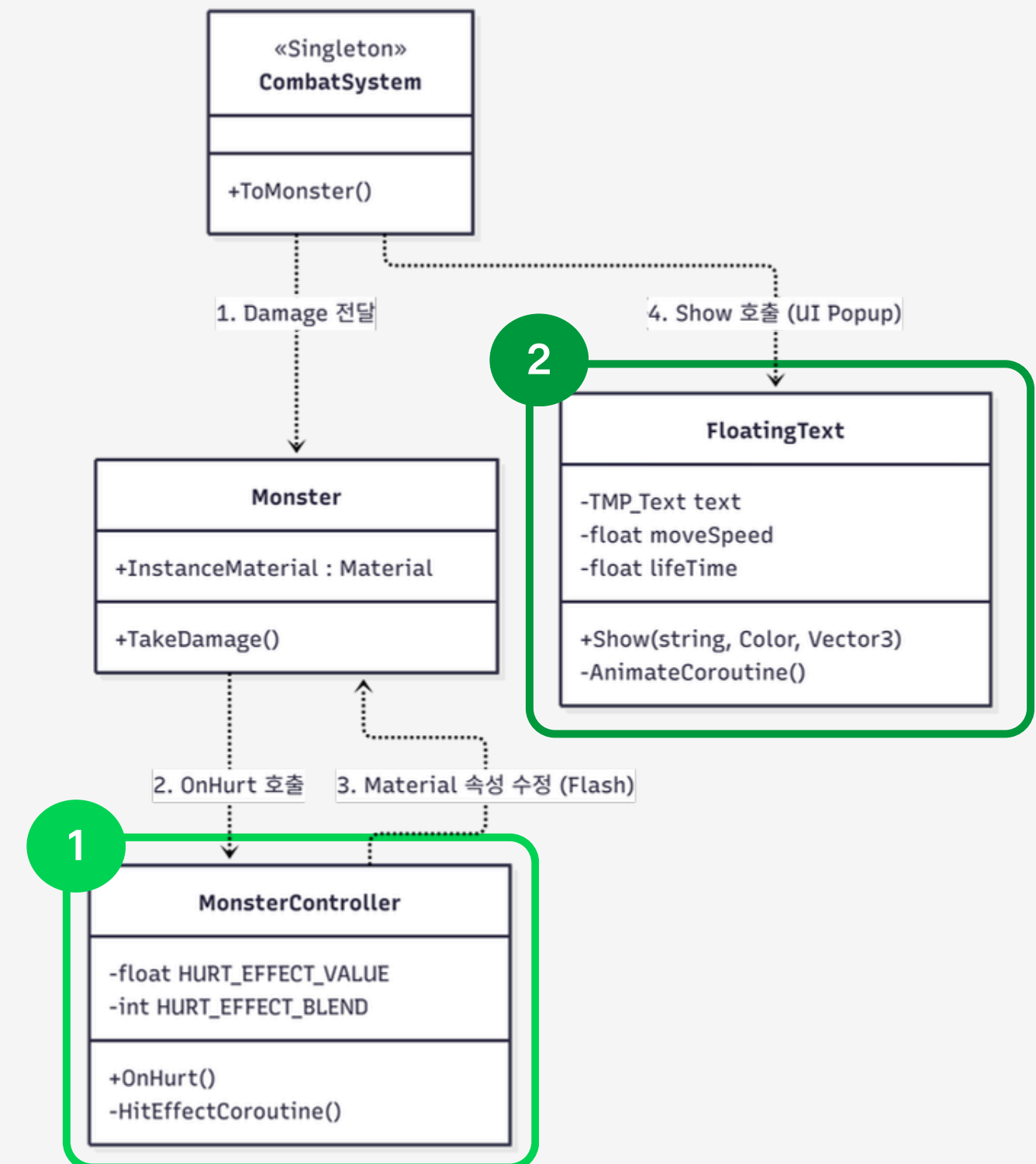
- Shader Property 제어 : \_HitEffectBlend라는 쉐이더 속성을 코드를 통해 실시간으로 수정
  - HURT\_EFFECT\_VALUE : 피격시 발동되는 화이트 플래시의 강도
  - HitEffectCoroutine을 호출해 화이트 플래시 효과를 코루틴으로 제어해 잠시동안 반짝이도록 연출
- FSM 연동 : Hurt 상태로 전이하여 공격 판정을 즉시 취소하고 행동을 멈추는 물리적 피드백을 담당

2

### FloatingText — 정보 표현 레이어

피격 대상의 머리 위에 입은 데미지의 수치가 표현되도록 정보 전달

- 월드 공간 UI인 FloatingText를 제작해, 타격 대상의 HeadUpPivot (머리 위 지점)을 인자로 받아 해당 좌표에 데미지 수치를 띄움
- AnimateCoroutine을 통해 텍스트의 이동과 투명도 변화를 독립적으로 처리해 자체 애니메이션 효과를 연출
- 지속 시간(lifeTime)이 종료되면, 스스로 객체를 파괴하여 메모리를 관리하는 구조



# 전투 이벤트에 대한 피드백

## 실제 구현

### 화이트 플래시 효과



- 코루틴을 활용해, 즉시 \_HitEffectBlend 값을 0.25f로 설정해 몬스터를 하얗게 반짝이게 만듦
- 피격 시간(hurtDuration) 동안 Mathf.Lerp를 통해 값을 다시 0.0f로 보간하며 연출을 마무리

```
private IEnumerator HitEffectCoroutine()
{
    float waitTime = 0.0f;

    while (waitTime < hurtDuration)
    {
        float currentBlendValue = Mathf.Lerp(HURT_EFFECT_VALUE, 0.0f, waitTime / hurtDuration);

        owner.InstanceMaterial.SetFloat(nameID: HURT_EFFECT_BLEND, currentBlendValue);

        yield return null;
        waitTime += Time.deltaTime;
    }

    owner.InstanceMaterial.SetFloat(nameID: HURT_EFFECT_BLEND, 0.0f);
}
```

### Floating Text



- CombatSystem에서 데미지를 처리할 때 FloatingText를 생성
- 전투 정보를 포함한 채 Show 함수를 호출해 화면에 표현

```
if (actualDamage > 0 && damagePopupPrefab != null)
{
    var ftObj: FloatingText = Instantiate(damagePopupPrefab, monster.HeadUpPivot.position, Quaternion.identity);
    var ft = ftObj.GetComponent<FloatingText>();

    Color textColor = combatEvent.IsCritical ? Color.yellow : Color.white;
    string message = actualDamage.ToString();

    if (combatEvent.IsCritical)
        message = "CRIT!\n" + message;

    ft.Show(message, textColor, combatEvent.Position);
}
```

# 주요 시스템 & 기능 상세 설명

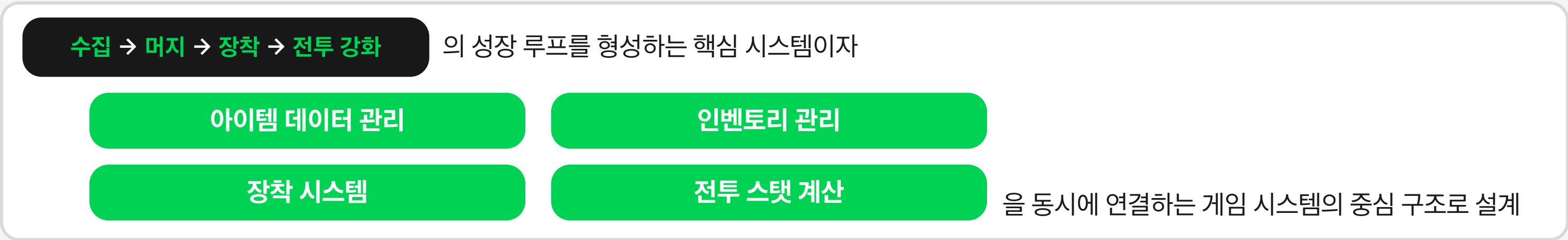
## 코어 / 인벤토리

- 코어 · 코어 관리 시스템
- 퀵슬롯 시스템 (장착)
- 인벤토리 시스템 (보관 · 머지)
- 퀵슬롯 / 인벤토리 데이터 저장



# 코어 및 인벤토리 시스템 개요

CoreMerge에서 **Core**는 플레이어의 전투 능력을 결정하는 핵심 성장요소  
플레이어는 전투 중 코어를 획득하고, 동일한 코어를 머지(Merge)하여 상위 티어 코어로 발전시키며, 이를 퀵슬롯에 장착해 자신의 전투 스타일을 구성



Core의 데이터는 **Scriptable Object 기반 구조**를 선택해 구현

Why?

ScriptableObject는 Unity 에디터에서 데이터 에셋을 독립적으로 관리할 수 있으며, 코드 수정 없이 새로운 데이터를 추가할 수 있다는 장점  
새로운 코어를 추가할 때도 에셋 하나만 추가하면 시스템에 자동으로 통합

→ 짧은 프로젝트 기간동안 빠른 데이터 추가와  
스탯 변경 등의 밸런스 테스트를 위한 선택

인벤토리 시스템

코어 머지(Merge)와 티어 기반 정렬(Sort) 기능을 보유

퀵슬롯 시스템

퀵슬롯에 장착된 코어는 플레이어의 스탯에 즉시 반영

설계 목표를 수립



- 불필요한 계산을 제거
  - 데이터 변경 시에만 갱신
  - UI와 플레이어 시스템이 이벤트를 통해 느슨하게 연결
- : 코어 데이터는 한명만 알자
  - : 매 프레임 확인하지 말자
  - : UI로 변경하면 이벤트로 플레이어에게 반영하자



# 코어 / 코어 관리 시스템

CoreMerge의 코어 시스템은 **코어 자체의 정의된 정보와 플레이 중 변하는 보유/장착 정보를 분리해** 관리하는 데이터 중심 구조로 설계

- 코어의 이름, 티어, 아이콘, 스탯 보정치처럼 변하지 않는 정보는 정적 데이터(DB) 로 관리
- 인벤토리와 퀵슬롯처럼 플레이 도중 계속 변하는 정보는 런타임 데이터로 별도 관리

## 시스템 구조

1

### CoreData

각 코어의 고정 정보를 정의하는 데이터 객체

코어의 ID, 이름, 티어, 아이콘, 스탯 보정치를 포함해 "이 코어가 무엇인지"를 설명하는 데이터

2

### CoreDBManager

모든 CoreData를 중앙에서 관리하는 데이터베이스 매니저

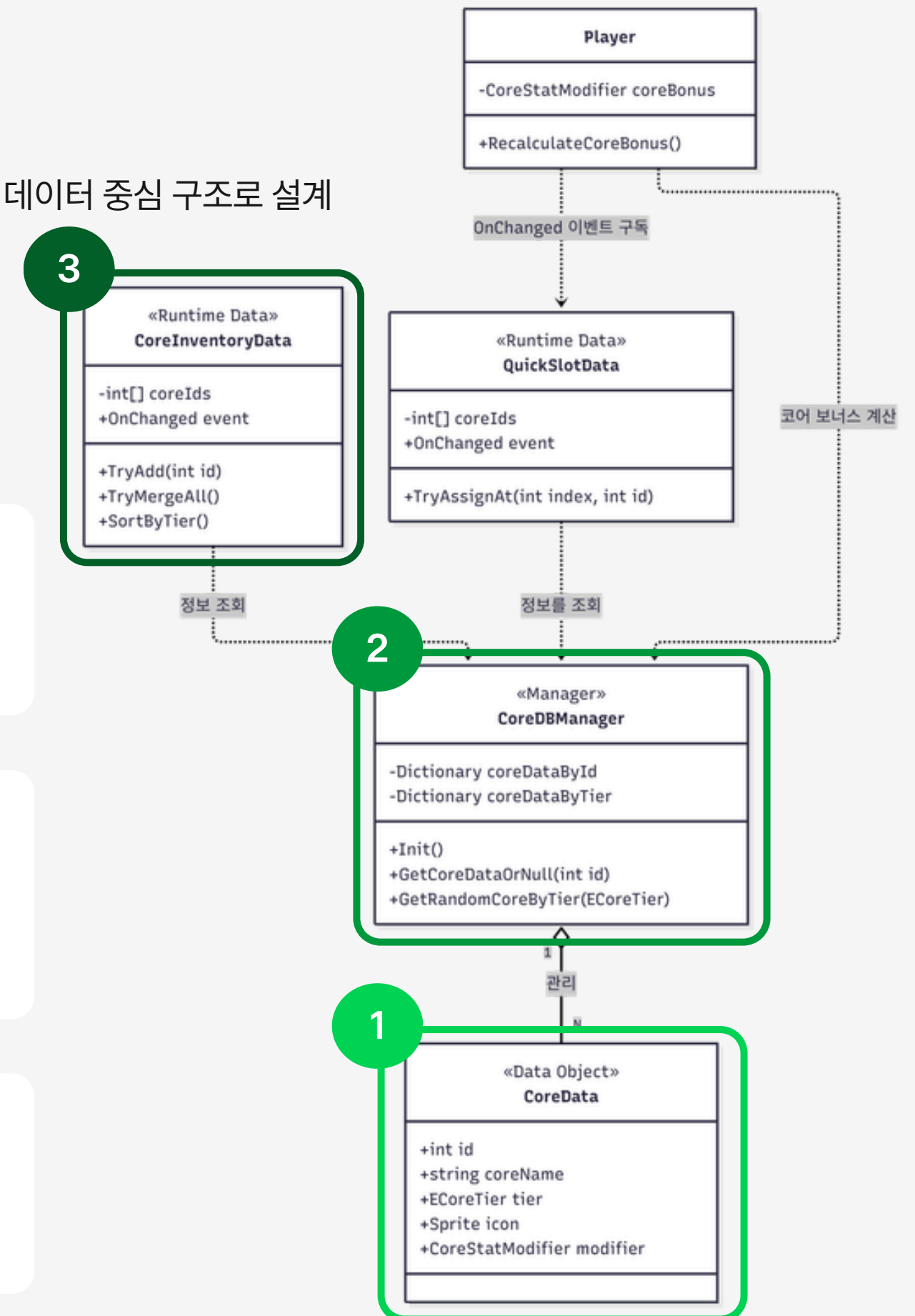
초기화 시점에 전체 코어 데이터를 로드하고, 이를 ID 기준 / 티어 기준 Dictionary로 캐싱  
이후 인벤토리, 퀵슬롯, 플레이어 스탯 계산은 모두 이 매니저를 통해 필요한 코어 정보를 조회

3

### CoreInventoryData

플레이어가 현재 보유 중인 코어 목록을 관리하는 런타임 데이터

인벤토리는 코어의 ID 배열만 관리하며 "무엇을 가지고있는가"만 알고 있고, 실체는 DB에 질의해서 가져옴



# 코어 / 코어 관리 시스템

## 시스템 구조

4

### QuickSlotData

플레이어가 실제로 장착해 효과를 받고 있는 코어 구성을 관리하는 런타임 데이터

인벤토리와 마찬가지로 코어의 ID 배열만 관리

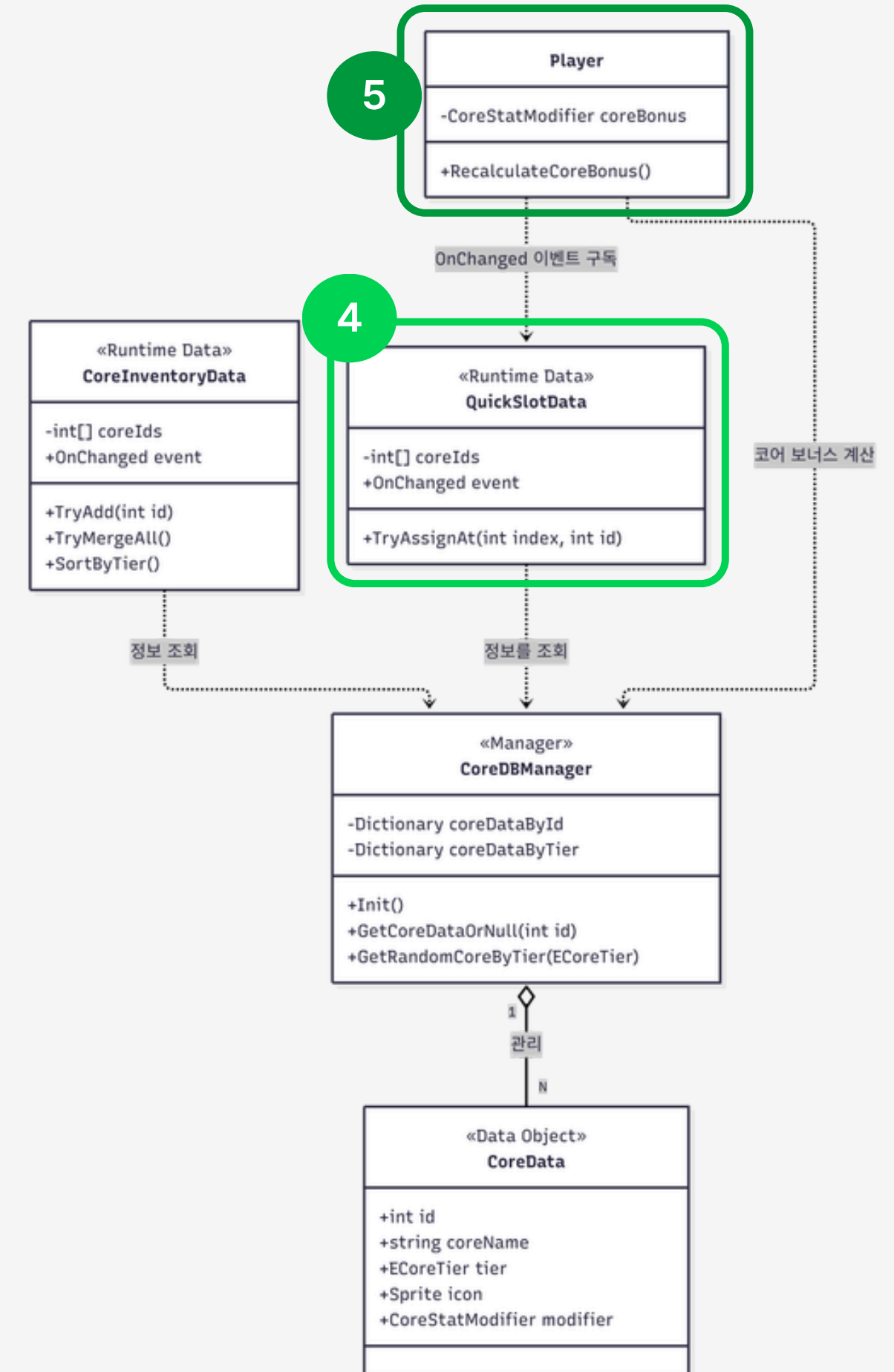
슬롯 변경 시 변경 이벤트(OnChanged) 를 발생시켜 플레이어 스탯이나 UI가 즉시 반응할 수 있도록 설계

5

### Player

퀵슬롯 데이터의 변화를 구독하고, 현재 장착된 코어 ID들을 기준으로 최종 보너스를 계산

코어를 직접 소유하고 관리하는 주체는 아니지만, 장착 결과를 실제 전투 수치에 반영하는 영향을 받는 계산 계층



# 코어 / 코어 관리 시스템

## 핵심 설계 포인트

### 정적 데이터와 동적 데이터를 분리한 이유

#### 1. 정의와 상태를 분리

- **정의** : 코어의 이름, 아이콘, 스탯 보정치 등 — 게임 중에 변하지 않는 정보
- **상태** : 인벤토리에 어떤 코어가 들어있는지, 어떤 슬롯에 장착했는지 — 플레이어 행동에 따라 계속해서 달라지는 정보

▶ 서로 다른 성격을 가진 데이터들을 한 구조에서 관리하게 되면, 인벤토리나 퀵슬롯이 불필요하게 무거워지거나 동일한 코어 데이터가 여러 곳에 중복 저장될 가능성이 크다고 판단

#### 2. 역할의 확실한 분리

- **DB** : 코어의 정의를 보관하자
- **인벤토리** : 보유 여부만 기록하자
- **퀵슬롯** : 장착 여부만 기록하자
- **플레이어** : 장착 결과를 전투 수치에 반영하자

### 정적 데이터 관리 방식

#### CoreData

하나의 코어를 설명한 정의이자, 전투시스템에게는 코어 효과의 원본 데이터



#### CoreDBManager

CoreData를 ID기준, 티어 기준으로 Dictionary로 캐싱해 빠른 조회와 랜덤 드랍, 스탯 계산 등 데이터 반영을 위한 **중앙 데이터 허브**

### 런타임 데이터 관리 방식

무거운 객체를 들고 있지 말자! : 오직 ID만

#### CoreInventoryData

#### CoreDBManager

- 이 슬롯에 **어떤 코어 ID가 들어있는가?**만 알고 있음
- 런타임에서 변하는 코어 상태를 최소 정보로 관리하고, 변경 사실을 시스템 전체에 전파하는 계층

# 코어 / 코어 관리 시스템

## 데이터 흐름

### 1. 데이터 로드 및 캐싱

- Scriptable Object를 활용해 작성한 CoreData는 데이터 추가 / 변경이 용이
- 게임 시작 시 CoreDBManager가 모든 CoreDataSO를 로드
- CoreDBManager는 로드된 Core 정보들을 ID / Tier 기준으로 캐싱

### 2. 코어 습득 및 장착

- 플레이어가 코어 습득 시, 해당 코어의 ID만을 CoreInventoryData에 등록
- 인벤토리의 코어를 퀵슬롯에 배치하면, 선택된 코어의 ID가 QuickSlotData에 등록됨

### 3. 변경 이벤트 발생

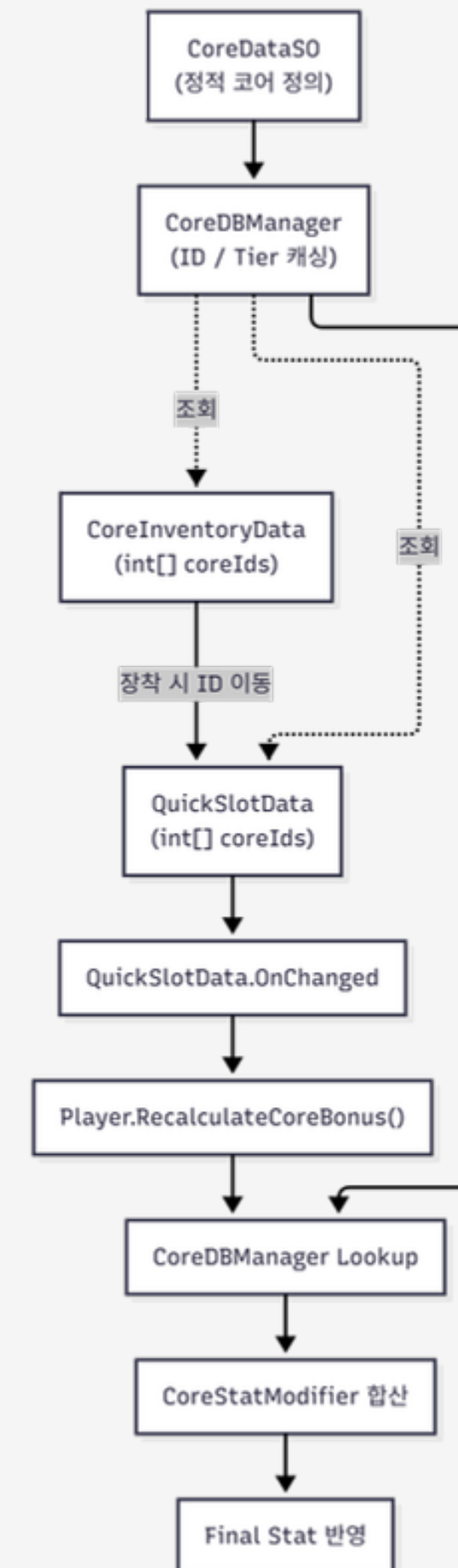
- 퀵슬롯 구성이 바뀌는 순간 QuickSlotData.OnChanged 이벤트가 호출
- 현재 장착 상태가 바뀌었다는 사실을 Player / UI에 전달하는 공통 갱신 신호

### 4. 플레이어 보너스 재계산

- Player는 OnChanged 이벤트를 구독하고 있다가, 변경이 발생할 경우 재계산 로직의 함수를 호출
- 이 과정에서 퀵슬롯에 들어있는 ID를 순회하고, DB를 통해 각 코어의 스탯 정보를 조회해 합산

### 5. 최종 스탯 반영

- 합산된 코어 보너스는 플레이어의 최종 공격력, 최대 체력, 기타 전투 스탯 계산에 반영



1

2

3

4

5

# 프로젝트 회고 및 향후 계획

당면했던 문제들

---

배우게 된 점

---

앞으로

---

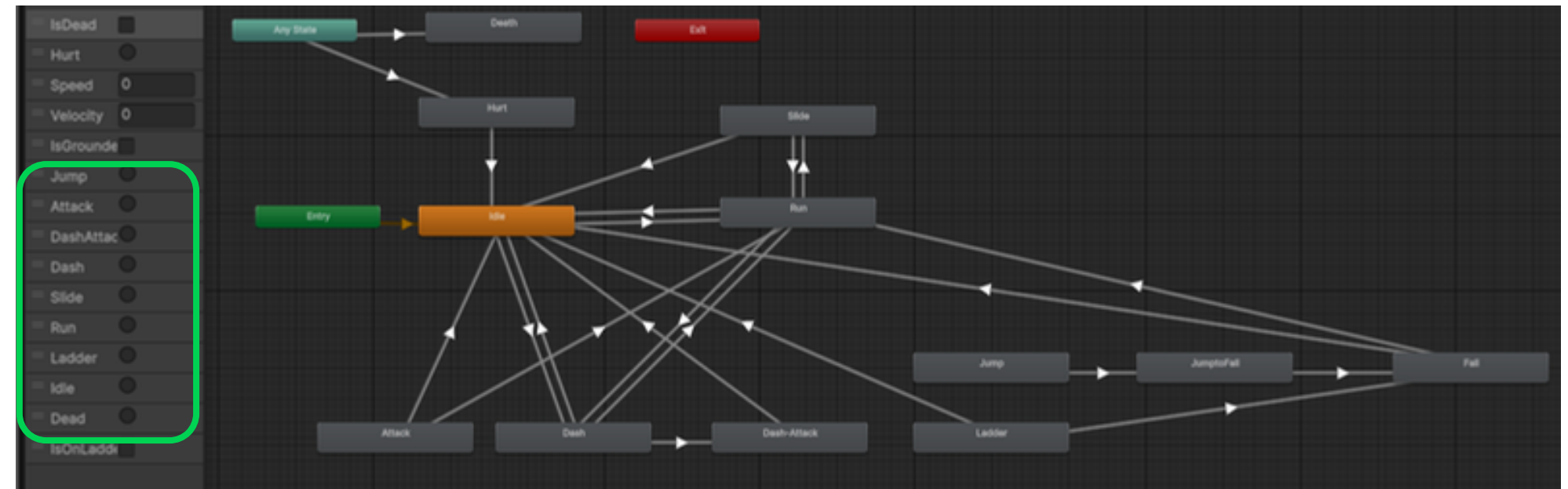


## 1. 상태 패턴 기반의 애니메이션 제어로 인한 애니메이션 트리거 누적 및 상태 동기화 이슈

### 구현 방법 및 의도

#### State Machine이 애니메이션 흐름을 직접 주도하도록 설계

- 플레이어의 액션 수가 많아지고, 피격 / 대시 / 공격 / 사망처럼 우선 순위가 다른 상태들이 겹치기 시작하면 Transition 조합만으로 흐름을 제어하기가 복잡해짐
- 따라서 상태 판단과 전이 결정은 State Machine이 전담하도록, 그리고 Animator는 그 결과를 재생하는 **[표현 계층]** 처럼 사용  
→ 즉, **어떤 상태로 들어갈 것인가는 코드에서 결정**하고  
**어떤 애니메이션을 보여줄 것인가만 Animator가 담당**
- Animator를 로직 판단 주체로 사용하기보다, State Machine의 상태 단위와 애니메이션 단위를 1:1 대응으로 가깝게 연결하려는 의도



### 경험했던 문제

#### Trigger는 이벤트에 적합했지만, 지속 상태를 Trigger 단위로 관리하자 잔여 Trigger와 동기화 문제가 발생

- Trigger가 본질적으로 한 번 발생하는 사건을 표현하는 파라미터라는 점을 고려하지 못했음
- 공격 시작이나 피격, 사망과 같은 이벤트 단위에는 적합했지만, Run, Ladder처럼 일정 시간동안 유지되어야 하는 상태까지 Trigger 중심으로 관리하면서 구조적 한계가 드러나게 됨

#### 가장 먼저 체감된 문제는 잔여 Trigger 문제

- 빠르게 상태 전환 시 이전 상태에서 설정한 Trigger가 완전히 소비되기 전에 다음 상태로 넘어갈 수 있었음
- 남은 Trigger가 뒤늦게 소비되면서 의도하지 않은 애니메이션 전이



## 1. 상태 패턴 기반의 애니메이션 제어로 인한 애니메이션 트리거 누적 및 상태 동기화 이슈

### 해결 방법

- 상태 진입(Enter) 시점에 ResetTrigger()로 **잔여 Trigger를 명시적으로 초기화**하는 가드 로직을 추가
- 프로젝트에서는 MonsterController 등에서 ResetAnimTrigger() 형태로 공용화해, 빠른 전이에서도 오작동을 억제

### 배운 점

**Animator도 독립적인 상태 머신이므로, 상태 표현과 이벤트 표현을 같은 방식으로 다루면 안된다는 것**

- FSM이 Animator를 제어한다고 해서 Animator가 단순한 재생기처럼만 동작하는 것은 아니라는 점을 가장 크게 느낌
- 실제 런타임에서 Animator는 소비 타이밍, 현재 클립 진행도 등 자체 규칙을 가진 독립적인 상태 머신으로 동작

**즉, 문제의 본질은 단순히 Trigger를 많이 사용한 것이 아니라, 이벤트성 전이와 지속 상태 표현을 동일한 파라미터 성격으로 다루었다는 점**

### TO-BE

이후에는 애니메이션 파라미터 설정의 기준을 더 명확히 가져갈 필요가 있다고 판단

- Attack, Hurt, JumpStart 처럼 한 번 발생하는 행동은 이벤트성 전이로 판단 : **Trigger**
- Idle, Run, Ladder처럼 현재 상태를 계속 설명해야 하는 값은 지속 상태 표현으로 판단 : **Bool / Float / Int**

## 2. Rigidbody2D 기반 물리 제어의 불안정성

### 선택 이유

프로토타입 완성을 우선하기 위해, 가장 익숙한 Rigidbody2D + FSM 방식을 채택

- 프로젝트 초반에는 DOTween 같은 보간 기반 이동 제어도 고려했음
- 특히 Dash / 사다리 오르기같은 정밀한 위치 제어가 필요한 액션은 물리 기반 이동보다 트윈 기반 방식이 더 직관적일 수 있다고 생각

하지만 제한된 기간 안에 완성된 프로토타입을 안정적으로 만드는 것을 우선순위로, 가장 숙련도가 높고 상태머신 구조와 연계가 자연스러운 Rigidbody2D + FSM 조합을 선택

### 얻은 장점

익숙한 물리 기반 구조 덕분에 빠르게 프로토타입을 구성

- Rigidbody2D 기반 제어의 가장 큰 장점 : Unity의 기본 물리 시스템을 활용해 이동 / 점프 / 낙하 / 충돌 반응 같은 기본적인 액션 구조를 빠르게 구축할 수 있었음
- State Machine을 결합하면서 각 State 안에서 필요한 물리 제어를 명시적으로 추가하는 식으로 빠른 확장 가능

### 경험했던 문제

사다리 오르기(Climb) 진입 시 캐릭터의 위치가 상단으로 튕거나, 대시(Dash) 중 가속도가 갑자기 급발진하는 등 물리 연산의 불연속적인 튜 현상 발생

- 사다리 오르기 진입 시 위치가 상단으로 튕는 현상
  - 대시 중 가속도가 갑자기 폭발하는 현상
- 상태 머신은 분명하게 상태를 바꾸고 있었지만, 물리 엔진 내부에는 이전 상태의 힘과 충돌 정보가 계속 남아있었음

즉, 상태 기반 액션 제어와 연속적인 물리 시뮬레이션이 서로 다른 기준으로 움직인 결과에 가까웠다고 생각

### 배운 점

물리 기반 제어는 빠른 구현에는 용이하지만,  
정밀한 액션 상태에서는 상태별 물리 개입을 명시적으로 차단할 필요가 있음

### 적용한 방어 코드

- 물리 계층 일시적 무시 (캐릭터 / 지면 사이의 물리 마찰과 반발 간섭 제거)
- 중력 강제 제어 (중력을 0으로 강제)
- 속도 명시적 초기화 (linearVelocity를 Vector2.zero로 강제)

2. Rigidbody2D 기반 물리 제어의 불안정성

TO-BE

향후에는 DOTween 기반의 위치 제어를 일부 도입해, 정밀한 액션 상태의 제어를 더 견고하게 만들 예정

현재의 Rigidbody2D + FSM 구조는 프로토타입 완성 측면에서 매우 용이했지만 의도된 연출을 방해하는 방향으로 작동하는 경우도 있었음

→ 모든 이동을 DOTween으로 바꾸기 보다는 Dash / 사다리 오르기 / 특수 이동 등 목표 위치와 이동 시간이 명확한 상태에 한해 트윈 기반 제어를 선택적으로 적용

- 이동 궤적을 더 예측 가능하게
- 충돌/중력/잔여 속도에 덜 휘둘리도록
- 상태 전환 연출을 더 안정적으로 제어

| 구분     | As-Is (Rigidbody2D 기반)         | To-Be (DOTween 기반)                      |
|--------|--------------------------------|-----------------------------------------|
| 제어방식   | linearVelocity를 통한 힘/속도 제어     | DoMove를 통한 목표 위치 (Transform) 제어         |
| 움직임    | 물리 엔진의 마찰/중력에 의한 가변적 움직임       | 수학적으로 정의된 Ease Curve 기반의 확정적 움직임        |
| 방어 로직  | 중력 0 처리, 충돌 무시 등 물리 엔진과의 사투 필요 | 물리 연산 자체를 생략하므로 물리 오류 원천 차단             |
| 연출 확장성 | 가감속 연출 구현이 복잡하고 디버깅이 어려움       | 탄성(Back), 바운스(Bounce) 등 풍부한 연출 즉시 적용 가능 |

## 프로젝트 진행을 하며 배우고 느끼게 된 점

### 기술적 성장 — 상태 패턴의 실무 적용 감각

- 기존에도 FSM로 캐릭터를 제어한 경험은 있었지만, 이 정도로 다양한 상태를 지닌 규모의 State를 다룬 적은 없었기에 State Pattern을 적용해 구현해봄
- 플레이어/보스의 행동을 독립 State 클래스로 분리하면서, **문제 발생 시 해당 상태만 추적**하면 되는 구조의 힘을 체감
- 특히 복잡한 연속 액션에서도 관심사 분리(SoC)가 디버깅 속도와 안정성에 직접 영향을 준다는 점을 명확하게 느낌

### 의사 결정 — R&D와 마감 기한의 균형

- DOTween 같은 신규 기술을 도입하고 싶었지만, 제한된 기간 안에 완성된 프로토타입을 확보하기 위해 가장 숙련된 방식(Rigidbody2D + FSM)을 선택
- 기술적 호기심보다 **완성/납기**를 우선한 결정을 통해 프로젝트 완성과 업무 일정을 지키는 결과를 도출

## 앞으로 — 향후 개발 계획

### DOTween 적용을 통한 액션 및 UI 고도화

- 제약: 현재의 물리 기반 이동은 완급 조절(Ease) 구현이 복잡하고 디버깅 비용이 큼
- 계획: 보스의 돌진 공격에 'Ease 함수'를 적용하여 박진감을 더하고, 코어 획득/합성 UI에 '탄성 효과(Elastic Effect)'를 추가하여 보상 획득 시의 타격감 제공

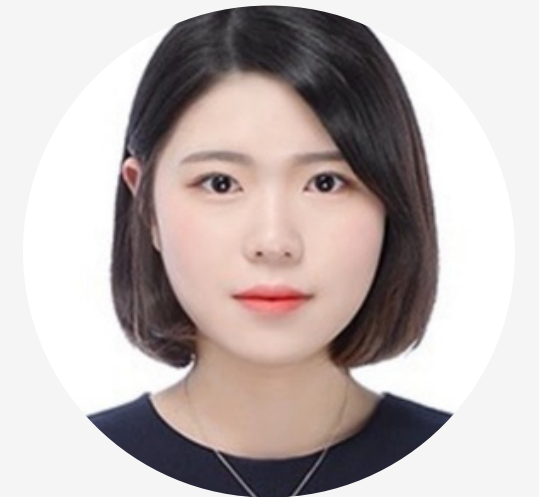
### 최종 티어 코어 기반의 '특수 스킬' 시스템 구현

- 현재 코어 합성의 결과가 단순 '스탯 상승'에 머물러 있어 최종 성취감이 다소 부족함
- 최고 등급(Legendary) 코어 장착 시, 코어 속성에 따른 '고유 액티브 스킬'이 해금되는 시스템을 추가 → 플레이어에게 '최종 병기를 얻기 위한 전략적 합성'이라는 명확한 동기부여를 제공

### 애니메이션 시스템 리팩토링 (Playable API 도입)

- 제약: Animator Controller의 화살표가 많아지며 높아지는 트랜지션의 복잡성
- 계획: 트리거 방식에서 탈피하여 유니티 Playable API를 활용, 애니메이션 전이 로직을 100% 코드로 관리함으로써 Animator 에셋에 대한 의존도를 낮춰보는 방법을 도입

# THANK YOU FOR READING.



**김주향 Ju Hyang KIM**  
Unity Client Programmer

---

 jjudykim0@gmail.com  010. 2421. 7343

---

 <https://github.com/jjudykim>

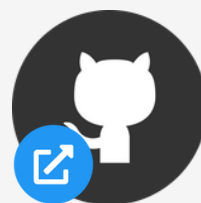
---



# PROJECT 02



시연 영상



Github

## CoreMerge

Unity 2D를 활용한 액션 플랫폼머 + 아이템 머지 로직의 게임

**프로젝트 기간** 2025.11 - 2025.12 (10일)

**개발 인원** 1인 개발

**주요 기술** State Pattern + FSM, Scriptable Object

본 프로젝트 CoreMerge는 전투에서 획득한 **코어**를 머지(조합) 및 장착하여 전투력을 강화하고, 이를 반복하며 성장하는 전투 - 수집 - 성장 루프의 2D 횡스크롤 액션 게임입니다.

콘텐츠 확장과 밸런스 조정 과정에서도 게임 루프가 안정적으로 유지되도록, 코어의 능력치와 필드 드랍 정보를

**Scriptable Object** 로 분리해 데이터로 관리하는 구조로 구현했습니다. 이를 통해 코어 추가, 수치 조정, 드랍 테이블 변경을 로직 수정없이 진행할 수 있습니다.

또한 플레이어의 연속 액션과 보스의 패턴/페이지 전환처럼 전환 로직이 복잡한 구간은 **상태 패턴 (State Pattern)** 을 적용해 구현하여, 예외 상황에서도 동작이 흔들리지 않도록 제어 흐름을 정리했습니다. 전투 판정 이후의 UI 갱신과 연출 피드백은 이벤트로 연결해 시스템 간 직접 의존도를 줄이고, 수정 범위가 명확한 클라이언트 구조를 목표로 개발했습니다



## 당면했던 문제점들

### 1. 상태 패턴 기반의 애니메이션 제어로 인한 애니메이션 트리거 누적 및 상태 동기화 이슈

- ◆ 선택 이유 (Why: Logic-First Design)
  - 제어권 단일화: Animator 전이 조건에 의존하지 않고 State 코드가 흐름을 주도하도록 설계
  - 관심사 분리(SoC): "상태 판단/전이"는 C#, "표현(재생)"은 Animator가 담당하게 해 결합도를 낮추기 위함
  - 복잡도 관리: 플레이어 다수 상태 + 보스 페이지즈를 Transition 그래프로만 관리할 때 생기는 복잡한 전이를 피하려는 선택
- ◆ 증상 (Symptom)
  - FSM으로 애니메이션을 제어할 때, 짧은 시간에 상태가 여러 번 전이되면 남아 있던 Trigger 때문에 다음 상태에서 엉뚱한 애니메이션이 재생되는 현상이 발생
- ◆ 원인 추적 (Root Cause)
  - Unity 애니메이터의 Trigger는 전이가 실제로 발생할 때까지 활성 상태로 유지
  - FSM이 로직적으로 상태를 강제로 바꾸면, 트리거가 "소비되지 못한 채" 남아 다음 전이에 영향을 주는 케이스가 발생
- ◆ 해결 (Fix)
  - 상태 진입(Enter) 시점에 ResetTrigger()로 잔여 Trigger를 명시적으로 초기화하는 가드 로직을 추가
  - 프로젝트에서는 MonsterController 등에서 ResetAnimTrigger() 형태로 공용화해, 빠른 전이에서도 오작동을 억제
- ◆ 개선 가능한 방향 (Next Improvements)
  - Trigger 의존도 축소: Trigger 대신 Stateld(Int/Enum) 파라미터 중심 전환으로 누적 리스크를 낮출 수 있음
  - 명시적 재생(CrossFade/Play): 조건 전이보다 "지금 이 클립을 재생"하는 방식이 오작동 방지에 유리
  - 동기화 규칙 통일: State 종료 기준(애니메이션 이벤트/정규화 시간/입력 락)을 정해 Desync를 줄일 수 있습니다.
  - Animator 구조화: Locomotion/Combat/Hit/Dead 등으로 서브 스테이트 머신/레이어 분리해 그래프 복잡도를 낮출 수 있음
- ◆ 장점 (Pros: Scalability & Precision)
  - SRP 준수: 각 State가 Enter에서 "어떤 애니메이션을 재생할지"를 책임져 코드 흐름이 명확해짐
  - 확장 용이: 신규 액션 추가 시 기존 전이 그래프 수정 부담이 줄고, State + 최소 트리거로 통합이 쉬워짐
  - 디버깅 용이: "현재 실행 State"만 보면 의도한 애니메이션이 무엇인지 빠르게 매칭 가능
- ◆ 단점 및 리스크 (Cons & Risks)
  - Trigger 누적(Trigger Pollution): 빠른 상태 전환 시 소비되지 않은 Trigger가 남아 오작동 전이 발생 가능
  - 논리-애니메이션 불일치(Desync): 코드 State와 실제 클립 진행이 어긋나면 끊김/미끄러짐 같은 체감 문제가 생길 수 있음

# 프로젝트 회고 및 향후 계획

## 당면했던 문제점들

### 2. Rigidbody2D 기반 물리 제어의 불안정성 해결

- ◆ 선택 이유 (Why: Logic-First Design)
  - 신기술(DOTween) 도입에 대한 욕심이 있었으나, 프로젝트 기간 내에 '완성된 프로토타입'을 제공하기 위해 가장 숙련도 높은 방식(Rigidbody2D + FSM)을 선택함
- ◆ 증상 (Symptom)
  - 사다리 오르기(Climb) 진입 시 캐릭터의 위치가 상단으로 튕거나, 대시(Dash) 중 가속도가 갑자기 급발진하는 등 물리 연산의 불연속적인 튕 현상(Jittering) 발생
- ◆ 원인 추적 (Root Cause)
  - 물리 충돌 간섭: 사다리 오르기 도중 캐릭터의 콜라이더와 지형지물의 마찰력/반발력이 충돌하여 수직 위치가 강제로 보정되는 현상
  - 중력/속도 누적: 대시나 점프 연계 시 물리 엔진 내부의 linearVelocity가 특정 프레임에서 중첩되거나, 프레임 드랍 발생 시 델타 타임 보정 과정에서 속도가 폭발적으로 증가하는 현상
- ◆ 해결 (방어적 코드 설계)
  - 물리 계층 일시 무시: ClimbState 진입 시 Physics2D.IgnoreLayerCollision을 사용하여 캐릭터와 지면의 물리적 마찰을 일시적으로 차단
  - 중력 강제 제어: 사다리나 대시 상태 진입 시 gravityScale = 0을 적용하여 물리 엔진의 중력 간섭을 무효화
  - 속도 명시적 초기화: 상태 전이 시점(Enter)마다 rigidBody.linearVelocity = Vector2.zero를 호출하여 이전 상태의 관성이 현재 상태에 영향을 주지 못하도록 차단
- ◆ 개선 및 확장 계획 : DOTween을 활용한 위치 제어 최적화
  - 현재의 Rigidbody2D 기반 방식은 '완성된 프로토타입'을 빠르게 구현하는 데는 유리했으나, 고도로 세밀한 액션 연출에서는 물리 엔진의 '예외 상황'을 방어하는 데 많은 비용이 소모되었음
  - 향후 DOTween을 도입하여 개선 예정

| 구분     | As-Is (Rigidbody2D 기반)         | To-Be (DOTween 기반)                      |
|--------|--------------------------------|-----------------------------------------|
| 제어방식   | linearVelocity를 통한 힘/속도 제어     | DoMove를 통한 목표 위치 (Transform) 제어         |
| 움직임    | 물리 엔진의 마찰/중력에 의한 가변적 움직임       | 수학적으로 정의된 Ease Curve 기반의 확정적 움직임        |
| 방어 로직  | 중력 0 처리, 충돌 무시 등 물리 엔진과의 사투 필요 | 물리 연산 자체를 생략하므로 물리 오류 원천 차단             |
| 연출 확장성 | 가감속 연출 구현이 복잡하고 디버깅이 어려움       | 탄성(Back), 바운스(Bounce) 등 풍부한 연출 즉시 적용 가능 |

# 프로젝트 회고 및 향후 계획

## 배운 점과 다음 단계

### 배운 점

#### ◆ 기술적 성장 — 상태 패턴의 실무 적용 감각

- 플레이어/보스의 행동을 독립 State 클래스로 분리하면서, 문제 발생 시 “해당 상태만 추적”하면 되는 구조의 힘을 체감
- 특히 복잡한 연속 액션에서도 관심사 분리(SoC)가 디버깅 속도와 안정성에 직접 영향을 준다는 점을 명확하게 느낌

#### ◆ 의사결정(Trade-off) — R&D와 마감 기한의 균형

: DOTween 같은 신규 기술을 도입하고 싶었지만, 제한된 기간 안에 완성된 프로토타입을 확보하기 위해 가장 숙련된 방식(Rigidbody2D + FSM)을 선택

: “기술적 호기심”보다 “완성/납기”를 우선한 결정을 통해 프로젝트 완성과 업무 일정을 지키는 결과를 도출

#### ◆ 다시 한다면? — 물리와 연출의 역할 분리

: 물리 기반 이동이 아닌 연출 목적의 이동/효과(UI/보스 돌진 등)는 Rigidbody보다 Tween 계열로 분리했다면, 더 부드럽고 명확한 UX를 만들 수 있었을 것이라 생각

### 다음 단계

#### 1. DOTween 통합을 통한 액션 및 UI 고도화

- \* 제약: 현재의 물리 기반 이동은 완급 조절(Ease) 구현이 복잡하고 디버깅 비용이 큼
- \* 계획: 보스의 돌진 공격에 'Ease 함수'를 적용하여 박진감을 더하고, 코어 획득/합성 UI에 '탄성 효과(Elastic Effect)'를 추가하여 보상 획득 시의 타격감 제공

#### 2. 최종 티어 코어 기반의 '특수 스킬' 시스템 구현

- \* 제약: 현재 코어 합성의 결과가 단순 '스탯 상승'에 머물러 있어 최종 성취감이 다소 부족함
- \* 효과: 최고 등급(Legendary) 코어 장착 시, 코어 속성에 따른 '고유 액티브 스킬'이 해금되는 시스템을 추가 → 플레이어에게 '최종 병기를 얻기 위한 전략적 합성'이라는 명확한 동기부여를 제공

#### 3. 애니메이션 시스템 리팩토링 (Playable API 도입)

- \* 제약: Animator Controller의 화살표가 많아지며 높아지는 트랜지션의 복잡성
- \* 계획: 트리거 방식에서 탈피하여 유니티 Playable API를 활용, 애니메이션 전이 로직을 100% 코드로 관리함으로써 Animator 에셋에 대한 의존도를 낮추고 런타임 디버깅 편의성을 획기적으로 높이는 방향으로 개선

# 시스템 아키텍처

## 전체 시스템 구조

### 1. 전역 매니저 계층 (Global Managers)

: 프로젝트의 모든 시스템을 중앙에서 관리하고 초기화 순서를 제어하는 루트 매니저 (Root Manager)

- 싱글톤 패턴을 통해 어디서든 하위 매니저에 접근할 수 있는 통로를 제공
- 게임 실행 시 모든 하위 모듈들의 생성 및 초기화 보장

#### Input Manager

- 유니티 클래식 Input 클래스를 활용해 입력 상태를 추상화하고, 게임 상태에 따라 입력 활성화 여부 제어
- 입력 처리와 플레이어 행동을 분리해 관리하기 위함

#### UI Manager

- HUD, 인벤토리, 씬 전환 연출(SceneFader) 등 UI 요소의 생명 주기와 바인딩 관리
- UI는 게임 로직을 직접 건드리지 않고, 필요한 데이터는 이벤트로 전달받음
- 새로운 씬이 로드될 때마다 UI가 현재 활성화된 플레이어의 데이터를 실시간으로 참조하도록 연결

#### Game Manager

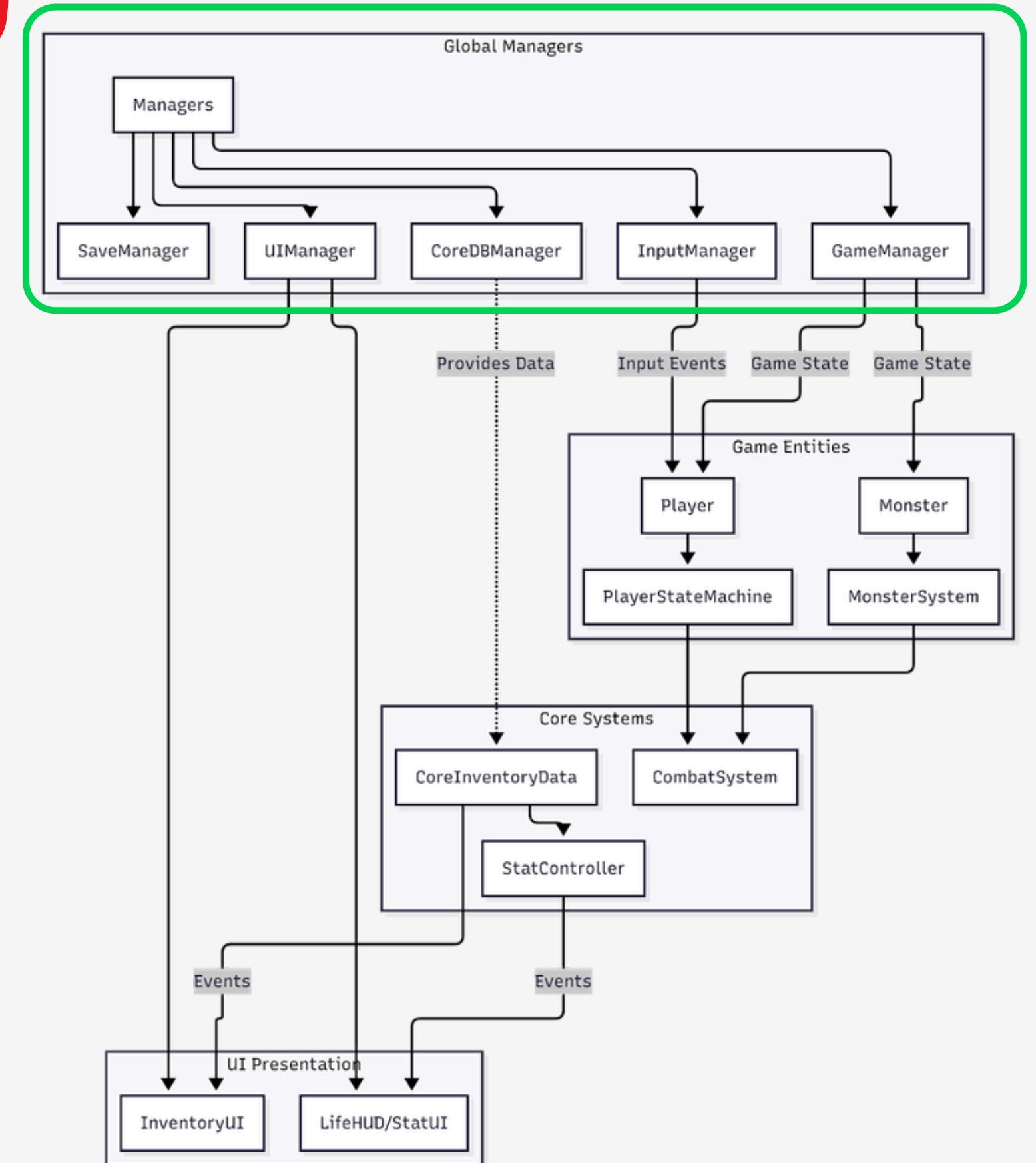
- 플레이어 스폰, 코어 드랍, 게임의 승리/패배 등 전체적인 게임 플레이 흐름의 규칙을 집행

#### CoreDBManager

- Resources 폴더 내의 모든 CoreDataSO를 로드해 런타임 데이터 디렉토리를 구축
- 모든 CoreData를 로드한 뒤, ID 기반의 Dictionary로 캐싱해 빠른 검색을 지원
- 티어별로 리스트를 그룹화해 몬스터 등급에 따른 랜덤 드랍 로직을 지원

#### SaveManager

- 현재 진행 중인 스테이지 정보와 인벤토리, 퀵슬롯의 데이터를 로컬 파일로 직렬화하여 저장 및 복구
- JsonUtility를 활용해 SaveData 객체를 JSON 문자열로 변환하고 저장
- 데이터를 불러올 때에는 CoreInventory와 QuickSlots 매니저에 데이터를 직접 주입해 게임 상태 복원



# 시스템 아키텍처

## 전체 시스템 구조

### 1. 전역 매니저 계층 (Global Managers)

: 프로젝트의 모든 시스템을 중앙에서 관리하고 초기화 순서를 제어하는 루트 매니저 (Root Manager)

- 싱글톤 패턴을 통해 어디서든 하위 매니저에 접근할 수 있는 통로를 제공
- 게임 실행 시 모든 하위 모듈들의 생성 및 초기화 보장

#### Input Manager

- 유니티 클래식 Input 클래스를 활용해 입력 상태를 추상화하고, 게임 상태에 따라 입력 활성화 여부 제어
- 입력 처리와 플레이어 행동을 분리해 관리하기 위함

#### UI Manager

- HUD, 인벤토리, 씬 전환 연출(SceneFader) 등 UI 요소의 생명 주기와 바인딩 관리
- UI는 게임 로직을 직접 건드리지 않고, 필요한 데이터는 이벤트로 전달받음
- 새로운 씬이 로드될 때마다 UI가 현재 활성화된 플레이어의 데이터를 실시간으로 참조하도록 연결

#### Game Manager

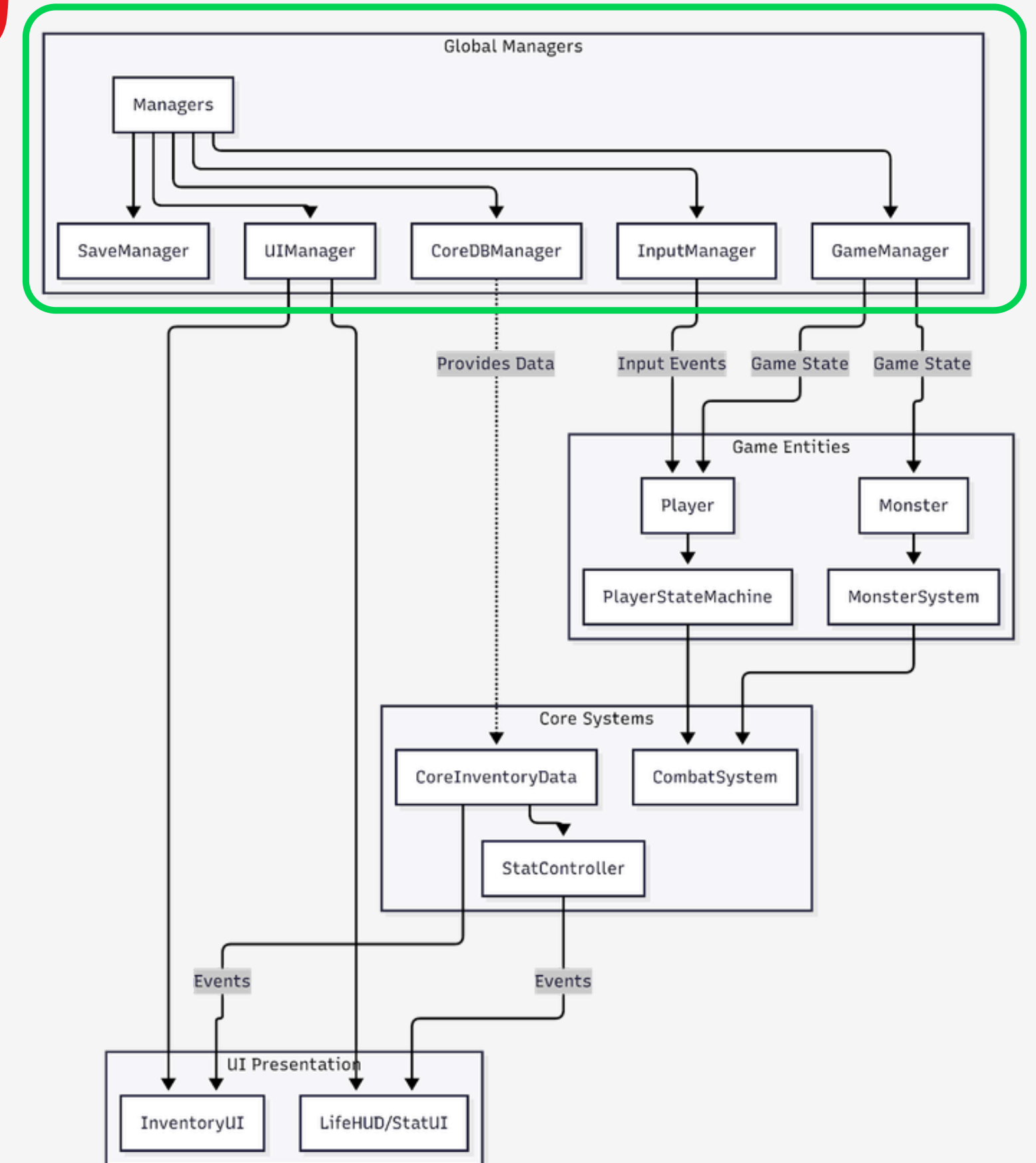
- 플레이어 스폰, 코어 드랍, 게임의 승리/패배 등 전체적인 게임 플레이 흐름의 규칙을 집행

#### CoreDBManager

- Resources 폴더 내의 모든 CoreDataSO를 로드해 런타임 데이터 디렉토리를 구축
- 모든 CoreData를 로드한 뒤, ID 기반의 Dictionary로 캐싱해 빠른 검색을 지원
- 티어별로 리스트를 그룹화해 몬스터 등급에 따른 랜덤 드랍 로직을 지원

#### SaveManager

- 현재 진행 중인 스테이지 정보와 인벤토리, 퀵슬롯의 데이터를 로컬 파일로 직렬화하여 저장 및 복구
- JsonUtility를 활용해 SaveData 객체를 JSON 문자열로 변환하고 저장
- 데이터를 불러올 때에는 CoreInventory와 QuickSlots 매니저에 데이터를 직접 주입해 게임 상태 복원





# 시스템 아키텍처

## 전체 시스템 구조

### 2 게임 엔티티 계층 (Game Entities)

프로젝트의 모든 엔티티들은 모두

상태(State)를 통해 무엇을 할지 결정하고, 전투 시스템(Combat)을 통해 결과를 확인한다

라는 공통된 규칙을 따르고 있음

→ 플레이어는 입력, 몬스터는 AI가 시작점인 차이

#### Player - 입력 기반의 유동적 상태 제어

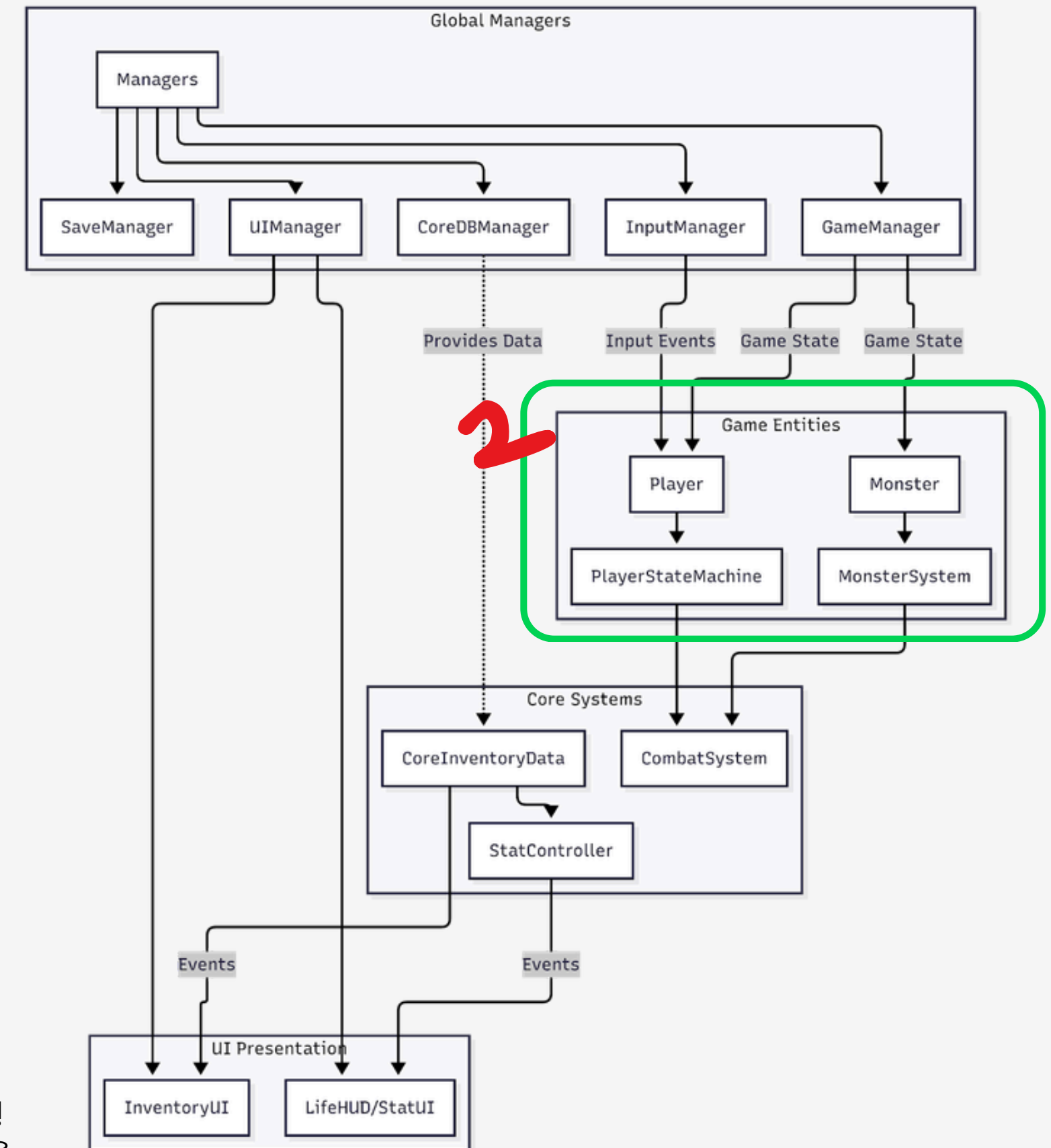
- InputManager (입력 감지) → PlayerController (상태 전이 판단) → PlayerStateMachine (액션 수행) → Combat System (결과 처리)
  - 입력 감지: PlayerController는 매 프레임 입력 값을 체크
  - 상태 전이 : 각 상태 클래스는 특정 입력 / 조건 시에 State Machine를 통해 다음 동작으로 전이
  - 전투 호출 : 니메이션과 동기화된 PlayerHitBox이 적과 충돌 시 PLayer.cs 를 거쳐 데미지 판정을 수행
- 상태 머신을 통해 애니메이션의 흐름과 물리 로직을 완전 일치시켜, 연계 액션을 자연스럽게 구현

#### Monster - 자율형 FSM 기반의 AI

- Monster Controller(FSM) → Monster (Entity) → CombatSystem
  - 의사결정(FSM) : enum을 활용한 유한 상태 머신 구조로, 플레이어와의 거리에 따라 상태를 전환
  - 공격 실행 : 공격 범위 내에 플레이어가 포착되면 개별 몬스터 타입에 맞는 공격 로직을 실행
  - 전투 상호작용 : 피격 시 본체의 TakeDamage를 통해 실제 피해량을 계산하고 결과 반환
- 행동을 결정하는 Controller와 데이터를 관리하는 Monster를 분리, 동일한 AI 로직을 공유하는 다양한 몬스터

#### Boss - 시퀀스 제어 중심의 페이즈 AI

- 페이즈 스케줄링: 보스의 체력 비율을 실시간으로 감시하며, 특정 시점에 PhaseTransitionState를 강제 실행
- 복잡한 패턴 실행 : 돌진 공격, 유도 투사체 발사, 손 소환 등 각각의 패턴이 독립적인 상태로 구현되어 있는 것을 BossController가 패턴 시퀀스를 조율해 실행
- 예외 판정 제어 : Boss.cs는 TakeDamage를 오버라이드해, 패턴 시전 중 무적 처리 등의 정밀한 판정 로직 수행





# 시스템 아키텍처

## 전체 시스템 구조

### 3 코어 시스템 계층 (Core Systems)

코어 시스템은 [인벤토리 데이터] → [퀵슬롯 필터링] → [스탯 컨트롤러 연산] → [UI 이벤트]의 단방향 흐름

#### 데이터 보관 및 관리 (Data Container)

- 인벤토리 데이터 : 획득한 모든 코어의 ID를 관리, 티어 정렬이나 합성같은 데이터 가공 로직 포함
  - 퀵슬롯 데이터 : 플레이어가 현재 장착하여 실시간으로 효과를 보고 있는 코어들의 ID를 보관
- 두 데이터 모두 ItemContainer 인터페이스를 구현해, 내부 데이터가 변경될 때마다 OnChanged 이벤트를 발생

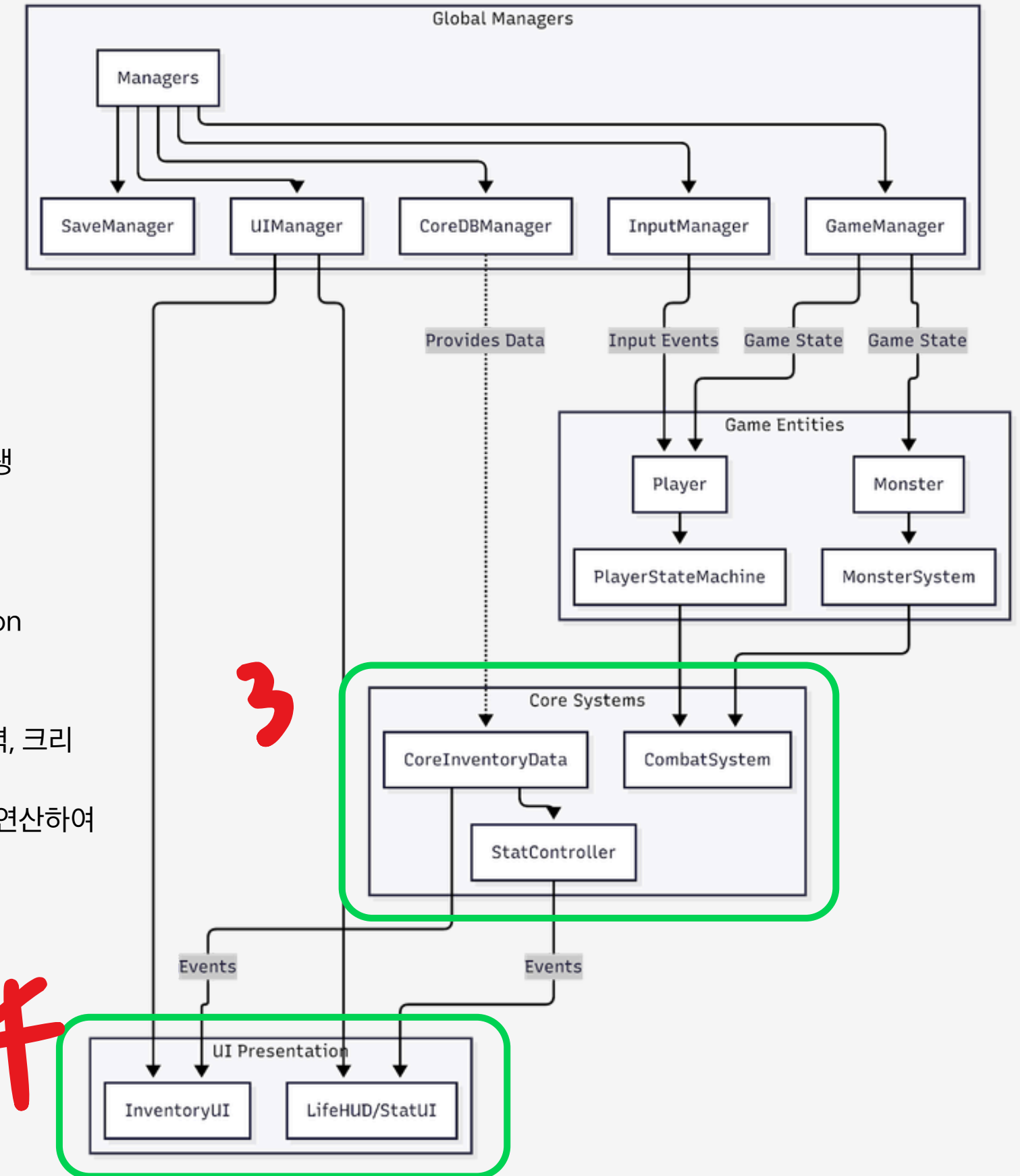
#### 스탯 산출 및 변조 (Stat Modification)

- 퀵슬롯에 담긴 데이터는 Player.cs를 통해 실제 전투 수치로 변환
- 변조 프로세스: QuickSlotData.OnChanged → Player.RecalculateCoreBonus() → FinalStat Calculation
- 핵심 로직
  - 이벤트 구독: Player 클래스는 OnEnable 시점에 퀵슬롯의 OnChanged 이벤트를 구독
  - 보너스 합산: 장착된 코어의 ID를 CoreDB에서 조회하여 각 코어가 가진 CoreStatModifier(공격력, 체력, 크리티컬 등)를 하나의 보너스 객체에 누적 합산
  - 최종 수치 제공: FinalAttack, FinalMaxHp 등의 프로퍼티는 "기본 스탯 + 합산된 보너스"를 실시간으로 연산하여 반환

### 4 UI 표현 계층 (UI Presentation)

최종 계산된 스탯과 데이터의 변화는 플레이어에게 시각적으로 전달

- 반영 프로세스 : Player.NotifyLifeChanged() → LifeHUD / StatUI Update
- 핵심 로직
  - 1. 상태 알림 : 스탯 재계산이나 체력 변화가 발생할 경우 OnLifeChanged 이벤트를 발생
  - 2. 동적 갱신 : LifeHUD와 StatUI는 이 이벤트를 받아 각 변화에 맞는 UI 정보를 갱신
  - 인벤토리 동기화 : InventoryUI는 CoreInventoryData.OnChanged를 구독  
→ 아이템 획득/합성 시 즉시 슬롯 아이콘을 새로고침



# 시스템 아키텍처

## 런타임 플로우

### 5단계 실행 파이프라인 (Execution Sequence)

CoreMerge는 액션 게임에서 중요한 입력 반응성과 행동/애니메이션의 일관성을 확보하기 위해, “한 프레임에 게임이 처리하는 순서”를 5단계로 나누고 각 단계의 책임을 분리

#### ◆ 1) Input Phase — 입력 우선 확보

프레임이 시작되면 InputManager.Update()가 먼저 실행되어, 그 프레임에서 사용할 입력값(이동 방향, 점프 버튼 눌림 등)을 한 번 확정

이렇게 입력을 “스냅샷”으로 고정하면, 프레임 중간에 입력 값이 바뀌어도 다음 단계들이 같은 기준으로 처리되어 입력 누락/지연처럼 느껴지는 흔들림을 줄일 수 있음

#### ◆ 2) Logic Phase — 행동 결정(상태 전이 판단)

다음으로 PlayerController.Update()와 MonsterController.Update()가 실행되며, 지면/사다리/거리 같은 현재 상황을 확인

그 후 상태머신(UpdateLogic())이 “지금은 달리기 상태로 갈지, 점프/공격/피격 상태로 바뀔지”처럼 이번 프레임에 해야 할 행동을 결정

이 단계는 ‘결정’에 집중하며, 실제로 캐릭터를 움직이거나 힘을 주는 물리 적용은 다음 단계에서 처리

#### ◆ 3) Physics Phase — 물리 적용(고정 시간 기반)

FixedUpdate()에서 상태별 UpdatePhysics()가 실행되며, 앞 단계에서 결정된 행동에 맞춰 캐릭터의 속도/이동(예: Rigidbody2D의 속도 설정)을 적용

Update(논리)와 FixedUpdate(물리)를 분리하면, 프레임이 흔들려도 이동/점프 감각이 비교적 일정하게 유지되어 조작감이 안정적

#### ◆ 4) Combat Phase — 판정 및 수치 확정

공격 애니메이션 타이밍에 맞춰 HitBox가 활성화되거나 충돌 이벤트가 발생하면, CombatSystem이 이를 받아 “맞았는지/누가 맞았는지/얼마나 맞았는지”를 계산

이 과정에서 방어력/감쇄를 반영해 최종 데미지(actualDamage)를 확정하고, 그 결과를 HP 감소, 사망 처리, 드랍 발생 같은 게임 데이터 변경으로 반영

#### ◆ 5) Push Phase — 피드백 & 알림 전파

상태가 시작/종료될 때 애니메이션 트리거가 발생해 시각적 연출이 이어지고, HP/스탯/인벤토리처럼 실제로 값이 바뀐 데이터는 이벤트(OnLifeChanged, OnChanged)로 UI에 전달

즉 “전투 결과로 데이터가 바뀌면 → 그 변화만 UI에 알려서 → 화면을 갱신”하는 방식이며, UI가 전투 로직을 직접 호출하지 않고 변화에 반응하도록 만들어 구조가 더 단단해짐

# 시스템 아키텍처

## 런타임 플로우

CoreMerge의 런타임 파이프라인은 "판단은 매 프레임 감시(Polling)하되, 결과의 전파는 사건 발생 시(Push)에만 수행한다"는 명확한 원칙으로 설계  
모든 시스템을 이벤트로만 처리하지 않고, 반응성이 필요한 영역은 Polling, 변화가 드문 데이터는 Push로 나눠 비용과 결합도를 균형 있게 관리

| 구분               | 적용 대상(예)                              | 처리 방식     | 이점                           |
|------------------|---------------------------------------|-----------|------------------------------|
| Polling (Update) | 입력 수집, 지면/사다리 체크, 몬스터 거리 체크, 상태 전이 판단 | 매 프레임 확인  | 즉각 반응성 확보(조작/AI 체감)          |
| Push (Event)     | HP 변화, 인벤토리 변경, 스탯 갱신, 드랍 결과 UI 반영    | 변경 시에만 전파 | 불필요한 연산/참조 오염 감소, UI 결합도 최소화 |

이벤트 기반 (Push / 중심 전략):

- UI & Data: 대부분의 UI 갱신과 데이터 동기화는 이벤트 기반입니다. (예: OnStatChanged, OnHpChanged)
  - 수치가 변할 때만 리스너에게 알림을 보내어 불필요한 Update() 호출을 방지하고 성능을 최적화
- Input: 입력을 매 프레임 체크하는 대신 Input System의 콜백을 사용

업데이트 기반 (Polling / 보조 전략):

- Physics & AI: 캐릭터의 이동, 몬스터의 추적 AI, 물리 연산 등 지속적인 변화가 필요한 로직은 FixedUpdate 및 Update를 통해 매 프레임 계산됩니다.

# 시스템 아키텍처

## 데이터 / 리소스 파이프라인

정적 데이터는 ScriptableObject로, 동적 데이터는 JSON으로 분리하여 관리"하며, 게임의 라이프사이클에 맞춰 전략적으로 데이터를 로드

### 1) 데이터 형태 및 책임 분리

\* 정적 마스터 데이터 (ScriptableObject):

- 파일: CoreDataSO.cs (Resources/SO/Core 경로에 저장)
- 역할: 코어의 이름, 티어, 스탯 보너스(CoreStatModifier) 등 변하지 않는 원천 정보를 보관
- 이점: 유니티 인스펙터에서 기획자가 직관적으로 수치를 수정할 수 있으며, 런타임 메모리 효율이 높음

\* 동적 유저 데이터 (JSON):

- 파일: SaveData.json (Application.persistentDataPath 경로)
- 역할: 플레이어의 현재 스테이지, 인벤토리 내 코어 목록, 퀵슬롯 장착 정보 등 실시간으로 변하는 진행 상태를 저장
- 이점: 가독성이 좋은 텍스트 기반 포맷으로, JsonUtility를 통해 유연하게 직렬화/역직렬화가 가능

### 2) 리소스 관리 및 로딩 전략 (Resources & On-demand)

중앙 집중식 초기 로딩과 필요 시 즉시 로딩(On-demand) 방식을 혼합하여 사용

\* 부팅 시 전체 로드 (Initial Boot):

- Managers.OnInitialize 단계에서 CoreDBManager.Init()이 호출됩니다.
- Resources.LoadAll<CoreDataSO>("SO/Core")를 통해 게임에 존재하는 모든 코어 데이터를 한 번에 메모리에 올리고 ID 기반으로 캐싱 (O(1) 접근성 확보)

\* 씬 진입 및 요청 시 로드 (Scene Entry / Request):

- GameManager.Init() 시점에 Resources.Load<GameObject>("Prefabs/Core")를 통해 필드 드랍용 코어 프리팹을 로드
- UIManager.EnsureHUD()는 HUD가 없을 때만 프리팹을 로드하여 인스턴스화

### 3) 데이터 흐름 및 라이프사이클 (Pipeline Flow)

1. 초기화 (Init): 게임 시작 시 CoreDB가 모든 SO 데이터를 사전(Dictionary)으로 변환하여 준비
2. 데이터 주입 (Inject): SaveManager.LoadGame() 실행 시, JSON에서 읽어온 아이템 ID를 CoreDB에서 조회하여 실제 CoreData 객체를 인벤토리에 주입
3. 데이터 가공 (Process): 플레이어가 코어를 합성하거나 장착하면 CoreInventoryData와 QuickSlotData가 메모리 상에서 업데이트
4. 최종 저장 (Save): 씬 전환 트리거(ChangeSceneTrigger)가 작동하는 순간, 메모리 상의 코어 ID 배열을 JSON으로 직렬화하여 파일로 영구 저장

# 핵심 시스템 1

## 상태 패턴 기반의 엔티티 제어 시스템

### 시스템 개요

#### What 어떤 기능인가?

Player / Boss / Monster가 수행하는 모든 행동을 “한 곳에 몰아넣는 방식” 대신, 행동을 ‘상태(State)’ 단위로 분해하여 관리하는 구조를 적용

상태는 행동의 실행 단위

- 입력 / AI 판단에 따른 의사결정
- 이동 · 점프 · 중력 등 물리 적용
- 공격 판정, 무적/경직 같은 전투 규칙
- 애니메이션 트리거/이펙트 등 표현

즉, 캐릭터의 행동을 “상태별로 격리된 실행 단위”로 관리

#### Why 왜 도입했는가?

##### 1. 복잡도 증가 방지

액션 게임은 “입력/물리/전투/애니메이션”이 동시에 얹히기 쉬워, 이를 한 함수에 처리하면 if-else / switch 구조가 기하급수적으로 커지기 마련  
→ 상태 패턴을 적용해 로직을 **상태별 파일/클래스**로 나누고 변경 범위를 축소

##### 2. 확정적 규칙 제어

[공격 중에는 이동 불가], [피격 중에는 대시 불가] 등과 같은 복잡한 상태 규칙이 정의되어야 하는 경우,  
→ 상태 패턴은 **현재 상태가 무엇인지**를 기준으로 가능한 행동을 제한하므로, 규칙 충돌을 구조적으로 차단할 수 있음

##### 3. 애니메이션과 로직의 동기화 (Sync & Ownership)

상태의 Enter / Exit 시점에서 애니메이션 트리거를 발생시켜, 로직 전이와 표현 전이를 1:1로 매칭되도록 설계  
이 과정에서 Trigger 누적 문제를 경험, Enter()에서 트리거를 정리하는 가드 로직을 추가해 안정성을 확보



# 핵심 시스템 1

## 상태 패턴 기반의 엔티티 제어 시스템

### 시스템 아키텍처

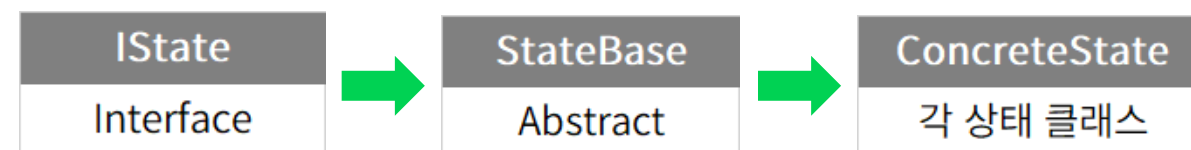
CoreMerge에서는 엔티티의 중요도 / 복잡도 / 개체 수 등, 개체가 지닌 특성에 따라 상태 시스템을 이원화해서 적용

#### Player & Boss

##### 클래스 기반 상태 패턴 (State Pattern)

- 행동이 복잡함
  - 상태마다 고유한 데이터 (타이머 / 거리 / 쿨다운 / 페이즈 값)가 필요
  - 정교한 타이밍 제어 중요
- 상태를 하나의 클래스로 두고, 상태가 자신만의 변수와 로직을 가질 수 있는 구조 채택

##### 구조



##### 장점

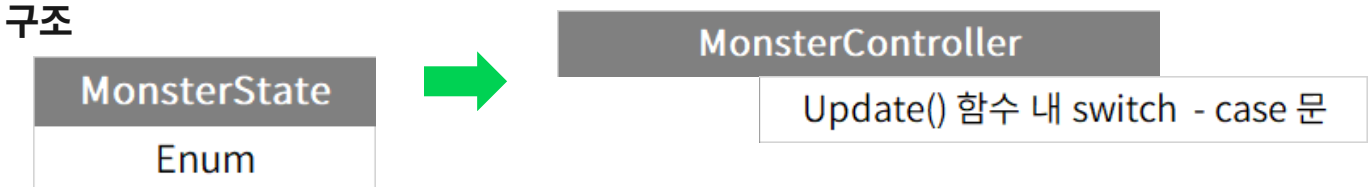
- 상태마다 고유 변수를 안전하게 보유할 수 있음
- 상태별 책임이 명확해 디버깅이 쉬움 (현재 State 클래스명만 보면 됨)
- Enter/Exit를 활용해 애니메이션/무적/히트박스 같은 "상태 시작 · 종료 규칙"을 일관되게 적용

#### 일반 몬스터

##### Enum 기반 유한 상태 머신 (Enum FSM)

- 동시에 여러 마리 등장할 수 있음
  - 각 개체의 행동이 상대적으로 단순함
- 클래스를 다수 생성하는 구조는 과할 수 있어, Enum + switch 기반 FSM 채택 (구현 복잡도 ↓, 성능 부담 ↓)

##### 구조



##### 장점

- 경량 구조로 대량 개체에서 유리 (상태 객체 관리 비용 최소화)
- 구현이 단순해서 유지보수 난이도가 낮음



# 핵심 시스템 1

## 상태 패턴 기반의 엔티티 제어 시스템

### 시스템 아키텍처

구성 요소와 책임 분리 → Context / StateMachine / State

#### State

행동의 결정 (Logic)

행동의 실행 (Physics / Combat)

행동의 표현 (Animation / UI)

행동의 결정(Logic)과  
실행(Physics/Combat),  
그리고 표현(Animation/UI)을 분리하되,  
상태 단위로 일관되게 묶어 제어

#### 1) 컨텍스트(Controller / Context)

- 역할: 엔티티가 가진 Unity 컴포넌트(Animator, Rigidbody2D 등)와 런타임 데이터(입력값, 체력, 방향 등)의 "단일 소유자"
- 책임
  - 입력/센서(지면 체크 등) 결과를 수집해 State에게 제공
  - Rigidbody/Animator 등 엔진 컴포넌트 접근을 캡슐화(상태는 직접 접근하지 않음)
  - "현재 상태가 판단에 필요한 데이터"를 한 곳에서 관리(참조 오염 방지)

#### 필요

- 상태가 엔진 컴포넌트를 직접 쥐면 결합도가 높아짐 → Context를 거치면 상태는 "데이터를 읽고 행동을 요청"하는 구조가 되어 테스트/유지보수가 쉬워짐

#### 2) 매니저(StateMachine)

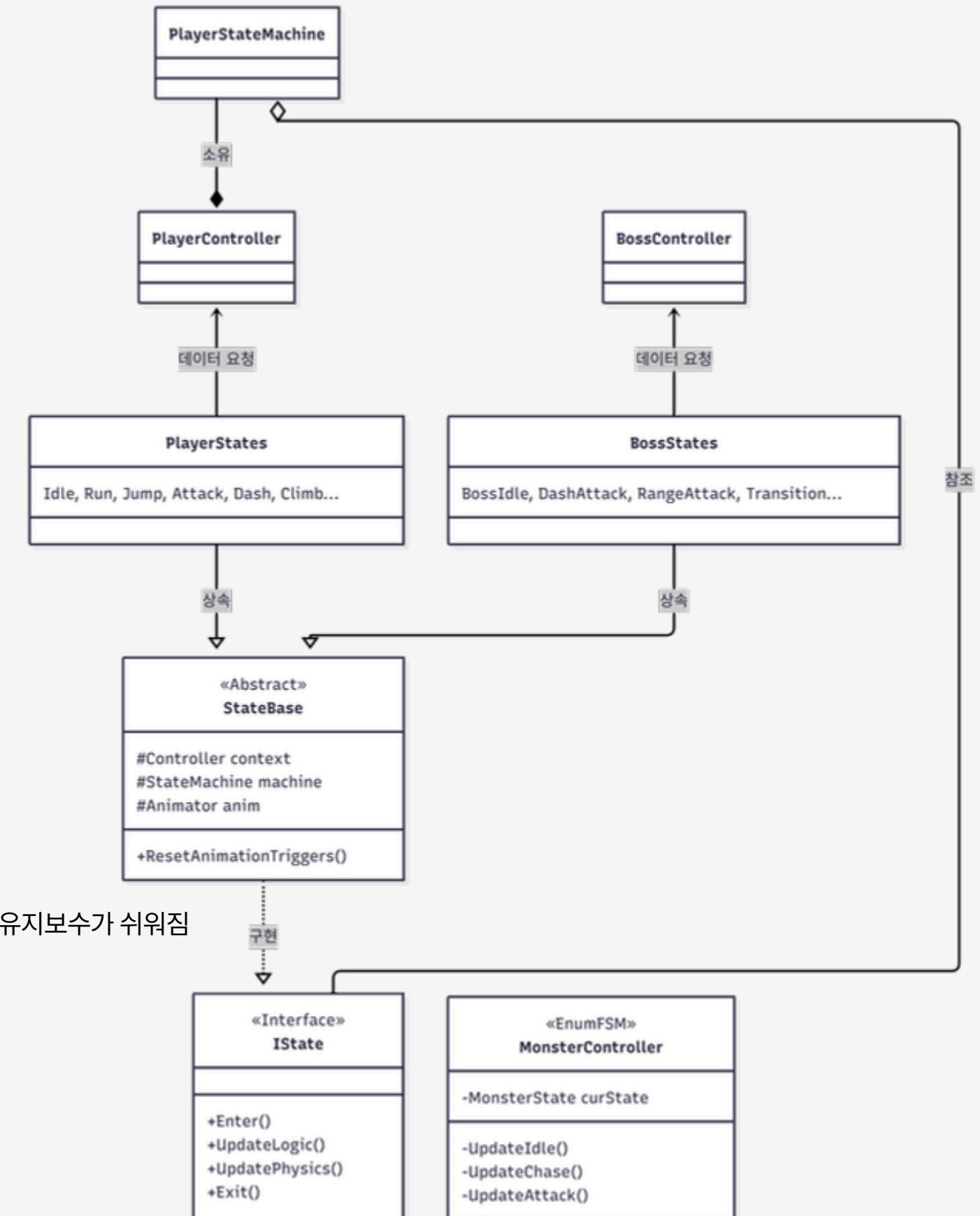
- 역할: 현재 상태 보관 + 상태 전이의 안전한 실행자

#### 책임

- CurrentState 관리
- 상태 변경 시 Exit() → Enter() 순서 보장
- 매 프레임 UpdateLogic() / 물리 틱 UpdatePhysics() 호출 분리

#### 필요

- 상태 전이는 버그가 가장 많이 생기는 지점이라, 전이를 StateMachine만 수행하게 제한해 "누가 언제 상태를 바꿨는지" 추적을 용이하게 하고 규칙 누락(Exit 미호출 등)을 방지



# 핵심 시스템 1

## 상태 패턴 기반의 엔티티 제어 시스템

### 시스템 아키텍처

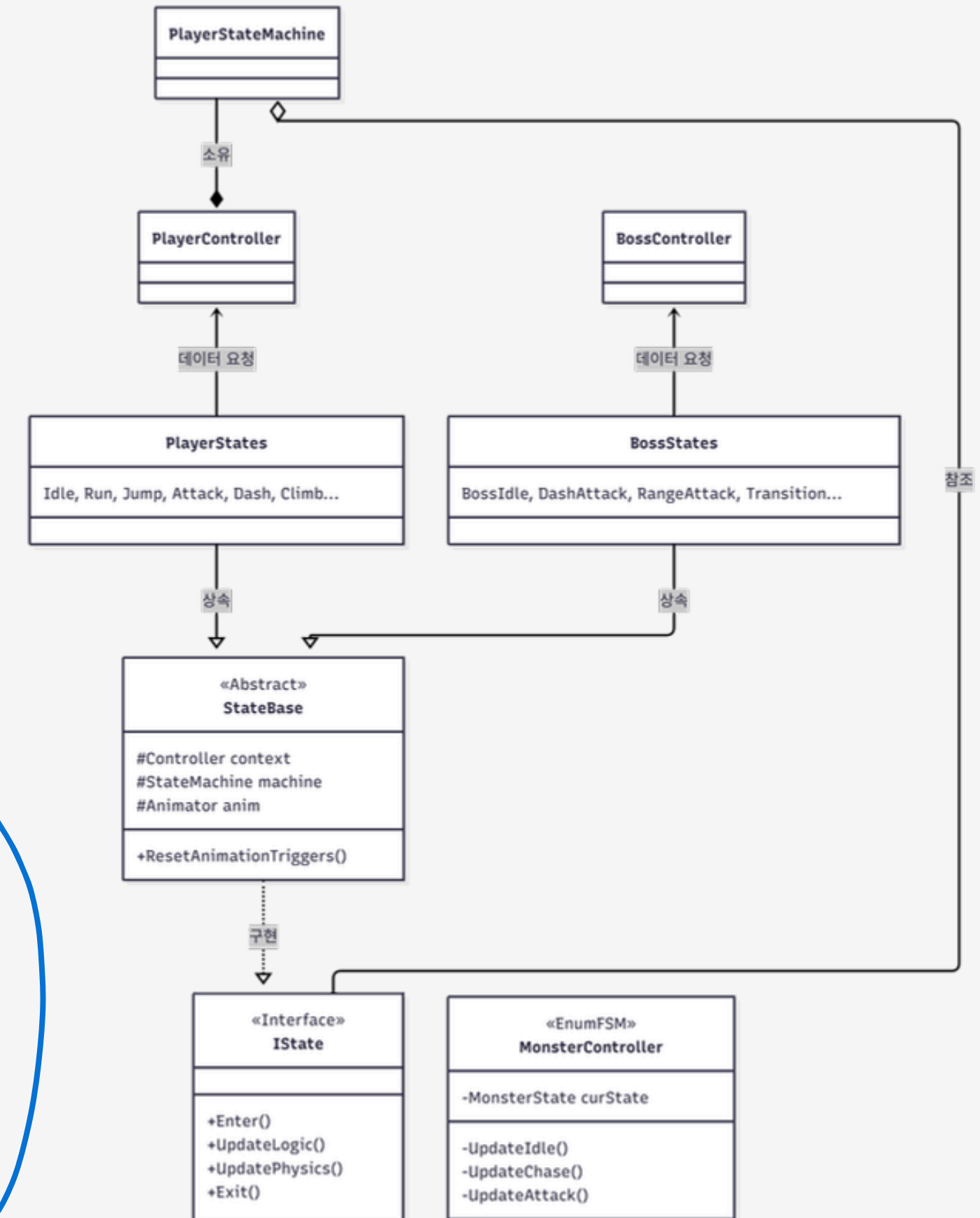
#### 3) 상태 클래스(Concrete State)

- 역할: 실제 게임 로직의 "실행 단위"
- 책임
  - Enter: 상태 진입 시 초기화(타이머/무적/애니메이션 트리거 등)
  - UpdateLogic: 입력/조건을 보고 전이 여부 판단(다음 상태 결정)
  - UpdatePhysics: 이동/속도/점프 등 물리 적용
  - Exit: 상태 종료 정리(플래그 해제, 트리거 정리 등)
- 필요
  - 애니메이션 트리거 누적 문제를 경험했기 때문에, Enter() 시점에 잔여 Trigger를 정리하는 방어 로직을 추가

StateMachine만이 전이를 수행하며, 항상 Exit → Enter 순서로 상태 생명주기를 보장  
Enter → (UpdateLogic 반복 / UpdatePhysics 반복) → Exit → 다음 상태 Enter

Conroller에서 StatePattern의 라이프 사이클을 어떻게 관리하는지  
도식화 + 코드 스니펫 첨부해서 보여주기

추가



# 핵심 시스템 1

상태 패턴 기반의 엔티티 제어 시스템

상세 구현

Player와 Boss의 StateMachine 정의 및 각 Controller와의 연계 스크린샷

구현된 State들 스크린샷

# 핵심 시스템 1

## 상태 패턴 기반의 엔티티 제어 시스템

### 기능 동작 흐름 - Player

#### Example. Player의 '공격' 시퀀스

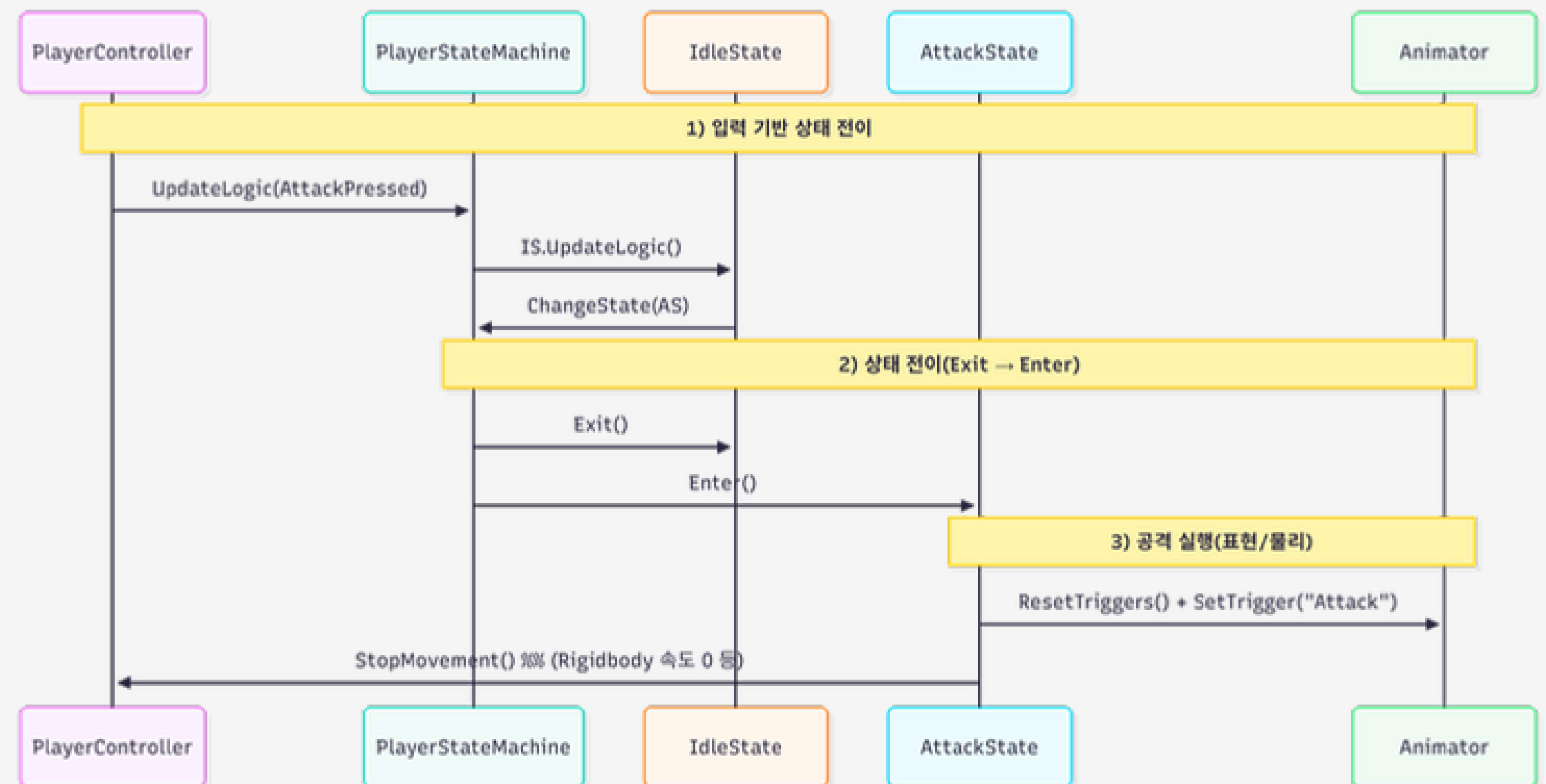
1. Idle 상태에서 입력 감지 및 상태 전이 판단

코드 스니펫 첨부  
IdleState.cs의 UpdateLogic()

2. Attack State로 진입하는 순간

- 애니메이션 트리거
- 물리적이동 제한
- 공격 중임을 확정

코드 스니펫 첨부  
AttackState.cs의 Enter()



# 핵심 시스템 1

## 상태 패턴 기반의 엔티티 제어 시스템

### 기능 동작 흐름 - Boss

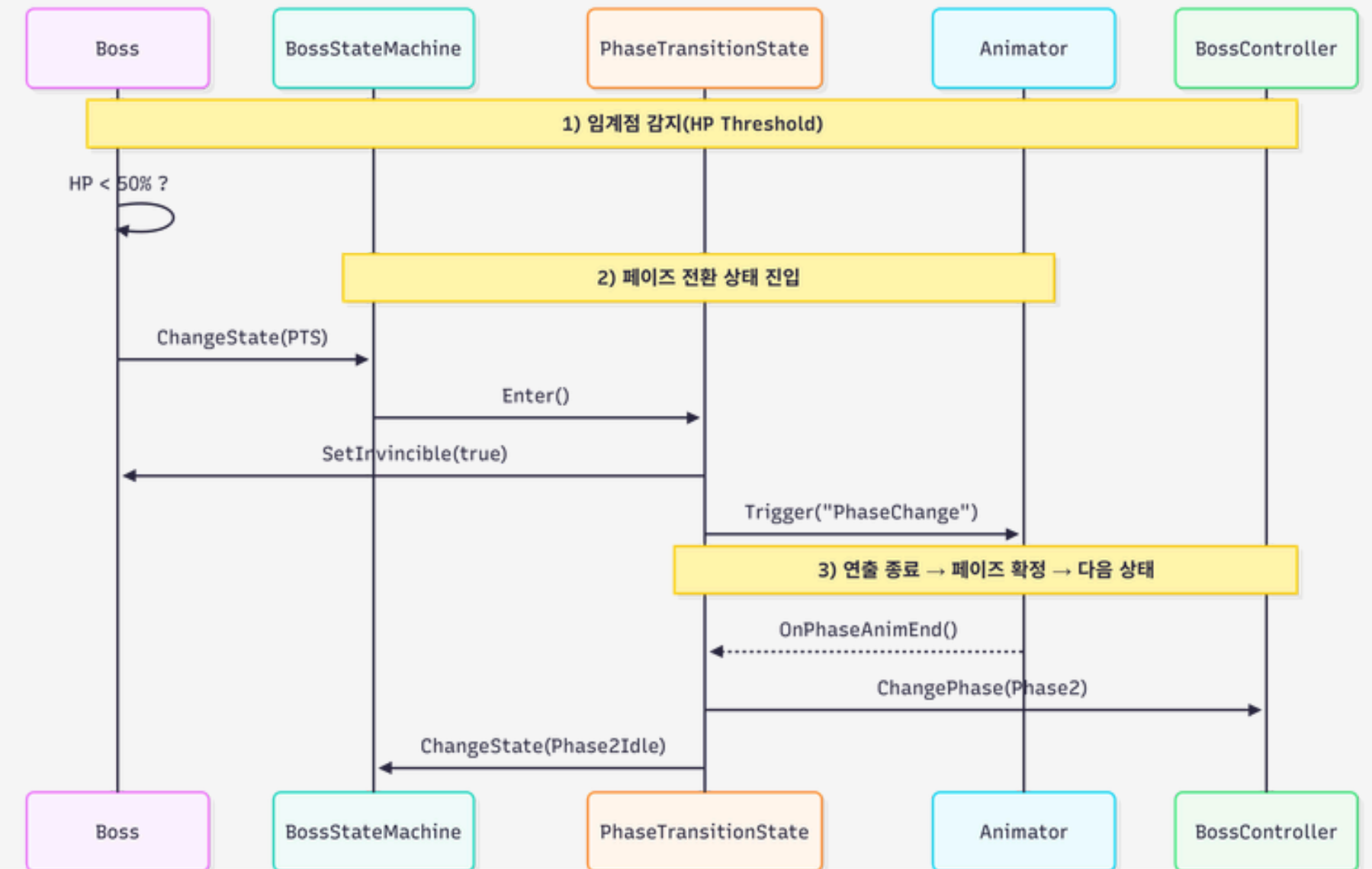
#### Example. Boss의 페이즈 전환

1. 데미지를 입을 때마다 자신의 체력 비율을 체크  
설정된 임계점에 도달할 경우 패턴 중단 후 페이즈 전환 상태로 전이

코드 스니펫 첨부  
Boss.cs의 TakeDamage()

2. 전환 연출 애니메이션 종료 후, 보스 페이즈를 공식적으로 변경  
한층 강화된 2페이즈 전용 AI 상태로 진입

코드 스니펫 첨부  
BossPhaseTransitionState.cs 의 UpdateLogic()





# 핵심 시스템 1

상태 패턴 기반의 엔티티 제어 시스템

Trade-Off와 선택



| 비교 항목 | 대안 A<br>단순 Enum + Switch-Case | 대안 B<br>클래스 기반 상태 패턴   | 대안 C<br>유니티 Playable API / Visual Scripting |
|-------|-------------------------------|------------------------|---------------------------------------------|
| 복잡도   | 낮음<br>하나의 스크립트에 모두 구현         | 중간<br>클래스 분리로 파일 개수 증가 | 높음<br>새로운 API 학습 및 R&D 필요                   |
| 확장성   | 낮음<br>패턴 추가 시 Switch문 비대화     | 매우 높음<br>클래스 추가만으로 확장  | 높음<br>시각적/코드적 유연성 확보                        |
| 성능    | 매우 우수<br>함수 호출 비용 X           | 우수<br>객체 생성 비용 관리 필요   | 중간<br>엔진 오버헤드 존재 가능                         |
| 작업 비용 | 최저<br>즉시 구현 가능                | 보통<br>초기 아키텍처 설계 필요    | 최고<br>R&D 및 러닝 커브 발생                        |

## B : 클래스 기반 상태 패턴을 채택한 이유

프로젝트 제약과 연결 : 제한된 기간 내에 '보스 페이지 전환'과 플레이어의 연계 액션'을 애니메이션 연동과 함께 구현해야 한다는 핵심 목표

- 대안 A는 구현은 빠르지만, 보스의 2페이지 패턴이 추가되는 순간 코드 관리의 난이도가 높아질 것으로 예상
- 대안 B는 각 액션이 독립된 파일로 존재해, 버그 발생 시 해당 파일만 보면 된다는 디버깅 가이드라인이 있어 오류 추적 비용을 낮추는 합리적 선택이라고 선택
- 대안 C는 연출력이 좋지만, 프로젝트 기간 내에 R&D 리스크를 맞닥뜨리기에 위험성이 컸음

→ 검증된 디자인 패턴인 대안 B를 선택함으로써 안정적인 릴리즈를 최우선으로 추구



# 핵심 시스템 2

유연한 확장을 위한 **분리형 인벤토리 시스템**

## 시스템 개요

### What

어떤 기능인가?

인벤토리 시스템은 “아이템 이동”과 “코어 머지”를 처리하는 통합 아이템 관리 시스템

플레이어는 전투로 획득한 코어를 인벤토리에 수집하고, 인벤토리 ↔ 퀵슬롯 간 이동/교체를 수행하며, 동일 등급 코어를 겹쳐 상위 등급 코어로 머지 후 즉시 빌드에 반영

이 시스템이 다루는 상호작용 3가지

- **Move** : 빈 슬롯으로 이동
- **Swap** : 아이템이 있는 슬롯과 교체
- **Merge** : 동일 등급 코어를 겹쳐 상위 등급 생성

### Why

왜 도입했는가?

해당 시스템을 데이터 중심으로 설계한 이유는, 인벤토리가 단순 UI가 아니라 **전투 루프의 전략적인 의사결정(머지 / 장착 / 해제)이 발생**하는 핵심 시스템이기 때문

#### 1. 데이터 무결성 : ‘UI 이벤트’가 데이터를 망가뜨리지 않게

UI가 ‘데이터를 직접 수정하는 구조’라면, 중간 상태에서 데이터가 불완전하게 변경되어 아이템 유실/복제/슬롯 꼬임 등과 같은 버그 발생 가능성  
따라서 데이터가 실제적인 정보를 가지고 있고, UI는 “명령 요청”만, 데이터 변경 결과만 이벤트로 수신하도록 단방향 흐름을 채택

#### 2. 결합도 최소화 : UI 변경 / 슬롯 확장에도 로직은 그대로

핵심 규칙(Move / Swap / Merge)은 순수 데이터 로직으로 고정, UI는 입력 표현과 갱신만 담당  
인벤토리 시스템의 확장성을 위해, UI가 로직과 강하게 엮이지 않도록 “화면이 바뀌어도 시스템은 유지”되도록 설계

#### 3. 복잡한 상호작용 / 머지 로직 관리

인벤토리에서 사용자가 기대하는 동작은, 같은 “드롭”이더라도 상황에 따라 결과가 달라지듯이 복잡성을 가짐

- 대상 슬롯이 비어있음 → Move
- 대상 슬롯에 다른 코어가 있음 → Swap

따라서 상호작용의 판정과 실행을 Controller(중재자) + Model(규칙 실행)에 집중시켜, 하나의 인터페이스에서 발생하는 복잡한 입력을 일관된 규칙으로 처리  
결과는 View(슬롯 UI 등)에만 보여지도록 하는 MVP 패턴을 채택

# 핵심 시스템 2

## 유연한 확장을 위한 분리형 인벤토리 시스템

### 시스템 아키텍처

#### 1 Model: CoreInventoryData & QuickSlotData - The Brain (진짜 정보)

이 아이템이 '실제로 어디에 무엇이 들어있는지'를 보관하고, 이동/교체/머지 같은 규칙을 실행하는 계층

- 책임: 슬롯별 아이템 ID 배열(또는 리스트)을 유지
- 규칙 실행
  - Move / Remove / Add / Swap
  - Merge (동일 등급 합성)
  - Sort (티어 기준 정렬)
- 기술적 특징
  - 순수 C# 클래스 : UnityEngine.UI, GameObject 같은 Unity 객체에 의존하지 X
  - IItemContainer로 접근 규격 통일 → "인벤토리든 퀵슬롯이든" 컨트롤러가 동일한 방식으로 다룰 수 있음
- Model이 절대 하지 않는 것
  - 버튼 / 드래그 같은 입력 처리 X
  - 아이콘 표시 / 텍스트 갱신 같은 화면 처리 X

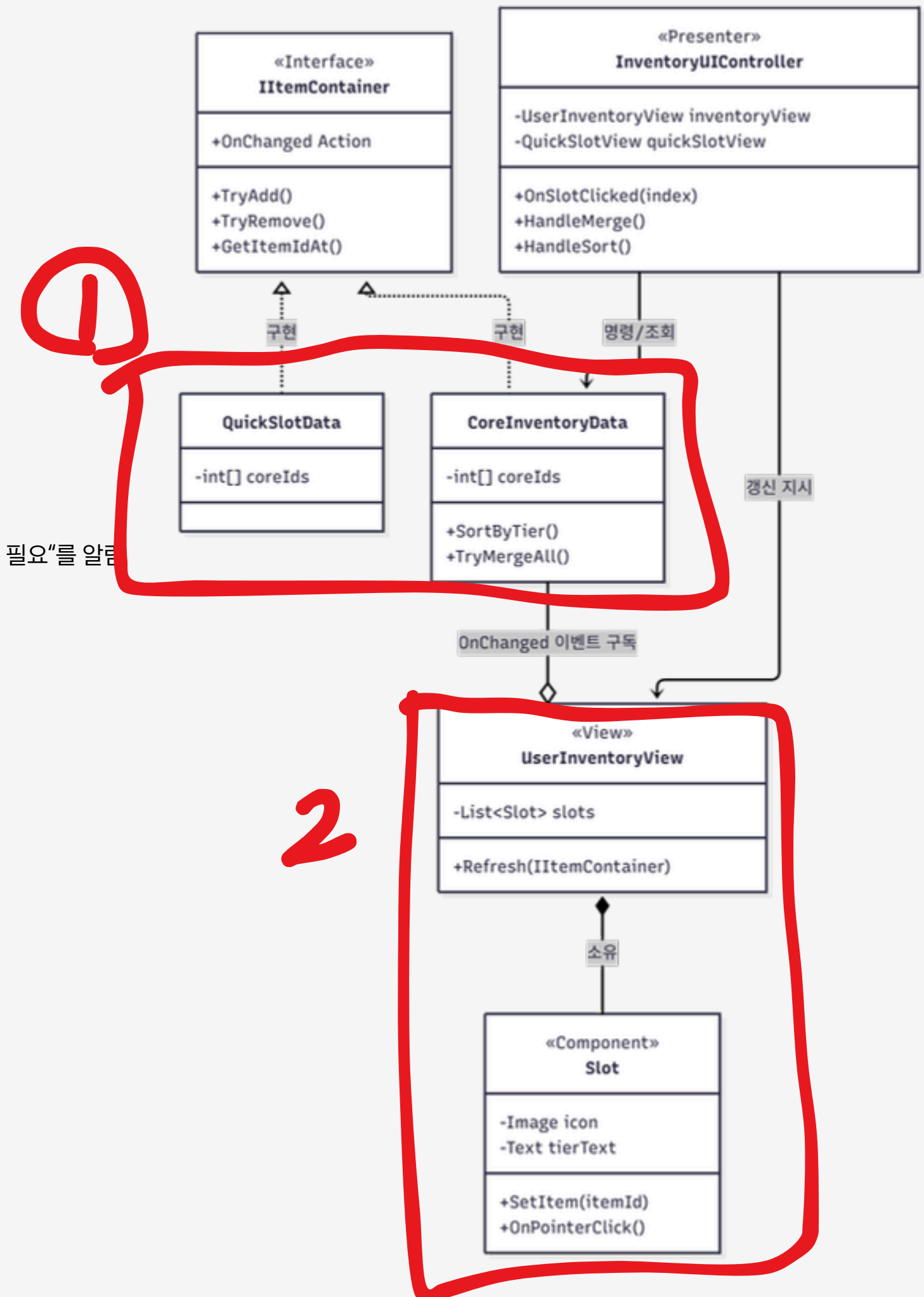
→ Model은 오로지 "데이터와 규칙"만 책임

#### 2 View: UserInventoryView & Slot — The Face(보여주는 곳)

Model의 데이터를 화면에 그려주고, 사용자의 입력을 감지해 Controller에게 전달하는 계층

- 책임
  - 슬롯 프리팹 관리 : Slot UI 생성 / 배치 / 재사용
  - 표현 갱신 : 슬롯에 아이콘 / 커서 슬롯 상태 등을 표시
  - 입력 감지 : 클릭 / 드래그 / 드롭 같은 UI 이벤트 감지
- 규칙 실행
  - View는 데이터를 직접 변경하지 않고, 읽기 / 출력만을 활용해 데이터를 조회하고 출력 및 표현
  - 입력은 즉시 Controller로 보고 (ex. 슬롯 3번에서 드래그 시작, 슬롯 7번에 드롭 시도)
- View가 절대 하지 않는 것
  - Move/Swap/Merge의 판정 로직 X
  - 합성 조건(동일 등급 여부)같은 규칙 판단 X

→ View는 "보이는 것"만 담당



# 핵심 시스템 2

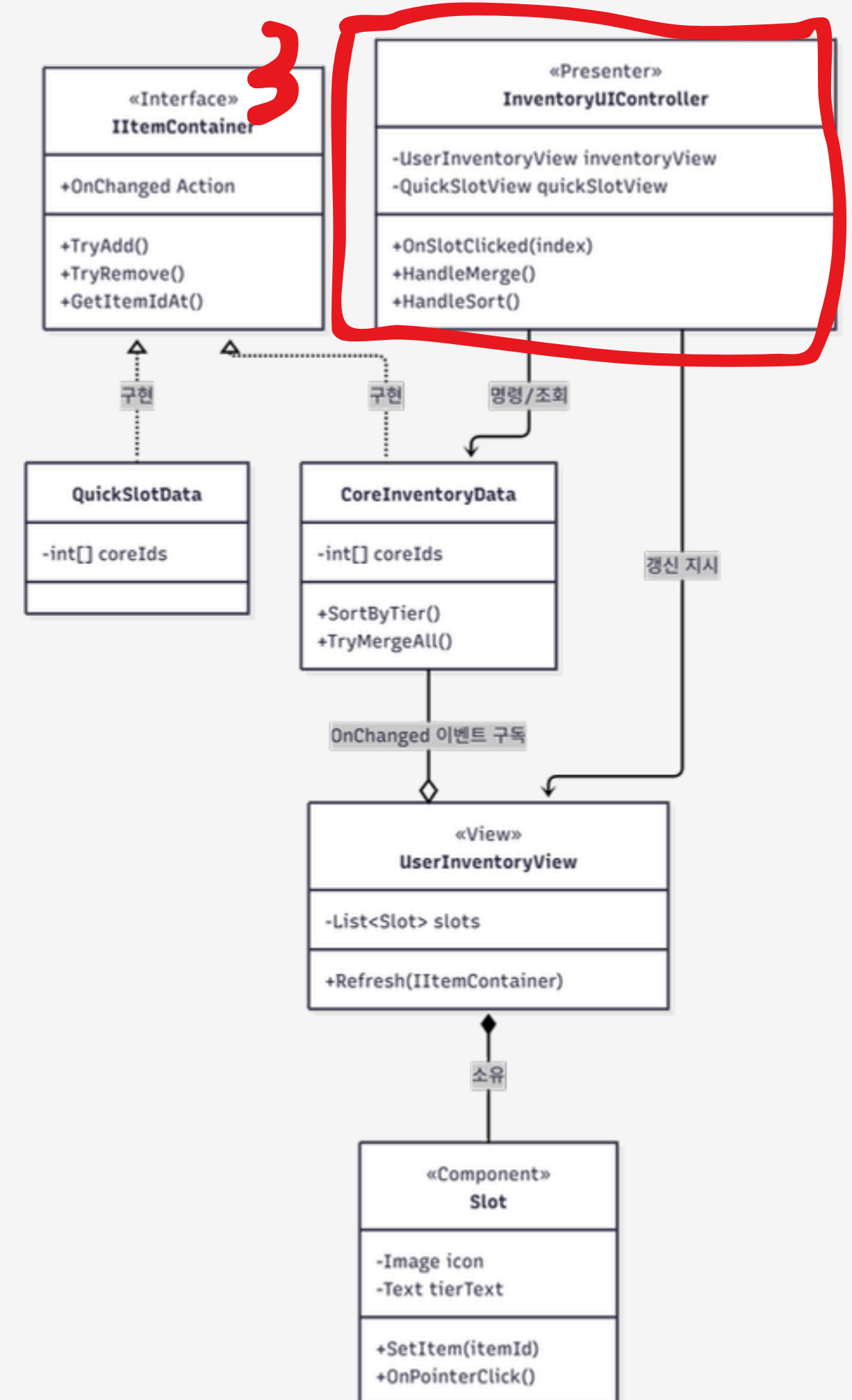
유연한 확장을 위한 분리형 인벤토리 시스템

## 시스템 아키텍처

### 3 Controller/Presenter: InventoryUIController — The Bridge(중재자)

사용자 입력을 '의미 있는 명령'으로 변환해 Model에 적용하고, 결과를 View에 반영시키는 흐름 조율자

- 책임
  - 입력 해석
    - “슬롯 A에서 B로 드롭” → 이 상황이 Move인지 Swap인지 Merge인지 판정
  - 명령 실행
    - Model의 TryMove, TrySwap, TryMerge 같은 메서드를 호출
    - TryMergeAll, SortByTier처럼 “시점이 중요한 기능”의 호출 타이밍 결정
  - 유효성 검증
    - 같은 슬롯에 드롭했는지, 빈 슬롯인지, 머지가 가능한지 등 예외 처리
  - View 갱신 트리거
    - Model의 OnChanged 이벤트를 구독하고, 변경 시 View가 최신 상태로 리프레시되도록 조율
- Controller/Presenter가 중요한 이유
  - 규칙이 View에 흩어지면 “슬롯마다 예외 처리”가 생겨 버그가 늘고 유지보수가 어려워짐
  - Controller를 통해 모든 상호작용 규칙이 한 곳에 모여 일관성이 생기고, UI 구조가 바뀌어도 핵심 로직을 지킬 수 있음



# 핵심 시스템 2

## 유연한 확장을 위한 분리형 인벤토리 시스템

### 기능 동작 흐름

#### 1) 사용자 입력 및 시스템 트리거 (Input & Trigger)

- 인벤토리 토글
  - 사용자가 I 키 또는 UI 버튼을 입력하면 InventoryUIController가 이벤트를 수신
  - 이때 단순 활성화가 아니라, SlideCoroutine 기반 슬라이딩 연출(Ease)을 통해 UI를 자연스럽게 노출
  - UI가 열린 동안 플레이어 입력 충돌을 방지하기 위해 UInputHandler를 활성화
- 합성 요청
  - 사용자가 UI의 Merge 버튼을 누르면 OnClickMergeButton()이 호출되어 "전체 합성 요청" 시작

#### 2) 명령 전달 및 유효성 검사 (Command & Validation)

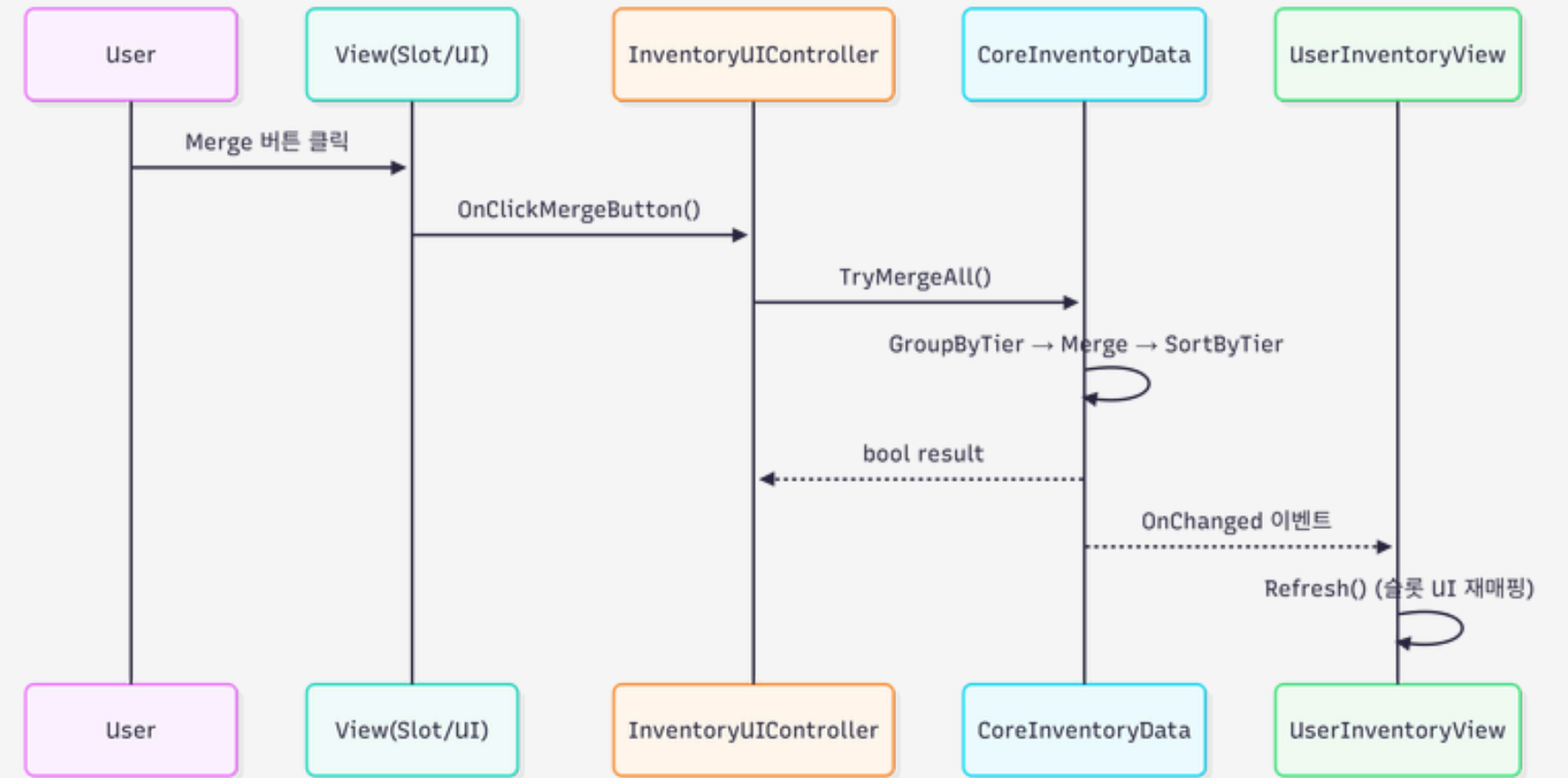
- InventoryUIController는 데이터를 직접 수정하지 않고, Managers.Instance.CoreInventory.TryMergeAll()을 호출해 합성 작업을 Model(데이터 계층)에 위임
- 컨트롤러는 TryMergeAll()의 반환값(bool)을 통해 성공/실패를 판정 → 실패 시 현재는 로그만 출력, 향후에는 사용자 피드백으로 확장 가능

#### 3) 데이터 가공 및 알고리즘 수행 (Data Processing)

- 티어별 그룹화: CoreInventoryData는 Dictionary<ECoreTier, List<int>>를 사용해 보유 코어를 등급별로 분류
- 머지 처리(ID 업그레이드): 동일 티어 코어가 2개 이상이면, 기존 슬롯을 비우고 상위 티어의 새 코어 ID를 생성/선택하여 지정 슬롯에 할당
- 정렬 강제(데이터 위치 확정): 머지 직후 SortByTier()를 호출해 티어 내림차순 → ID 오름차순 기준으로 재정렬하여, 데이터 배열의 물리적 위치를 항상 예측 가능하게 유지  
→ 머지 이후 결과 슬롯 위치가 매번 달라지면 UX와 디버깅이 흔들리므로, SortByTier로 데이터를 정규화해 항상 같은 결과를 보장

#### 4) 이벤트 전파 및 반응형 갱신 (Event Propagation & Refresh)

- 데이터 처리가 완료되면 OnChanged?.Invoke()로 변경 이벤트를 발행
- UserInventoryView는 OnEnable 시점에 이를 구독하고 있으며, 이벤트 수신 즉시 Refresh()를 수행
- Refresh()는 container.GetAllSlots()를 순회하여 데이터 슬롯과 UI 슬롯(Slot.cs)을 1:1로 재매핑(아이콘/티어 텍스트 갱신)함으로써 화면을 최신 상태로 동기화



# 핵심 시스템 2

유연한 확장을 위한 분리형 인벤토리 시스템

상세 구현

[슬롯 드래그 / 드롭의 코드 스니펫 첨부](#)

[Merge 버튼의 합성 검증의 코드 스니펫 첨부](#)



# 핵심 시스템 2

유연한 확장을 위한 분리형 인벤토리 시스템

Trade-Off와 선택

| 비교 항목 | 대안 A                     | 대안 B  | 대안 C                          |
|-------|--------------------------|------------------------------------------------------------------------------------------|-------------------------------|
|       | 직접 참조형                   | 반응형 MVP 패턴                                                                               | ECS (Entity Component System) |
| 복잡도   | 최하<br>UI에서 데이터를 직접 수정    | 중간<br>인터페이스 및 이벤트 분리                                                                     | 최고<br>데이터 중심 설계의 극단           |
| 확장성   | 낮음<br>아이템 추가 시 모든 UI를 수정 | 매우 높음<br>데이터만 추가하면 UI 자동 대응                                                              | 매우 높음<br>대량 데이터 처리에 유리        |
| 성능    | 보통<br>매 프레임 UI 갱신 필요     | 우수<br>변경 시점에만 이벤트 갱신                                                                     | 최고<br>캐시 히트율 최적화              |
| 유지보수  | 어려움<br>데이터 오염 위험이 큼      | 용이<br>SoC로 버그 추적이 쉬움                                                                     | 최고<br>러닝 커브 발생 및 디버깅이 난해      |

## [B : 인터페이스 기반의 반응형 MVP 패턴]을 채택한 이유

- 1. 기획 단계에서 코어의 종류와 합성 규칙이 변할 수 있겠다는 불확실성이 있는 상황
  - 대안 A는 구현은 빠르지만, 기획 수정 시 모든 UI를 전부 일일이 고쳐야 하는 작업 병목 현상을 우려
- 2. CoreMerge의 핵심은 “장착한 코어에 따라 스탯이 실시간으로 변하는 것”이므로, 유연한 스탯 계산이 요구됨
  - 대안 B는 QuickSlot.OnChanged 라는 이벤트 하나로 플레이어 스탯과 HUD를 동시에 갱신할 수 있어, 데이터 무결성을 보장하는 가장 합리적인 선택
- 3. 프로젝트 제약 및 R&D 비용 관리
  - 대안 C는 성능은 압도적이지만, 현 프로젝트에 도입하기에는 오버 엔지니어링의 여지가 보였으며 R&D 리스크가 크다고 판단



# 핵심 시스템 3

## 데이터 중심의 코어 시스템

### 시스템 개요

### 이해를 위한 “코어” 설명 추가? (아래 내용)

플레이어의 모든 능력치(공격력, 체력, 크리티컬 등)를 결정짓는 핵심 파츠입니다. 몬스터로부터 획득하며, 동일 티어 2개를 합성하여 상위 티어로 진화시키는 수집-합성-성장의 중심축입니다.

#### What

어떤 기능인가?

게임의 핵심 리소스인 “코어(Core)”를 ScriptableObject(SO) 데이터 단위로 정리하고 관리

각 코어는 1. 능력치 (스탯 / 효과), 2. 메타 정보 (이름 / 설명 / 티어), 3. 에셋 참조 (아이콘 / 프리팹 등)를 하나의 데이터 자산으로 보유  
런타임에서 CoreDBManager가 이를 중앙에서 로딩 / 조회해 ID 기반으로 참조할 수 있도록 구성

CoreDBManager는 코어 데이터를 ID→SO로 매핑하고, 런타임 시스템(인벤토리/스탯/드랍/UI)이 동일한 방식으로 코어 정보를 조회할 수 있도록 접근 경로를 제공

→ 코어는 “데이터로 정의되고, 게임 로직은 데이터를 조회해 동작한다”는 데이터 중심 설계(Data-Driven Design)을 적용

#### Why

왜 도입했는가?

프로젝트 진행 시 ‘기능 개발’과 ‘콘텐츠 추가’의 작업 성격을 분리해, 빠르게 콘텐츠를 확장할 수 있는 기반을 확보하기 위함

#### 1. 확장성 - 콘텐츠 추가 비용을 ‘데이터 작업’으로 전환

새 코어를 추가할 때마다 코드를 수정하는 구조는 버그 위험이 커지고 콘텐츠 추가 비용이 증가할 수 있음

코어의 정의를 SO로 외부화하면, 에디터에서 SO를 생성 / 세팅하는 것만으로 새로운 코어를 즉시 게임에 반영할 수 있음

#### 2. 메모리 효율 - 정적 데이터 공유(Reference)로 중복 낭비 최소화

전투 / 드랍 / 인벤토리 과정에서 코어 인스턴스(슬롯 / 드랍 아이템)가 다수 등장하더라도, 코어의 정적 정보는 단 하나의 SO를 참조

따라서 런타임에서 반복 생성되는 객체가 많아지더라도 “정적 데이터 복제”가 일어나지 않고 “참조”만 이루어지기 때문에 메모리 낭비를 줄이고 데이터 관리가 단순해짐

#### 3. 데이터 ↔ 로직 분리 - 저장/런타임 상태를 ID로만 유지 (무결성 강화)

코어 정의(기획 데이터)와 플레이 중 변화하는 상태(런타임 / 세이브)를 명확하게 분리

런타임에서는 인벤토리 / 퀵슬롯이 코어의 ID만 보관하고, 실제 표시 / 효과 계산은 DB에서 SO를 조회해 수행하기 때문에 다음과 같은 효과를 제공

- 저장 데이터의 크기를 최소화 (정적 정보는 저장하지 X)
- 데이터 무결성 강화 (저장 / UI가 코어 정의를 임의로 변경 불가)
- 밸런스 수정 시 (스탯 / 설명 변경) 저장 데이터에 영향 없이 즉시 반영 가능

# 핵심 시스템 3

## 데이터 중심의 코어 시스템

### 시스템 아키텍처

우리 프로젝트의 코어 시스템은 데이터(무엇인가), 로직(어떻게 되는가), 엔티티(어디에 있는가)를 엄격히 분리하여 관리하는 3계층 구조  
→ 이를 통해 수백 개의 아이템을 추가하더라도 시스템 전체의 안정성을 유지

#### ① 데이터 레이어 (The Master Data)

: 정적인 정보를 관리하는 시스템의 뿌리 단계

- 원천 데이터 (CoreDataSO)
  - ScriptableObject를 기반으로 설계
  - 개별 코어의 ID, 등급, 시각적 아이콘, 그리고 플레이어의 스탯을 변조할 수치 보너스 정보(CoreStatModifier)를 보관
- 중앙 저장소 (CoreDBManager)
  - 게임 시작 시 프로젝트 내의 모든 코어 에셋을 메모리에 로드하여 딕셔너리를 구축
  - ID로 상세 정보를 즉시 찾아주거나, 특정 등급 내에서 무작위 데이터를 추출하는 핵심 검색 인터페이스를 제공

#### ② 로직 레이어 (The Business Service)

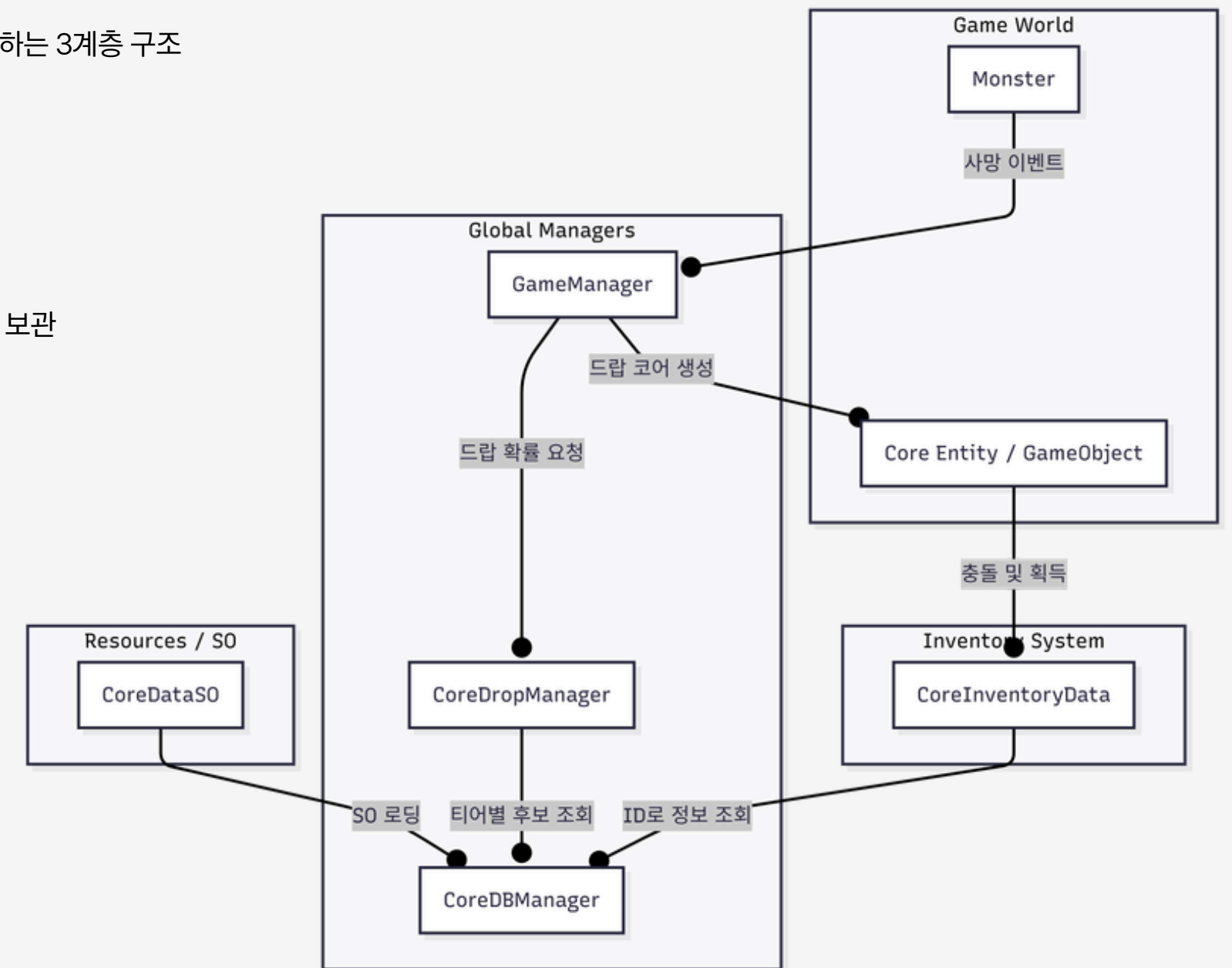
: 데이터를 가공하고 게임의 규칙을 집행하는 두뇌 단계

- 드랍 연산 서비스 (CoreDropManager)
  - 몬스터의 태생적 등급(일반/에픽/보스)에 따라 아이템이 생성될지 여부와 어떤 가중치를 적용할지 결정
  - 데이터 테이블 형식의 설정을 기반으로 런타임 확률을 계산하여 최종적인 드랍 품목을 산출
- 스폰 제어 서비스 (GameManager)
  - 연산 서비스에서 결정된 결과물을 실제 게임 월드에 물리적으로 구현하는 '생산 공장'
  - 프리팹 리소스를 관리하며, 월드 내 특정 좌표에 코어 개체를 안전하게 배치하고 초기화 명령을 내림
  -

#### ③ 엔티티 레이어 (The Physical Instance)

: 플레이어와 상호작용하는 물리적 실체 단계

- 월드 오브젝트 (Core.cs)
  - 필드에 드랍되어 떠다니는 실제 물리 개체
  - 스스로 복잡한 정보 (데이터 전체)를 담지않고, 자신의 정체성인 ID만을 기억
- 생성되는 순간 중앙 저장소에서 자신의 ID에 맞는 외형 정보를 빌려와 입히며, 플레이어와 충돌하는 찰나에 자신이 품고 있던 ID를 인벤토리 시스템으로 전송, 스스로를 파괴



# 핵심 시스템 3

## 데이터 중심의 코어 시스템

### 기능 동작 흐름

#### ◆ 1) 초기화(Init)

- 게임 시작 시 CoreDBManager가 프로젝트에 등록된 CoreDataSO들을 로딩
- ID → CoreDataSO 조회가 가능한 데이터 사전을 구성
- 모든 시스템은 개별 데이터 파일을 직접 읽지 않고, 이 '중앙 창고'를 통해서만 정적 정보에 접근

#### ◆ 2) 몬스터 처치(Drop Trigger)

- 몬스터가 사망하면 GameManager가 해당 이벤트를 수신하고 몬스터의 등급 정보 추출
- 드랍 처리 책임을 CoreDropManager에 위임

#### ◆ 3) 드랍 계산(Calculate)

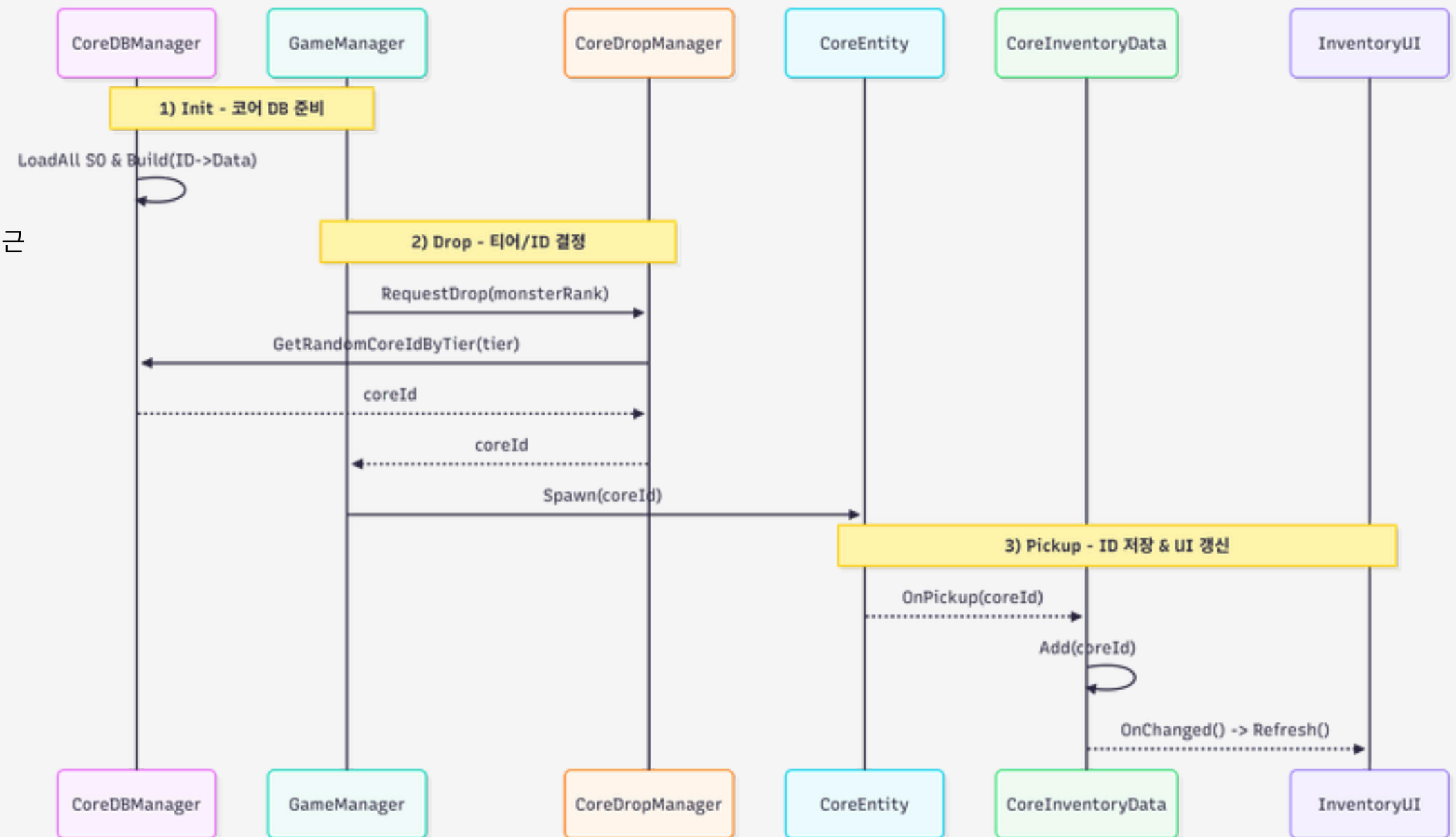
- CoreDropManager는 몬스터의 등급 및 드랍 가중치 테이블을 기반으로 드랍 티어를 확정
- 티어가 결정되면 CoreDBManager에 해당 티어의 코어 후보군을 요청
- 그 중 하나를 랜덤으로 선택해 드랍 ID를 결정
- 이때 "무슨 코어를 드랍할지"는 데이터 기반으로 결정되고, 월드 오브젝트는 ID만으로 생성

#### ◆ 4) 스폰(Spawn)

- 결정된 코어 ID를 포함한 Core를 몬스터 사망 위치에 생성
- Core는 충돌 시 전달할 코어 ID만 보유하고, 코어의 상세 정보는 필요할 때 DB에서 조회

#### ◆ 5) 습득(Pickup)

- 플레이어가 Core와 충돌하면 오브젝트 소멸, CoreInventoryData에 해당 코어 ID가 추가
- 인벤토리 데이터가 변경되면 OnChanged 이벤트가 발행  
→ InventoryUIController / View가 이를 구독해 UI를 즉시 갱신
- 결과적으로 "코어 획득"이 데이터 변경 → 이벤트 전파 → UI 반영으로 안정적으로 연결



# 핵심 시스템 3

## 데이터 중심의 코어 시스템

### 상세 구현

#### ① 효율적인 데이터 검색 (ID-Based Caching)

CoreDBManager는 부팅 시 모든 코어 데이터를 로드한 뒤, 두 가지 형태의 Dictionary로 캐싱

- Dictionary<int, CoreData> coreDataById: ID를 통한 즉각적인 정보 조회용
- Dictionary<ETier, List<CoreData>> coreDataByTier: 특정 티어(예: Epic) 내에서 랜덤한 코어를 뽑아야 하는 드랍 시스템용

#### ② 런타임 최적화 (Resources.LoadAll)

- 매번 파일을 읽지 않고, Managers.OnInitialize 단계에서 Resources.LoadAll<CoreDataSO>("SO/Core")를 실행
- 게임 플레이 도중 아이템 획득 시 발생할 수 있는 I/O 병목 현상(끊김)을 사전에 방지

#### ③ 데이터 중심의 월드 개체 (Core.cs)

- 필드에 생성되는 Core 프리팹은 빈 껍데기에 가깝고, 생성 직후 Init(coreId)를 통해 CoreDB로부터 정보를 받아와 자신의 스프라이트와 데이터를 갱신해 실체화
- 따라서 수백 종류의 코어가 있어도 프리팹은 오직 '하나'만 관리하면 됨

### 각 코드 스니펫 추가

# 핵심 시스템 3

## 데이터 중심의 코어 시스템

### Trade-Off와 선택

#### 1) 데이터 저장 방식: ScriptableObject vs 외부 데이터(JSON/Excel)

##### \* 대안 A: 외부 데이터 (JSON/CSV)

- \* 장점: 유니티 에디터 없이도 수치 수정이 가능하며, 모딩(Modding)에 유리함
- \* 단점: 런타임에서 데이터를 읽어와 객체에 할당하는 직렬화/역직렬화 오버헤드가 발생함

##### \* 대안 B: ScriptableObject

- \* 선택 이유:
  - 에디터 내에서 인스펙터로 아이콘을 바로 확인하고, CoreStatModifier 수치를 직관적으로 수정할 수 있는 방법을 선택
  - 또 유니티의 네이티브 메모리 관리 기능을 활용해 수백 개의 데이터를 가장 효율적으로 로드할 수 있다고 판단

#### 2) 드랍 로직 위치: 몬스터 개별 구현 vs 중앙 집중형 매니저

##### \* 대안 A: 각 몬스터 스크립트(Monster.cs)에서 드랍 처리

- \* 장점: 특정 몬스터에게만 특수한 드랍 규칙을 적용하기 직관적임
- \* 단점: 전역 드랍률 수정이나 이벤트(예: 드랍률 2배) 구현 시 모든 몬스터 코드를 수정해야 함

##### \* 대안 B: 중앙 집중형 서비스(CoreDropManager)

- \* 선택 이유:
  - '관심사의 분리(SoC)'를 실현하는 방향의 설계를 모색
  - 몬스터는 "나는 죽었다"는 사실만 알리도록, 그리고 "무엇을 줄 것인가"는 코어 드랍의 밸런스를 관리하는 CoreDropManager가 결정하도록 선택
  - 이러한 설계로 Monster를 구성하는 코드를 건드리지 않고도 전체 게임의 드랍 밸런스를 실시간으로 제어할 수 있는 유연성을 확보